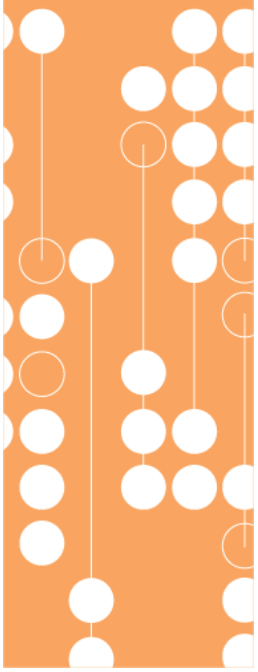
Programming Foundations

Learner Guide



QA
Learn. To Change.

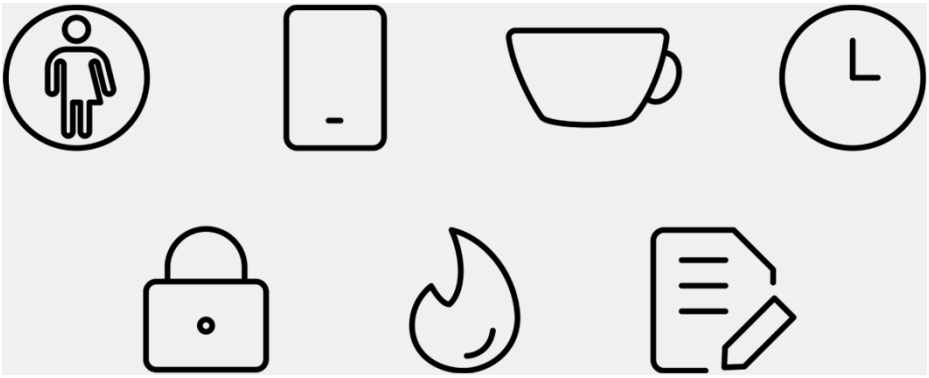
Programming Foundations

Outline



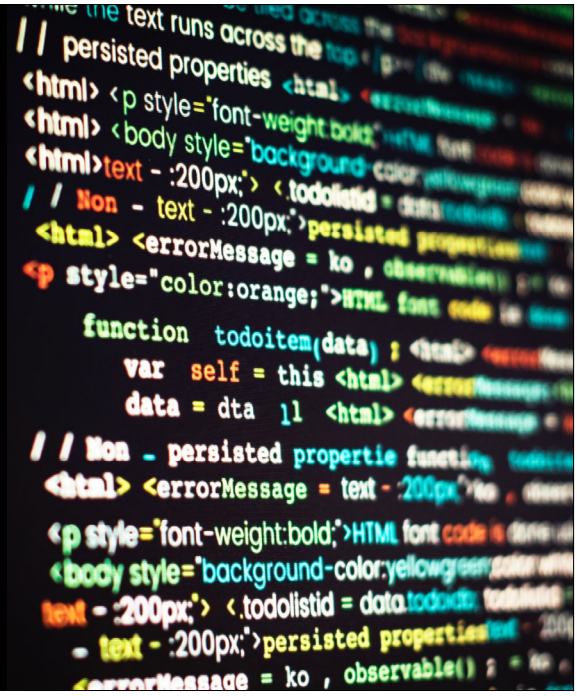
QA
Learn. To Change.

Housekeeping



Understanding Computers

Programming Foundations



QA
Learn. To Change.

Overview

- What are computers for?
- Information
- The language of machines
- Computer components
- What do CPUs do?
- Operating System 101
- Programs and processes

What are computers for?

Surfing the Internet?

Hosting the Internet?

Managing our entire existence?

QA

Maybe the first question to ask on a computer programming course is: what are computers actually for?

The answer is actually quite scary when you think about what computers actually do for us. They are so integrated into nearly every aspect of our lives it's tempting to say everything.

Information and processes

- Computers act as big, powerful calculators.
  - Data processing was and is a vital role
  - Millions of calculations per second
- Computers can also implement complex processes.
  - We can write programs that control behavior and implement processes
- Computers can communicate...
  - With each other
  - With other machines
- Computers can learn.
  - Scary?

QA

The first computational devices were simply adding devices the simplest of which was the Abacus. Many variations of mechanical adding machines later led to the concept of the first programmable computer known as the "difference engine" by Charles Babbage.

Computers have come a long ways since then and we have moved from mechanical devices to electronic ones that can perform millions of calculations per second. They can execute complex sequences of programmed instructions implementing logic-based processes we define. Linking computers and machines together has led to the internet, automated robotic car factories, the iPhone and the Apple Watch. With machine learning technology computers are developing the ability to learn.

The future is both exciting and unclear.

The language of machines

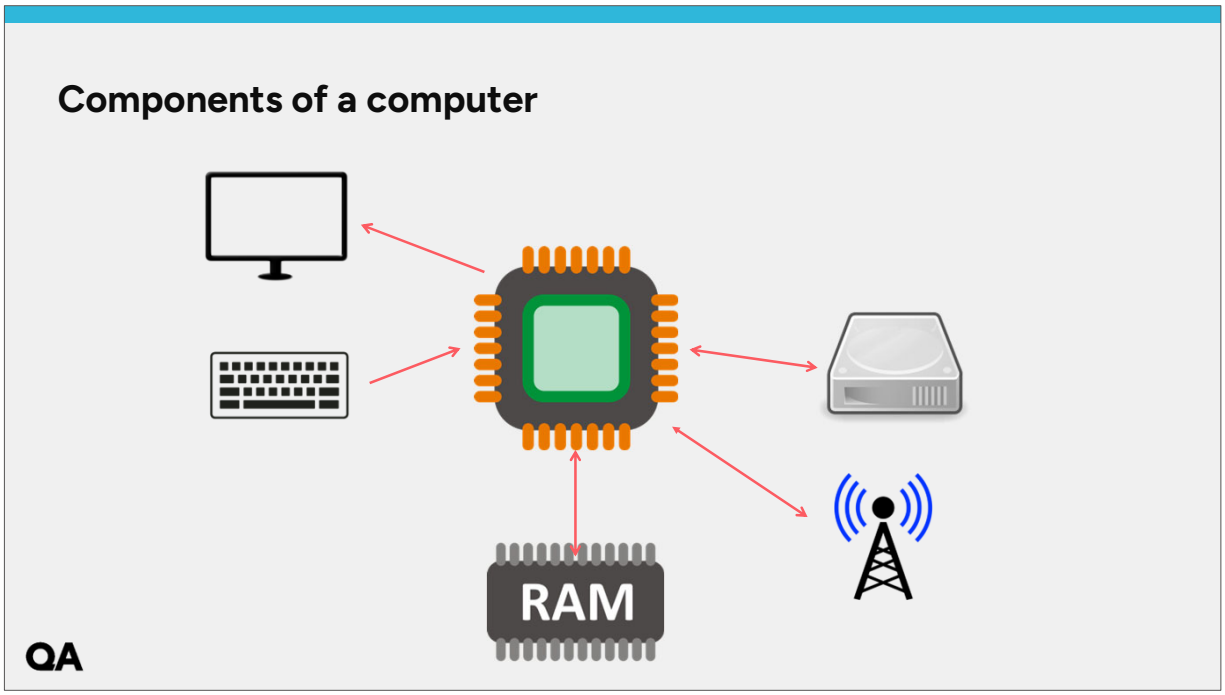
- What language do computers speak?
  - Binary
  - 1s and 0s
- In electronics, 1 and 0 mean on or off.
  - Can actually mean higher voltage or lower voltage
- Does on or off sound a lot like a switch?
  - Computers contain millions of tiny switches called transistors





QA

At the moment computers understand one thing the "bit". A bit can be thought of as a 1 or a 0 or even On or Off. On or Off makes sense because we are dealing with electronic circuits and most electronic circuits have switches of some sort.



Can you identify the main components of a computer?

Components explained

• What does the CPU do?	• The "brain" of the computer, executes instructions
• What does RAM mean?	• Random Access Memory
• What is persistent storage?	• Stores data and computer programs • Hard drive, USB drives, CD
• Why do we need persistent storage?	• RAM is volatile, save for later!
• What about Input/Output Peripherals?	• Keyboard, Mouse, VDU, etc.
• Did we miss anything?	• Where would you be without Wi-Fi and Networking?

QA

The CPU is the brain of the computer. It executes the instructions contained within programs and coordinates the transfer of data between the other components including the RAM and the other peripheral devices.

Why do we need RAM and Persistent Storage? The answer is because RAM is volatile. That means if you switch off your computer you lose your memory (on the computer obviously!).

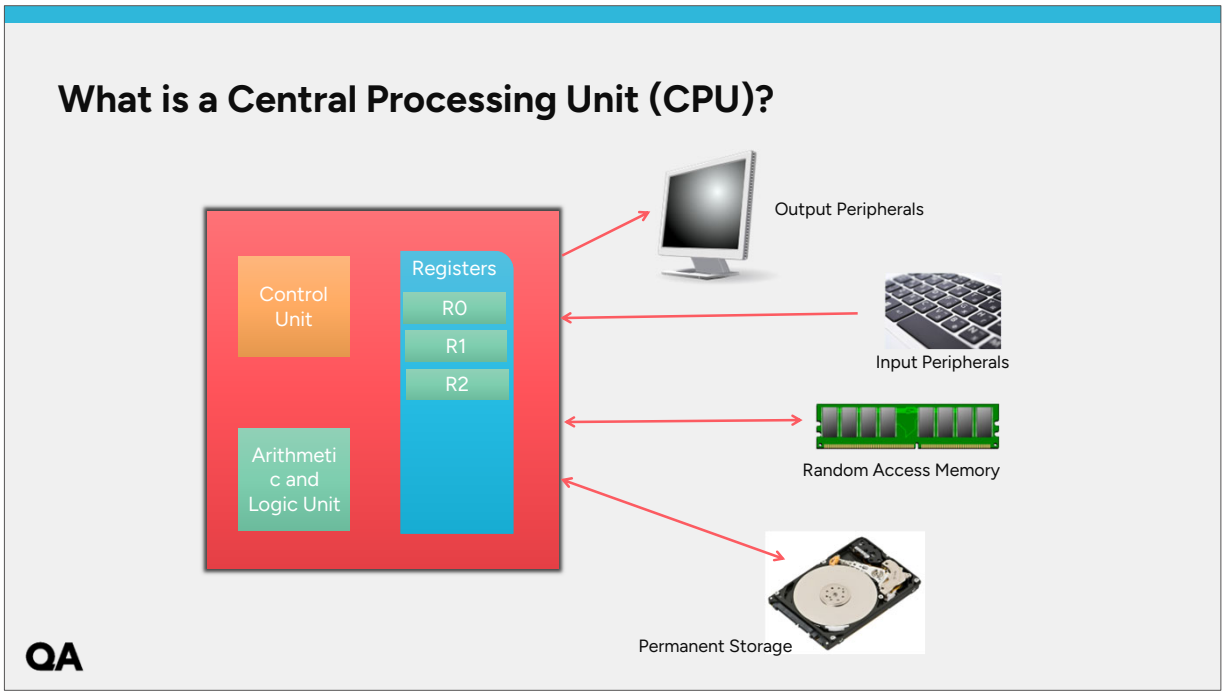
What do CPUs do?

- Keep track of program execution.
- Execute computer program instructions.
  - Native Computer Programs contain instructions in binary format
  - Binary instructions talk to the CPU
- Store data.
- Transform data.
- Store data in RAM.
- Retrieve data from RAM.
- Send and receive data to and from hardware devices.



QA

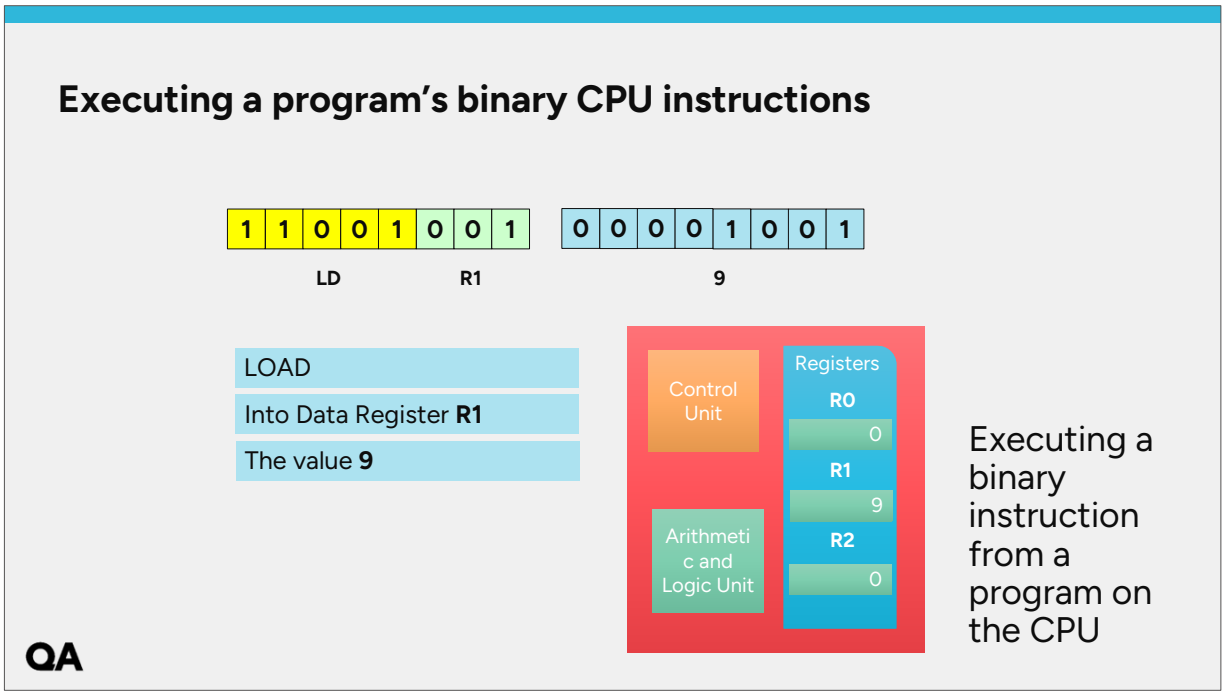
Native Computer programs (ones designed to run on a specific platform) contain instructions in binary format. These instructions talk directly to the CPU. They tell the CPU to perform actions like ADD, LOAD, MOVE etc. Data can be loaded into internal storage registers where calculations can be performed with the results moved out again into memory.



The CPU uses the Arithmetic and Logic Unit to perform logic and maths operations and the Control Unit to manage the flow of data in and out of the processor.

CPU instructions

- CPUs can execute instructions.
  - Many different kinds of instructions, usually involving data
- Arithmetic and Logic Unit (ALU) performs:
  - Add, subtract, compare, etc.
  - Logic operations such as And, Or, NAnd, Xor
- Control Unit does:
  - Data dispatch and fetch operations
  - Instruction Execution Control – Jumping to different execution points
- Registers:
  - Act as temporary storage for performing operations on data
  - A bit like M+ on a calculator



Above you can see a single binary instruction. The first 8 bits is made up of two parts.

110011 tells the CPU to perform a LOAD operation.

001 indicates Register 1

1001 is data value 9

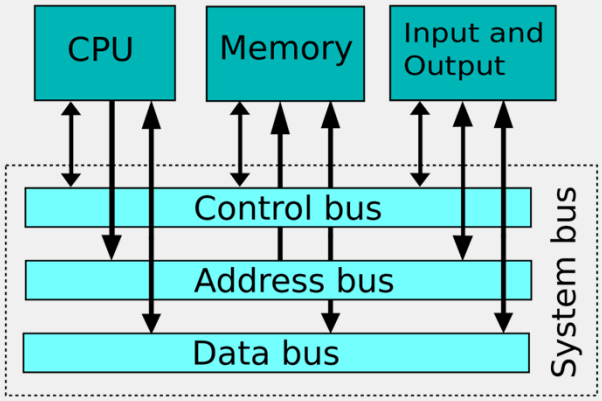
So Load into Data Register R1 value 9

Or its assembly language equivalent might be:

LD R1, 9

The Bus

- CPU uses a device called a Bus to talk to other components.
  - Allows data to flow between components and the CPU in a controlled way



QA

32

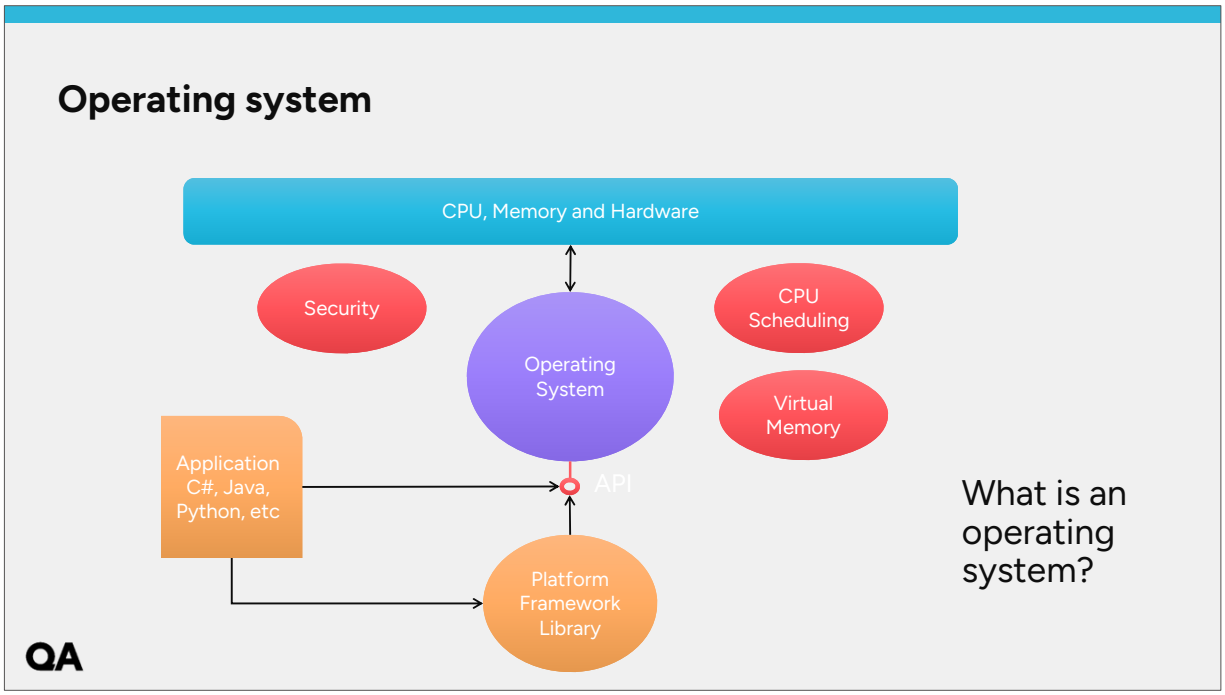
To move data from the CPU to memory and other hardware locations, such as peripherals, we use the system BUS.

The system bus combines the functions of the **data** bus, the **address** bus and the **control** bus.

The data bus carries the information (data).

The address bus determines where the data should be sent.

The control bus determines its operation. It carries commands from the CPU and returns status signals from the devices.



The operating system is itself a program that provides an environment in which other programs can safely execute. It provides a buffer between the hardware and the application software. Applications software can never directly access hardware devices or each other's memory.

The OS also provides sets of services that include CPU Scheduling, Memory Management and Security Management (usually encompassing an authentication and resource authorisation mechanism).

Operating Systems (OS)

- Interface between hardware and applications.
- Provide an environment in which programs can run:
  - In a convenient manner
  - In an efficient manner
- Prevent applications from damaging each other.
  - Security
- Provide services to make programming easier.
  - Application Programming Interfaces (APIs)
- The core of the OS is known as the **kernel** or **executive**.
- Examples:
  - Microsoft Windows / Android / Linux / OS X / iOS



Quite early on in computer system development it was realised that coding low-level operations required a certain skill and specialist knowledge. It also became obvious that every program required certain services, like, for example, initialising a device. Commonly used routines giving such functionality were shared between programmers, and eventually these became bundled into services. Now a major function of an operating system is to provide applications with programming interfaces (APIs) to these services, enabling the programmer access to low-level features, without the labour-intensive effort.

It was also obvious early on that the hardware resource (the expensive part) was under-utilised when only running one program. Sharing resources such as the CPU and memory became necessary.

CPU scheduling

- Single CPU computers can only run one instruction at a time.
- How can multiple programs run on one machine?
  - Answer: Multitasking.
- Multitasking operating systems:
  - Allow multiple programs to run apparently at the same time
- Time slice:
  - Each program is given a slice of time to execute on the CPU

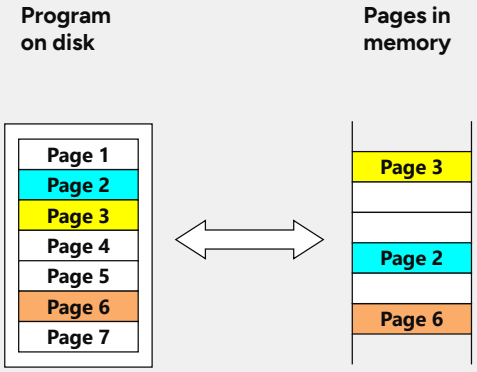


Conventional computer design only allowed one set of operations to be executed at once, so while a program waited for data to be transmitted by a peripheral, the CPU and memory were unused. Enter ***time-sharing***, a concept where a "Master Program" (operating system) allocates hardware resources to other waiting programs (applications).

Switching between programs requires some co-operation from the hardware. Early operating systems could only switch between programs when one of them voluntarily gave up the processor, for example to access a peripheral. This required programs to co-operate, and so is called co-operative multitasking. Obviously one "CPU hog" could hold up the entire system, so pre-emptive multitasking was introduced. This is where a program is allocated an amount of time (for example 20ms) to execute code, then the hardware generates an interrupt which wakes up the operating system. The values which the program was working on (known as its context) are stored, so that when its turn comes again it can carry on as if nothing happened.

Virtual memory

- The operating system saves portions of RAM to disk:
  - To use memory for something else
  - Read in again when required (swapping)
- Program and data is split into small portions.
  - Fixed size "pages" (4kb and 16kb are typical)
  - Only those pages required are read into memory
  - "Loading" a program may only use a few pages of RAM



As we have seen, a process spends much of its time in a wait state, either waiting its turn for the processor or for some device transfer to complete. Storing the program and data in RAM would be extremely wasteful, and it would not be long before we ran out of memory. The operating system therefore has a policy of saving portions of RAM onto disk while they are not being used, and reading them in again when required. This process is known as swapping, and the disk areas used are called swap files (or swap space).

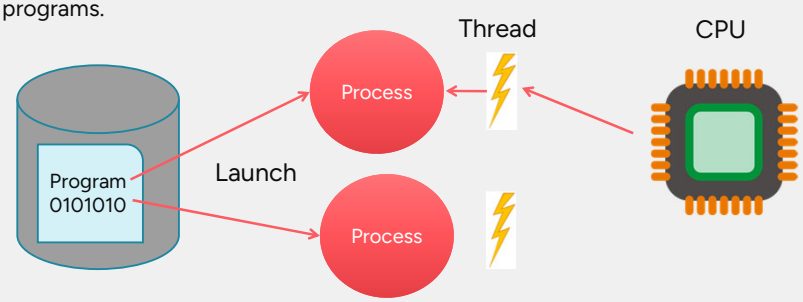
If we swapped the entire program and its data in and out of memory each time the process went into a wait state (every few milliseconds) the system would be very slow. The process is actually split into small slices known as pages, and it is these that are loaded into RAM. Pages are swapped, so this process is known as paging, and on some operating systems swap files are called page files (e.g. Windows uses pagefile.sys).

All this is done transparently to the program – it has no idea where any of its pages are at any moment in time. So how can the program find anything? Every item in a process has a virtual address, which consists of a page number and its position within the page. The operating system uses this to cross-reference its own page tables to find the required page, and the item within it.

Of course it could be the case that only a few processes are running, and we actually have enough memory. Pages are not always swapped out when the CPU is lost, operating systems use various algorithms to work out which pages should be swapped (usually based on use). High privilege processes can even lock pages into memory (with care!).

Programs and processes

- What is a program?
  - A file containing instructions and data
- How do you run a program?
  - From the OS User Interface (if it has one)
  - From a Command Line Interface
- Processes loaded from programs.



QA

A program is a file containing instructions and data. It can be launched from the OS User Interface, if it has one, or from its command interface.

When a program executes the OS creates (or clones) an instance process to manage execution and schedules a thread for the execution of the program instructions. Additional execution threads can subsequently be scheduled from this main thread.

Multiple instances of a process can be create from a single program. Think about it, you can load more than one instance of Word.

Creating computer programs

- Programmers write apps in a human-readable language.
  - C#, Java, Python, C, C++, JavaScript, etc.
- Source code will need to be prepared for execution.
  - Compilation-based languages (C, C++)
    - Compiles and links source to CPU binary instructions, producing an executable program.
  - Interpreted languages (Python, JavaScript)
    - An interpreter application interprets and executes the source code at the point of execution.
    - Source code executes on platform-independent interpreters.
  - Virtual machine languages (C#, Java)
    - Compile to Byte code (not CPU-specific).
    - Execute within a virtual machine.
    - Virtual machine is platform-specific, not the application.



You could write a program in binary but that would be tedious. Assembler would be better but still tedious so we usually pick a higher level language like C or Java or maybe even Python.

Types of computer programs

- Console applications
  - Character-based, low resource.
- Graphical User Interfaces (GUI)
  - Use Windows/Forms to present data and retrieve data from user.
- Web applications
  - Applications are presented in a browser.
  - Use HTML, JavaScript, and CSS.
- Service applications
  - Don't usually present a User Interface.
  - Provide services to other applications.
- Mobile applications
  - Applications written specifically for mobile devices.



A first program

- Python programming language
  - Hello World

```
print("Hello World")
```

  - Hello World++

```
name = input('What is your name?')
print('Hello ' + name)
```

QA

Here is a first taste of a computer program written in Python.

Python does not distinguish between single and double quotes and treats them the same. Some languages, such as PHP, interpret single and double-quotes differently.

Question

- Data is distributed from the CPU to the other system components via:
  - A. BUS
  - B. RAM
  - C. Hard drive



Question

- A program is an executing instance of a process.
  - A. False
  - B. True

QA

It's the other way around a Process is an instance of an executing program.

Summary

Computers involve multiple components:

- CPU, Memory, Persistent storage, Network Interface, User IO, BUS

Software components:

- Operating system, Applications, Tools and utilities

Programs run as processes.

- Share system resources
- Are not necessarily resident in RAM all the time
- Programs access Virtual Memory

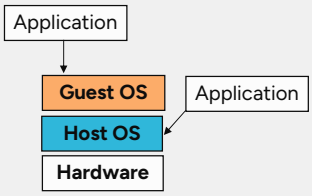
Applications frequently call other programs.

- Using an Application Programming Interface (API)

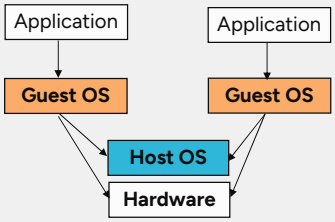
QA

Virtualization

- The running of one operating system from another.
  - Windows on UNIX, Linux on Windows, and so on
- Can be slow, but saves having numerous machines.
- Several designs are possible.



Applications run on the Guest or Host



Applications run on a Guest, which may have direct access to the hardware

QA

Virtualisation is a general term given to running a program in an alien environment. It has become popular as machines have become powerful enough to support it.

Usually virtualisation means hosting an entire operating system, often on a completely different one. Applications running on this "guest" operating system are unaware that they are running in any special way. Some systems require all hardware service requests to go through the Host operating system, where kernel calls have to be translated. Others allow Guests to access the hardware directly, which is much faster.

Hardware has developed recently to assist in virtualisation, and the most well known is Intel's Hardware Assisted Virtualization (VT) support, but AMD have a similar technology known as AMD-V.

Virtualization

- Virtualization has many uses:
  - Temporary environment while moving from one OS to another
  - To run non-portable programs
  - For application testing in multiple environments
  - To make best use of "big-iron" supercomputers
  - For training courses!
- Some virtualization products:
  - KVM — Open Source for Linux, but hosts other OSs
  - VirtualBox — from Oracle, runs on Windows, Linux, Mac, Sun
  - Hyper-V — from Microsoft, for Windows but hosts other OSs
  - VMWare — several versions, including those without a host
  - Xen — Open Source, from Citrix

QA

Virtualisation has made running different operating systems much easier. Previously one machine was required for each operating system, or elaborate dual-boot options were required. Nowadays it is quite common to run Linux on Windows, or different versions of Windows on the same machine.

Packaging an environment is much easier using a virtual machine. Rather than expecting numerous extra add-ons to be installed for a specific environment, which is error prone, we can just install the whole lot in one go. This is used, for example, by Nokia when offering a test environment for Open Source developers. QA uses virtual machines for teaching Linux, which is particularly useful in a third-party training centre which might not appreciate their Windows systems being overwritten by a Linux installation.

You might wonder why Microsoft appear to have two products, Virtual PC and Hyper-V, that do the same thing. The chief difference is that Virtual PC is a stand-alone product (like the others), whereas Hyper-V is part of the host operating system. If you run Hyper-V you can't run any other virtualisation products on the same system. Virtual PC is not offered on Windows 8 and is being phased out in preference to Hyper-V. Microsoft also had Virtual Server, which has been dropped and replaced by Hyper-V.

There are many more virtualisation products available than just those shown on the slide. Sun Microsystems (Oracle) produce their own, for example. All of the example products shown on the slide (and the Sun versions) can use Hardware Assisted Virtualization (Intel-VT or AMD-V).

Understanding Data

Programming Foundations

QA
Learn. To Change.

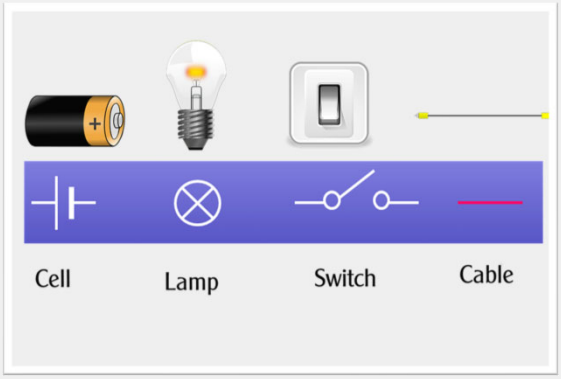


Overview

- Understanding numbers
- Decimals and binary
- Bits, bytes and words
- Switches and transistors
- Bits and memory
- Storing data
- Performing calculations

Understanding numbers

- How can an electric circuit understand a number?
  - Such as the number 12
- Circuits are made up of components such as switches, cells, and lamps.



QA

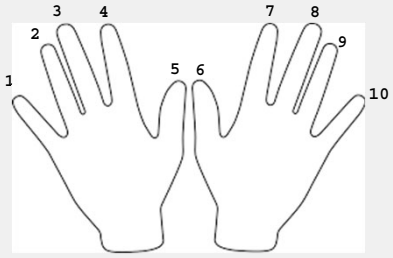
8

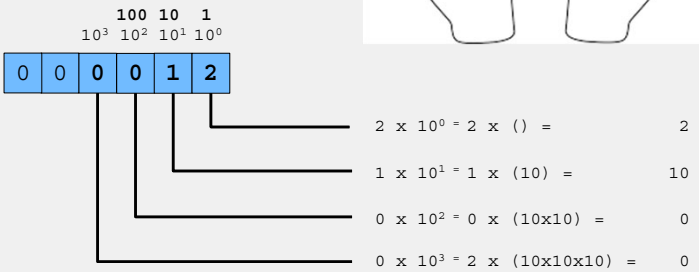
In the last chapter we looked at how computers were originally designed to be big calculating engines.

To perform calculations computers need to understand numbers. The question is how do we get an electronic circuit to understand the concept of a number?

Decimal numbers

- Why do we count in 10s?
- Each finger represents:
  - 1 to 9
  - Tenth finger – counts the number of tens



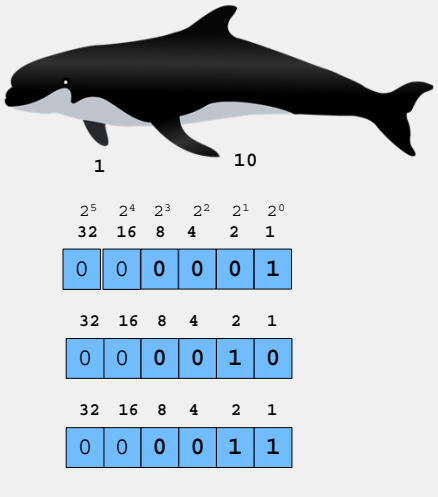


QA

With ten fingers we count to ten and then count the number of tens, hundreds, thousands etc.

Binary

- If whales could count, they would use a binary numbering system.



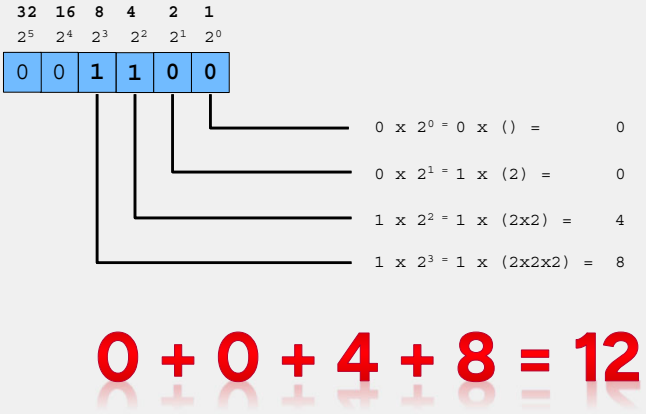
QA

8

Seems like a surreal point but if whales could count (and maybe they can) and they followed a similar pattern to humans, they would use a binary numbering system because they have two flippers.

Binary numbers

- We can use binary to represent numbers.



0 + 0 + 4 + 8 = 12

QA

8

In binary we count the number of ones, twos, fours, eights etc.

Collecting bits together to store numbers

- One binary number is called a bit, and can store a max value of 1.
- Two bits together can hold a maximum value of 3.

Bits	Max Binary Number	Decimal Equivalent	Number of States (Including Zero)
1	1	1	2
2	11	3	4
3	111	7	8
4	1111	15	16
5	11111	31	32
6	111111	63	64
7	1111111	127	128
8	11111111	255	256

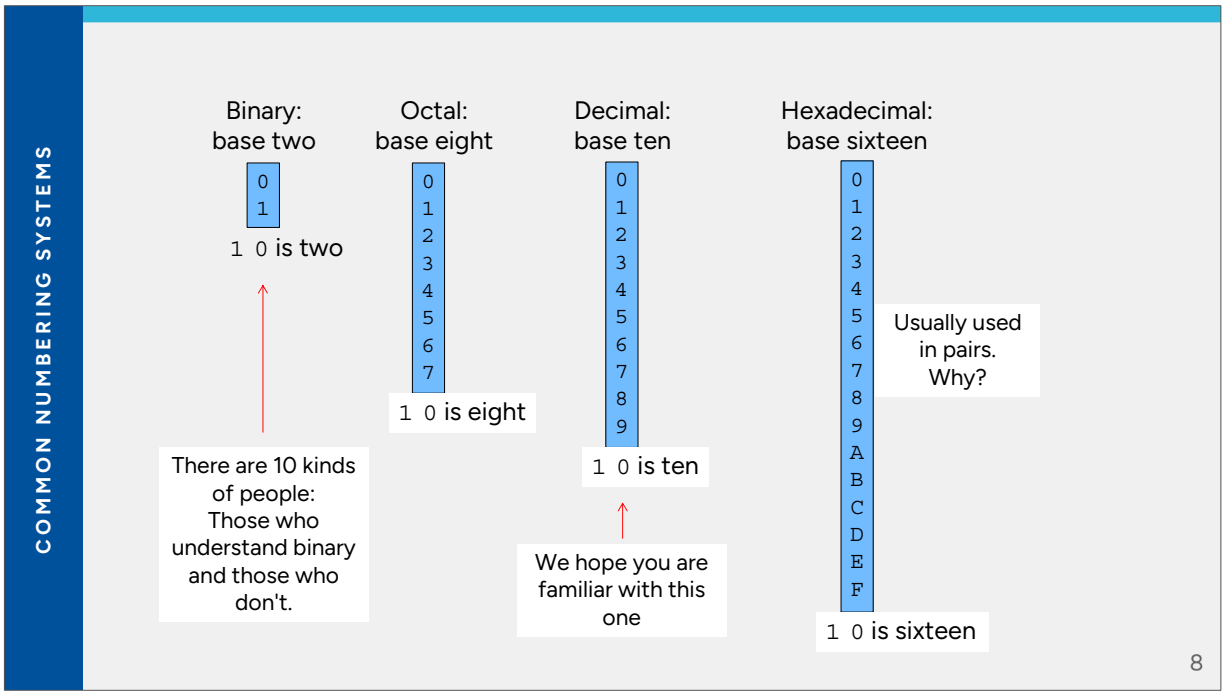
2^{10}

- 8 bits can store a maximum number of 255.
  - 256 states including 0 = Byte

QA

8

Using binary we can represent any whole number as long as we have a sufficient number of bits to store that number.



The common numbering systems used in programming are shown. It should be made clear that we rarely need to convert numbers between these systems. We usually only express a number in it's base. So we don't usually have to convert a hexadecimal number to decimal. *Usually not*, not *never*.

The simplest, to a computer at least, is binary. Just two values, 0 or 1, True or False, Red or Blue, Up or Down, Ying or Yang, express it how you like. As a programmer it is unlikely that you will need to construct values in binary, but you might on occasion need to be able to read binary numbers, depending on the type of programming you are doing.

Octal is not seen much these days. The most common place is in UNIX/Linux file permissions. Three binary digits are expressed for read-write-execute permissions. As you might know, 111 in binary is seven. It makes sense to use octal because 7 is the largest number which can be expressed in three bits.

Hexadecimal is very commonly seen. As an example, the maximum memory address in 4 Gigabytes of RAM expressed as a base 10 number is 4,294,967,295, but in hexadecimal is 0xFFFFFFFF (8 x F's). Hexadecimal is commonly used to represent bits.

BYTES, WORDS AND ALL THAT STUFF

Classifying collections of bits

Bits	Number of Bits	Biggest Number
Nibble	4	15
Byte	8	255
Word (16 bit system)	16	65535
Word (32 bit system)	32	2^{32}-1
Word (64 bit system)	64	2^{64}-1

8

Bits are typically organised into units with the byte representing 8 bits and a word which is usually the unit of processing in a system. Words usually refer to the data size that can be manipulated and exchanged by the CPU and its BUS.

STORAGE OF BITS

Classification	Abbrev	Number of Bytes	
Kilobyte	K	1024B	2^{10}
Megabyte	MB	1024K	2^{20}
Gigabyte	GB	1024MB	2^{30}
Terabyte	TB	1024GB	2^{40}
Petabyte	PB	1024TB	2^{50}
Exabyte	EB	1024PB	2^{60}
Zettabyte	ZB	1024EB	2^{70}
Yottabyte	YB	1024ZB	2^{80}

8

The bit is the smallest unit of data. To describe large amounts of data usage we classify it in terms of bit groupings from Kilobytes through to Yottabytes.

Binary and switches

- Switches can be either:
  - On or Off
- Binary represents either:
  - 1 (On) 
  - 0 (Off) 
- Switches can be used to represent binary values

QA 8

It's now time to make the leap from binary maths across into electronics because it's the humble switch that gives us the ability to store a binary value in an electronic circuit.

Because a bit can either be 1 or 0 that correlates perfectly with a switch's states of On and Off.

We can use switches to store binary states and thus binary numbers.

BITS AND SWITCHES

- One switch can be used to store 1 data bit.
  - Max value 1
- Four switches can store 4 data bits allowing a data value.
  - 0 to 15 to be stored



15

- Eight switches give 8 bits and a range of 0 to 255.
  - 8 bits is one byte



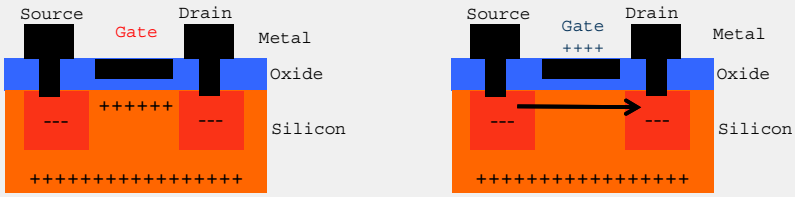
255

8

Four switches together all switched on represent the binary number 15.

Switches and Transistors

- Invention of the solid-state transistor paved the way for modern computers.
  - Bardeen, Brattain, and Shockley, 1947
  - Electrically controlled switches/semiconductors
- Field Effect Transistors (FET) use silicon and metal oxide.
  - Creating semiconductors



- Microprocessor made up of millions of tiny transistors.
- Each one can be on or off.

QA

8

In truth CPUs don't use simple switches they use Field Effect transistors organised into logic gates and memory cells.

So...

- Electric circuits can use switches to store binary values.
- Transistors can be used as very small switches.
- Millions of transistors can be miniaturized and integrated into micro-processors.
- Numbers can be stored in micro-processors using binary patterns organized into bytes.



Bits and memory

- Bits are stored in Random Access Memory (RAM).
- Each Word (collection of bits) has a memory address.

32FD00	0	0	0	0	0	0	0	0
32FD01	0	0	0	1	1	1	0	0
32FD02	1	1	0	0	1	0	0	0
32FD03	0	0	1	0	1	1	0	0
32FD04	1	1	0	1	1	0	0	0
32FD05	0	0	1	1	1	0	0	0
32FD06	0	1	0	1	1	1	0	0
32FD07	0	0	1	1	0	1	0	0
32FD08	1	0	1	0	1	0	0	0

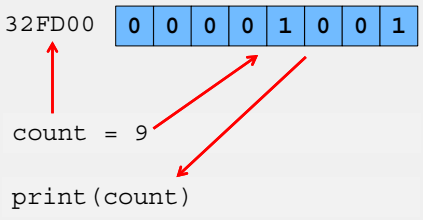
QA

8

Memory cells use transistors to store bits as a volatile storage of data at known addressed locations.

Storing data in memory

- Memory addresses and binary numbers are difficult to work with!
- Programmers use:
  - Meaningful symbolic names instead of memory addresses
  - Decimal numbers instead of binary



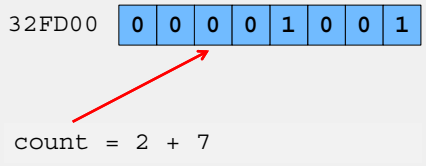
QA

8

To write programs we store data in memory by assigning values to symbolic variable names. These symbolic variables are simple aliases for the actual locations (memory addresses) that are used to store the data.

Performing calculations

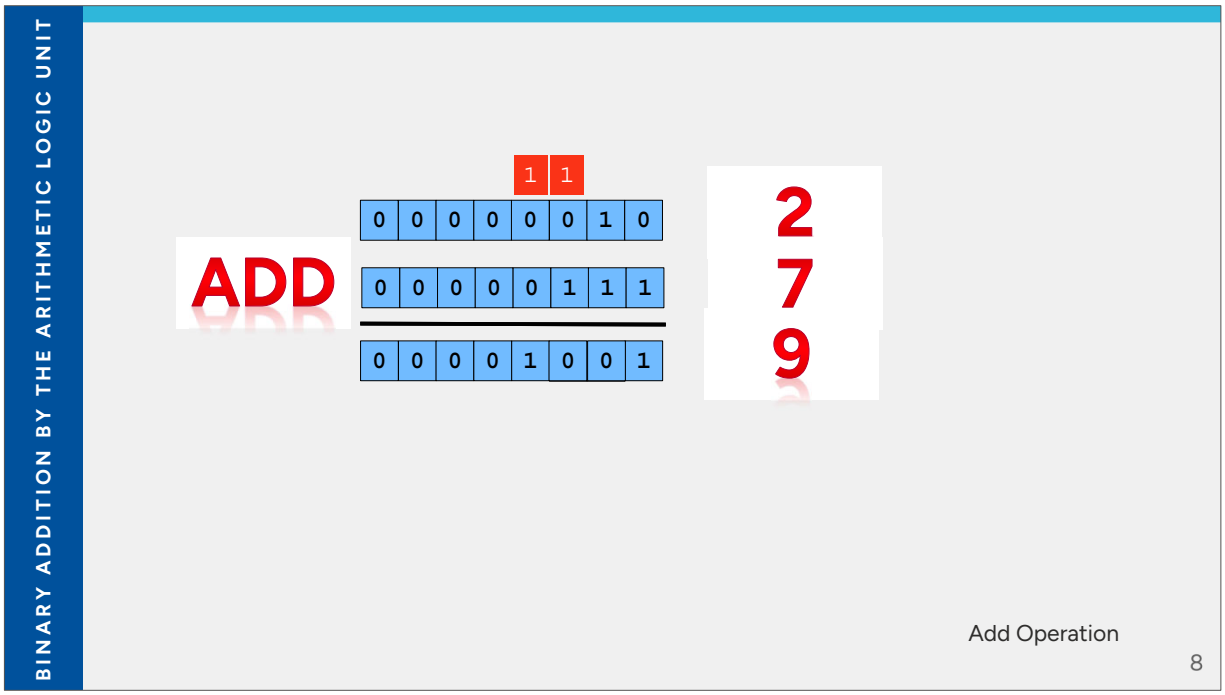
- Computers are great at performing calculations.
- We can write code to:
  - Perform calculations
  - Store the result



QA

8

If you can store data in memory you can perform calculations on it. Think about what might be happening in the CPU.



The CPU would perform and ADD instruction with operations on values in the CPU registers. The ADD operation would use a specific add logic circuit to carry out the calculation.

Question

- How many bits in a nibble?
  - A. 2
  - B. 1
  - C. 8
  - D. 4

Question

- What is binary 1101 in decimal?
  - A. 5
  - B. 7
  - C. 13
  - D. 15

Summary

- Switches can represent a 0 or 1 = bit.
- 8 bits is a byte.
- 32 or 64 bits is a Word = Unit of processing.
- Collections of bits represent numbers.
- Words are used to store data in memory.
- Memory locations have addresses.
- We use symbolic names in computer programs.

Discovering Data Types

Programming Foundations



QA
Learn. To Change.

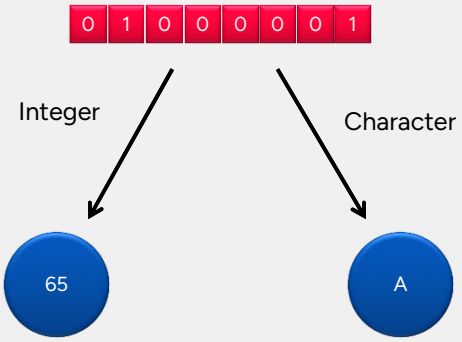
Overview

- Understanding data and type
- Storing numbers
  - Whole numbers
  - Floating point numbers
- Characters and ASCII plus strings
- Boolean data
- Statically and dynamically typed languages
- Data sizing
- Composite types
- Variance
- Displaying and retrieving data

QA

Data and type

- Computer program data stored in a bit pattern.
  - Stored in binary at specific memory locations.
  - What that bit pattern means depends on how we classify it.
  - Is it a whole number? Could it be treated as text?

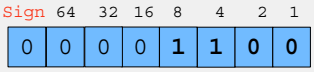


QA

Computer Data stored in binary only has meaning when we apply type information to it. The same binary byte will be interpreted differently if it is an integer or a character type.

Storing numbers in memory

- Whole numbers
  - Numbers without a decimal point
  - Stored as a collection of bits
  - 8 bits allows the storage of a number between 0-255
  - Known as positive integers (aka unsigned integers)
- What about negative whole numbers: -55 ?
  - To store signed integers, we need to be able to remember the + or the –
- Sign Bit is set to 1 when negative
  - All other bits are flipped (0 becomes 1 and vice versa)
  - 1 is added to result
  - Known as Two's complement storage



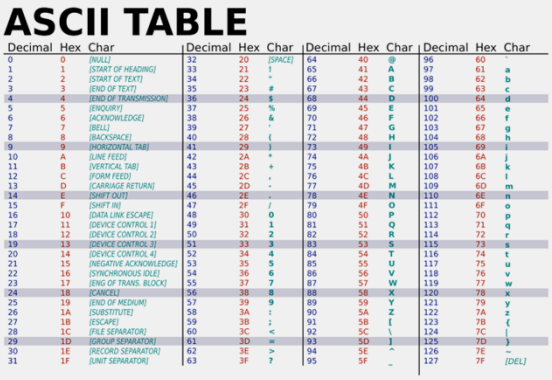
QA

An integer is a whole number, normally represented by a single machine word. As such it should be the fastest unit of arithmetic, and its range is determined by the word size of the machine. One bit, however, is lost for the sign bit, which indicates if the number is positive or negative. The most significant bit (the leftmost bit) is set to 0 for positive numbers and 1 for negative numbers. Negative numbers are usually held in two's complement, in which you find the positive value of the number, then each bit value is switched (zeros become ones and ones become zeros) and then one is added (in one's complement the one is not added). For example: positive 12 as a binary value is 00001100. As a negative value it is 11110011 + 1 -> 11110100. The loss of the sign bit gives us a range of (-2147483647 - 1) to 2147483647 in a 32-bit word, and (-9223372036854775807 - 1) to 9223372036854775807 in a 64-bit word.

Some languages support unsigned integers. These obviously allow a larger range, but must only be used for non-negative values, like, for example, the size of a file. A very large file of 4GB (gigabytes) has a file size expressed in hexadecimal of 0xFFFFFFFF, as a signed number that is –1 !

Characters

- Computers understand numbers, but humans also use letters.
- We need to be able to translate numbers to characters.
  - Character Sets associate a number with a character
  - ASCII is a table of 7 bit numbers and associated characters

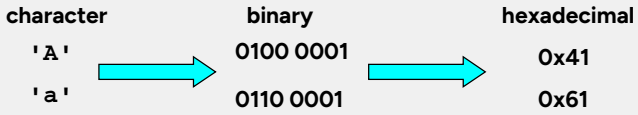


QA

Computers understand numbers not text. To allow the use of text we need to create a translation between number codes and a character values. ASCII was defined as a 7 bit character table defining a list of characters with associated unique numeric codes from 0 to 127.

Representing characters

- How is text represented?
  - Everything is a binary pattern, a number.
  - How do we represent the letter “A”?
- Answer: assign a bit pattern for each character.
  - One 8-bit byte gives us 256 combinations.
  - With a 26-character alphabet, that's more than enough! Isn't it?



- Problem: some alphabets have > 256 characters!
  - Use 2 or more bytes to represent characters.
  - *Unicode* characters use at least 16 bits: 65,536 characters

QA

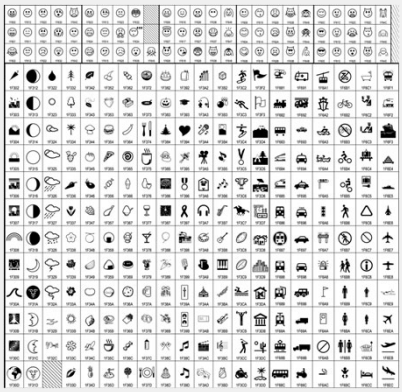
On PC's and UNIX the most common character set used in the Western hemisphere is ISO Latin 1 (8859/1), which has as its first 128 characters the American Standard Code for Information Interchange (ASCII, pronounced ass-key). In this code the character "A" is represented by binary 0100 0001, which is easier to write as the hexadecimal number 0x41 (notice how we prefix a hexadecimal number with 0x). If Hebrew, Arabic, or Cyrillic character sets are used instead, then the symbol represented by this value will be different.

Many mainframe computers use a different standard, EBCDIC (Extended Binary Coded Decimal Interchange Code), in which the letter 'A' is represented by 0xC1, and 'a' by 0x81.

Some alphabets, for example Japanese Kanji, require more than the 256 possible patterns available in a single byte. The solution to this is to use a multi-byte character set, and the most commonly used today is known as Unicode; there are others, like Microsoft's DBCS (Double Byte Character Set).

Unicode

- ASCII was limited because it was 7 bits
  - Largest number = 127 or 128 characters including zero
- Extended ASCII — ISO 8859-1 used 8 bits giving 256 characters
- Unicode was further developed to use 16 bits
  - Up to 65536 characters

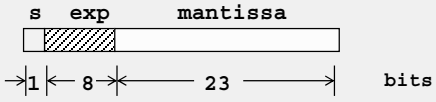


QA

Original ASCII was 7 bit allowing 127 character codes. Extended ASCII gave an 8 bit set with 256 codes. Unicode extends it further to 16 bits and 65535 codes.

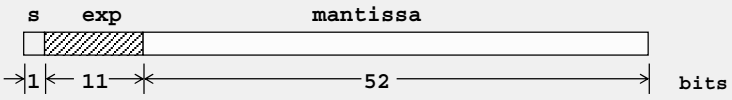
Representing floating point numbers

- IEEE 754 floating point formats
  - Single precision floating point
  - Number range: 1.175494351e-38F to 3.402823466e+38F
    - 6-7 decimal digits



s	sign
exp	exponent (says where the decimal point is)

  - Double precision floating point
  - Number range: 2.2250738585072014e-308 to 1.7976931348623158e+308
    - 15-16 decimal digits



QA

Floating point numbers are used extensively for scientific and engineering work, and also in calculations for computer graphics.

These numbers are held according to precision. Since we do not have infinite computers, then limitations are placed on the number format, which can lead to rounding errors in arithmetic. The greater the precision, the smaller the rounding error, but they cannot be eliminated entirely so never use floating point numbers for monetary calculations!

Single precision uses 32 bits to store the number, and double precision uses 64. There are three components to both, the sign bit (0: positive, 1: negative), the exponent, and the mantissa. For example, 43e2 means 43 squared (to the power of 2), where 43 is the mantissa, and 2 is the exponent.

These formats were standardised in 1985 by IEEE (pronounced eye-triple-ee), the Institute of Electrical and Electronic Engineers.

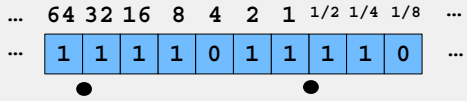
So why have two standard formats? A 32-bit machine must use two machine words for double precision calculations, which is inherently slow. The CPU can only do single word calculations, so double precision arithmetic may have to be emulated by software. Hardware manufacturers now produce maths co-processors that are much faster than software emulation.

Some languages may have higher levels of precision available, for example a "long double", which may be 128-bits, or even a "long long double". These allow numbers of very high accuracy. For example, C# has a 128-bit type called **`decimal`** (literals are suffixed with an 'm') which does not conform to the IEEE layout. Other formats also exist.

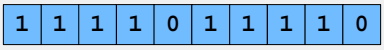
EXAMPLE – FLOATING POINT

123.75

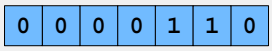
- In binary has the following form:



- Move decimal point to most significant bit — 6 Places
  - Mantissa



  - Exponent=6

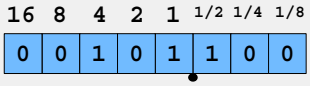


To represent decimal numbers the bit pattern is used to store the decimal portion of the number. The fractional part of a number is represented as constituent fractions. So 0.75 is made up of ½ + ¼

The overall number is stored as the mantissa and the exponent of the number is calculated from shifting the decimal point to the most significant bit and storing the number of bits shifted.

Rounding errors in floating points

- To allow fast calculations, Mantissa represents fractions within bit pattern:



4 + 1 + ½ = 5.5

- Allows fast calculations but makes representation of all numbers difficult
  - Produces rounding errors
- Binary for 0.1 is:
  - 0.0011110111001100110011001101
  - But represents value 0.10000000149011612

QA

Because floating point numbers represents the fractional part of the number using a fractional bit pattern some numbers aren't entirely accurately represented.

The number 0.1 is actually stored as 0.10000000149011612

E-numbers

- Very small, or very large, numbers can be laborious and error-prone to type.
  - They can also be more difficult to read.
- Most languages support E-notation.
  - Most support the "e" in upper or lower case.
  - Some convert to E-notation automatically on output.

Conventional	E-notation
38	3.8e+01
3800	3.8e+03
38000000000000000	3.8e+16
0.38	3.8e-01
0.0038	3.8e-03
0.00000000000000038	3.8e-16

QA

To convert from +E-notation to conventional notation, move the decimal point to the *right* e-number of times.

To convert from -E-notation to conventional notation, move the decimal point to the *left* e-number of times.

In a Python Integrated Development Environment such as IDLE or PyCharm, try:

print(1234567890.1 * 1234567890.1)

The format for printing numbers using this notation in Python is: `print "%e" % (`*number*`)`

`The output should look as follows:`

`1.5241578752659656e+18`

Other data types

- Strings are collections of characters



  - Strings are often terminated with a null character \0

- Boolean
  - Represents true or false states
  - Primarily used to record the outcome of logical tests
  - Also used to store simple state
  - Can be as simple as 1 or 0

QA

A character is a numeric code representing a character in a character set. Collections of text characters in programming languages are called strings. In many languages strings are terminated with a null characters so that it's clear where the string ends.

Fundamental types

- A single data item is known as a *scalar*
- What does that item contain?
  - Integer? Characters? Floating-point number?
  - A memory address?
- Programming languages have two alternate strategies:
  1. Impose discipline
     - Programmer must specify an item's *type*
     - Precise
  2. Figure it out from the context
     - Compiler and run-time libraries have a lot more work to do
     - Programmer has to be aware of context
     - Can lead to side effects

QA

A single data item is known as a scalar, but this does not indicate what kind of data it contains or its size. We need to indicate somehow if the item contains characters, numbers, or any other data type. There are two main ways a programming language decides the type of a data item.

The language could resolve the problem by context. For example, if we add two values together using the + operator, then they must be numbers. If we try and join them together then they are probably strings of characters. This gives the underlying software a lot more work to do, and there can still be issues. Most of these languages have to allow the programmer to force a type and override context.

Conventional languages require that the programmer declares the type of a scalar before it is used, so everyone knows where they stand. It is important to recognize the key role played by typed declarations in a high-level language. It is by means of type that the programmer specifies how the contents of the memory are to be interpreted by the program. In doing this, a typed declaration specifies how much memory is needed for a particular entity, how the contents of that memory are to be interpreted, and other vital details.

Even with typed declarations many languages allow types to be overridden (often called casting), but this is considered dangerous as it may lead to data loss.

Declaring data variables

- Computer programs store data in variables.
- Variables have:
  - A location in memory
  - A memory address
  - A symbolic name/alias used to refer to the memory location
  - A value
- Dynamic Typed Languages — Python
  - Data Type interpreted from assignment

```
age = 21
```



- Static Typed Languages — e.g. C#

```
int age = 21;
```



QA

Computer programs capture, process, persist and transmit data as part of their normal operations. In order to do this the data has to be stored in memory.

To do this we write code to hold data in data variables. A variable is an area of memory that holds a piece of data of a specific type.

To be useable in a program we tend to label variables with symbolic names that make sense to us. Age, FirstName, Address would all be great descriptive variable names.

Choosing variable names

- Most languages are case sensitive.
  - MyName, MYNAME, and myname are different variables.
  - Case–sensitive languages include: C, C++, Java, C#, Pascal, Delphi, Perl, Python, and Ruby.
  - Use descriptive, meaningful names.
  - Long variable names do not slow down your program.
- Don't use variable names that are:
  - Reserved words/keywords.
    - if, while, return, int, float, char, string, sub, function, def, etc.
    - These vary depending on the language.
  - Built-in or library function names.
  - Most language documentation includes a list.

QA

The name you give a variable will stay with it for the life of the program, which could easily be 10 years or more. Many programmers and support staff will be rifling through your most intimate lines of code, so it is worth spending some time on good descriptive names. The only justification for using variable names like x, y, and z are if these are graphic co-ordinates! Since, in compiled languages, the name will be translated into an address anyhow, its length will make no difference to the time a program takes to run, or the memory it uses. Yet descriptive names can save significant time in debugging.

Most programming languages allow only alphabetic characters for the first letter of a variable name, then alphanumeric for others. So you can't use punctuation characters or spaces. The underscore character _ is often allowed, but might have a special meaning when used as a first character.

There are a number of naming conventions, and if your project or site uses one then it makes life easier for everyone if you stick to it.

Generally a *constant* will be given an all UPPERCASE name. This is just a convention in most languages, but Ruby enforces this. Use an uppercase name in Ruby and you get a constant.

Declaring data of different types

- Dynamic Typing: Python/Basic

```
age = 21            # Integer
firstName = "Fred"  # String
married = True      # Boolean
salary = 25000.50   # Floating Point
```



- Static Typed Variables: C#

```
int age = 21;                // Integer
string firstName = "Fred";   // String
bool married = true;         // Boolean
float salary = 25000.50;     // Floating Point
```



QA

Dynamic Typed languages can infer type from the initialisation variable. This doesn't mean type isn't stored for the variable but usually does mean the type of the variable can change.

Static types languages like the C based languages require variable declaration although do allow type inference in some cases. Crucially though, once a type has been set it is fixed.

Dynamically typed languages — Python



- Variable Type Inference

```
x = 21
y = "This is a string!"
```

- Type Mutation in dynamic languages

```
x = 21
x = "Hello"  # x is now a string
```

QA

In dynamic languages the type of a variable can mutate from one to another.

Strictly speaking, the type of x does not change. In the above Python example, x is a variable that points to an area of memory that contains an integer with a value of 21. Then the variable x is changed to point to a different area of memory that contains a string with a value of “Hello”.

Note: Variables in Python do not have a data type. They are reference pointers to memory resident objects that have a data type and hold a value.

Using variables before declaring them

- You must declare a variable before using it.

```
print(x)
NameError: name 'x' is not defined
```

- Better would be:

```
x = None
print(x)
```

- Or:

```
x = 99
print(x)
```

QA

Variables must be declared before they are used in code. In the above example we are trying to print variable x before it has been declared, resulting in an error.

It is best to declare it first even if you set it to **None**, which in Python essentially means "unknown" or "nothing". Other languages use the keyword of **null** or **Nothing**.

Even better would be to initialise your variable with a known value, such as 99.

Determining variable type

- How do you find out the type of a variable?
  - Most languages have a command that tells you.
- Python

```
x = 21
if type(x) == int:
   print("x is an int!")
```

QA

Most languages allow you to determine the type of a variable and then make a decision based on the data type. Above you can see an example in Python.

The equivalent C# would look like this:

```
var x = 21;
        if (x.GetType() == typeof(int))
        {
           Console.WriteLine("x is an int!");
        }
// or
        if (x is int)
        {
           Console.WriteLine("x is an int!");
        }
```

Controlling the size of data variables

- Static typed languages like:
  - C#
- Integers can use just 8 bits to store a number.
  - Max value is 255 if it's an unsigned integer.
  - Other sizes are available.



```
sbyte    eightS;       //  signed 8-bit integer
short    sixteenS;     //  signed 16-bit integer
int      thirtyTwoS;   //  signed 32-bit integer
long     sixtyFourS;   //  signed 64-bit integer
byte     eightU;       //  unsigned 8-bit integer
ushort   sixteenU;     //  unsigned 16-bit integer
uint     thirtyTwoU;   //  unsigned 32-bit integer
ulong    sixtyFourU;   //  unsigned 64-bit integer
```

QA

With statically typed languages you can fix the data type which also allows you to control the number of bits used to hold the data.

In the example above, using the C# programming language variables are declared as signed or unsigned 8, 16, 32 or 64 bit integers.

Generally it is advisable to choose the smallest data type that accommodates the data range you require. For example, you don't need a signed 64 bit integer to hold a person's age.

Getting it wrong

- Ariane 5 rocket's computer system

  Horizontal velocity in 64-bits (inertial guidance system)

  `x` `1111111111111111111111111111111111111111111111111111111111111111`

- Program

  Horizontal velocity in 16-bits (steering control system)

  `y` `1111111111111111`

- Result

  `y = int(x)` `Error: out of range, overflow`

  - Computer shutdown
  - Loss of altitude control
  - Automatic self-destruction

QA

What happens if the programmer ignores such low-level details?

In 1996 the un-manned Ariane 5 rocket went off course and was destroyed 40 seconds into its maiden flight. The rocket and it's four satellites were uninsured and worth $500 million. The cause: an overflow error in a conversion from a 64-bit floating point value into a 16-bit integer. The programmer ignored or was not aware of these limitations. The software was the same for the less powerful Ariane 4, but the horizontal distance covered in the first 40 seconds was small enough to fit into 16-bits. It is reported that a complete system test would have found the problem but was vetoed for budgetary reasons. Oops!

See http://www-users.math.umn.edu/~arnold/disasters/ariane.html for further details.

Data variance

- Will its value alter during program execution?
  - No? Then use a constant.

```
const double VAT_RATE=20.00; //C#
```



- What operations will be applied to it?
  - Remember, not all operations are sensible on all types.
- Never use a variable name for more than one thing.
- Don't use an unsigned variable unless you need it.
- Be careful with floating point and be aware of rounding errors.
- Avoid global variables.

QA

Before defining a variable, do we really need it to vary? If the value will not change at run-time, use a constant (if the language supports it). What difference does that make? Apart from the obvious one - you cannot alter it by accident, constants often reside in different pages in virtual memory. Since its contents never change, a page of constants (also known as a pure page) does not need to be swapped out, it can be overwritten when the memory is required — we already have a copy of the values in the program.

An important factor in deciding what variable type to use is the operations to be performed on it. Use the right type for those operations. Many languages allow data to switch between types, this can have unexpected side-effects and is confusing to the reader. A variable should only ever be used for one thing.

Most languages provide signed versions as the default, and CPUs are designed to work with signed integers – so unless you need an unsigned number, use a signed number.

Displaying data

- Simple Console Type Applications allow data printing to the console window.
  - Python:

```
age = 21
firstName = "Fred"
print(firstName, "is", age)
```



  - C#:

```
int age = 21;
string firstName = "Fred";
Console.WriteLine($"{firstName} is {age}");
```



QA

The simplest types of applications interact with users from a basic console window. We can interact with these types of applications by reading from, and writing to, this console.

Python allows us to print to the console to display data. Other languages have their own ways of interacting with the console.

The Python print() function accepts any number of arguments and will concatenate (join) the text strings together, by default, with spaces.

C# uses the WriteLine() method of the Console class to produce output. It takes a format string which interprets the variable values as part of the outputted string.

Retrieving data

- Simple Console Type Applications allow keyboard input data collection:
  - Python:

```
age = 21
firstName = input("Enter your name:")
print(firstName, "is", age)
```



  - C#:

```
Console.WriteLine("Enter Your Name:");
string firstName = Console.ReadLine();
Console.WriteLine("Hello " + firstName);
```



QA

To ask the user for data we need to read data from the console.

In Python we can use the input() function, which returns a string containing the data entered by the user.

C# uses a two-stage approach. Firstly, we prompt the user for the required information with a Console.WriteLine() method. Stage two, we read the data entered by the user with the Console.ReadLine() method.

Composite types

- More complex data types
- Dates and times
  - Store date and time
  - Allow regional date formatting
  - Perform date/time calculations
- Decimal data types
  - Usual 128-bit type to store accurate decimal numbers
  - No rounding errors
  - Used for monetary calculations

QA

Most languages define a set of primitive data types including integers, floats, chars and booleans. From these types more complex types are created and these may include dates, times and even types specifically designed to process money.

Representing time

- Most computers have an internal clock.
- Count the number of seconds since *epoch*.
  - A well-known date/time.
  - Often stored as an integer.

00:00:00 1st Jan 1970
00:00:00 1st Jan 1980 ← ***Busts a 32-bit word in 2036***
00:00:00 1st Jan 1900
00:00:00 1st Jan 1904

- Libraries supply routines to convert:
  - To and from Coordinated Universal Time (UTC).
  - To and from local time, in various formats.
  - See: xkcd: ISO 8601
  - Some languages have specific date/time types.

QA

Dates and time are displayed differently around the world, for example in the USA the standard date format is MM/DD/YYYY (MM is the month, DD is the day, and YYYY the year), whereas in Europe the format is usually DD/MM/YYYY. Americans do not generally use the 24-hour clock, yet it is common elsewhere.

The number of days in each month does not follow any logical sequence, and the calculation of leap days is not so straightforward as it first appears. There is even the occasional leap second!

Some countries have Daylight Saving Time, and of course the world is split into time zones, which usually follow political rather than geographical boundaries.

Computer hardware has to have a simpler system than that, so generally computers count the number of seconds from some well-known fixed time and date, known as the epoch. It then leaves the software to sort out all the fiddly parts! Fortunately programmers do not have to code date validation and conversion themselves, there is a range of library routines available to do it. The display convention is determined by the locale, which was mentioned when discussing character sets. Programmers may have to force the locale, or the libraries might read it from the environment (typically LC_TIME).

The standard time zone used is Coordinated Universal Time, or UTC. This is the same as Greenwich Mean Time. Time Zones are not always handled well, they tend to be based on the assumption that the time difference is a whole number of hours from UTC, which is not always the case.

Some languages have specific data types for dates and time, for example Visual Basic .Net has Date, and SQL has a number of date/time types.

Date and time

- Can be initialized to a specific date or to the system date.
- Basic unit of time is the Tick.
  - Tick = Seconds since 1 Jan 1970 12:00AM (Epoch)

```
import datetime

# calculate now
now = datetime.datetime.now()

print("Current date and time:")
print(now.strftime("%Y-%m-%d %H:%M"))
print("Current date:")
print(now.strftime("%d-%m-%Y"))
print("Current time:")
print(now.strftime("%H:%M"))
```

```
Current date and time:
2018-12-03 16:15
Current date:
03-12-2018
Current time:
16:15
```

QA

Using a tick, a date and time can be established as an interval since a fixed date.

You can work with just date, time or both date and time. In Python, you import a module that provides these useful types and their functions. You can import the **time** module, the **date** module, or the **datetime** module.

```
import time
ticks = time.time()

t = time.localtime(time.time())
print("Time", t.tm_hour, t.tm_min, t.tm_sec, sep=":")
```

In the example on the slide, the datetime module is being used to display both the current date and time. In the example above, the time module is being used to display the individual components of the current local time.

This outputs:

Time:16:15:13

Exercise

Question

- Which type can have rounding errors?
  - A. Integer
  - B. Char
  - C. Boolean
  - D. Floating Point

Question

- ASCII can hold character codes between 0 and?

  A. 8

  B. 127

  C. 255

  D. 65536

Question

- Python is a dynamically typed language
  - A. True
  - B. False

Summary

- Data requires a type to have meaning.
- Integers can range in size: 8, 16, 32, 64 bit.
  - Signed
  - Unsigned
- Characters are translated from character sets into character codes.

ASSIGNMENT STATEMENTS

- Assign `42` to an integer variable called `var`, then print it

Language	Assignment statement	Console print
Bash, Korn	`integer var=42`	`echo $var`
Visual BASIC	`Dim Var As Integer = 42`	`Console.WriteLine(Var)`
C	`int var = 42;`	`printf("%d\n",var);`
C++	`int var = 42;`	`std::cout << var << std::endl;`
C#	`int var = 42;`	`Console.WriteLine(var);`
Java	`int var = 42;`	`System.out.println(var);`
FORTRAN	`var = 42`	`write(*,*) var`
Pascal, Delphi	`integer var := 42;`	`writeln(var)`
Perl	`my $var = 42;`	`print "$var\n";`
PHP	`$var = 42;`	`print "$var\n";`
Prolog	`Var = 42.`	`display(Var)`
Python	`var = 42`	`print(var)`
Ruby	`var = 42`	`puts var`

8

Assigning a value to a variable is intuitive in most languages, and a single equals sign is commonly used. Not all languages agree, and some ALGOL derivatives (like Pascal and Delphi) use := pronounced "assigned to". COBOL uses a different approach, with the MOVE statement (and assembler mv), which actually describes the operation rather well.

Most languages make spaces around the = optional, though it is usually better to put them in for readability. UNIX shells (Korn and Bash, etc.) do not allow spaces.

Notice that the statically typed languages all require the variable to have a type, integer (often shortened to int) in this case.

FORTRAN should declare the variable first. Unlike many languages, FORTRAN does not allow a declaration and an assignment on the same line.

Korn shell (not Bash) also supports print.

The Python print statement depends on the version, shown is for Python 3. Python 2 would be print var

printf (as shown for C) is also supported by Bash, Korn shell, C++, PHP, Perl and Ruby.

Notice that variable names in Prolog must start with an UPPERCASE letter, or an underscore.

COMMENTS BY LANGUAGE

Language	Comment to end-of-line	Multi-line comment
Bash, Korn shell	`# This is a comment`	none
BASIC (& VB)	`REM This is a comment`	none
Visual BASIC	`' This is a comment`	none
C89, C90	none	`/* Multi-line */`
C99, C++, C#, Java, PHP	`// This is a comment`	`/* Multi-line */`
FORTRAN 77	`C This is a comment`	none
FORTRAN 90	`! This is a comment`	none
Pascal, Delphi	none	`{ Multi-line }`
Pascal, Delphi (2)	none	`(* Multi-line *)`
Perl	`# This is a comment`	`=head1 Multi-line =cut`
PHP (2)	`# This is a comment`	
Prolog	`% This is a comment`	`/* Multi-line */`
Python	`# This is a comment`	`""" Multi-line """`
Ruby	`# This is a comment`	`=begin Multi-line =end`

8

The comments shown may be on a line on their own, but also (and commonly) at the end of a line of code.

Some very old languages, like BASIC and FORTRAN 77, support full-line comments only, prefixed REM for BASIC and C (in column 1) for FORTRAN 77.

Note that some languages, like C prior to C99, FORTRAN earlier than 90, and Prolog, do not have single-line comments.

Visual Basic does not have multi-line comment delimiters, but the IDE (typically Visual Studio) has key-stroke short-cuts (<CTRL>+k, <CTRL>+c) to comment a highlighted block of code. Note that ASP.Net uses the XML comment delimiters <%...%>.

The Python multi-line comment can use either double or single quote characters (but must be consistent at either end of the comment). It's not really a comment though, it's actually a bare string without any operations (a *no-op*). In Python a multi-line string can also serve as documentation, known as a *docstring*.

Hello world!

- Bash

```
#!/bin/bash
echo 'Hello world!'
```

- Korn shell

```
#!/bin/ksh
print 'Hello world!'
```

- Visual Basic

```
Module Hello

    Sub Main()
        Console.WriteLine("Hello World!")
    End Sub

End Module
```

QA

8

In Bash, usually it does not make much difference if you use single quotes or doubles quotes around the text string. In this case though it would. Double quotes would not work, because in Bash `!` is a special character which is *interpolated* (translated) inside double quotes. This is not the case in Korn shell.

Hello world!

- C

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    puts("Hello world!");
    return 0;
}
```

- C++

```
#include <iostream>

int main(int argc, char *argv[])
{
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

- C#

```
using System;

class hello
{
    public static void Main()
    {
        Console.Write("Hello World!");
    }
}
```

Hello world!

- Java

```
class hello
{
    public static void main(String args[])
    {
        System.out.println("Hello World!");
    }
}
```

- FORTRAN

```
      program hello
      write(*,*) "Hello World!"
      end program hello
```

- Pascal, Dephi

```
program hello;
uses crt;
begin
    writeln('Hello world!');
end.
```

QA

8

The FORTRAN has a mandatory 6 space indentation (FORTRAN 77).

In Pascal, `crt` comes from an out-dated term for a screen - Cathode Ray Tube.

Hello world!

- Perl

```
use warnings;
use strict;

print "Hello world!\n";
```

- PHP

```
<?php
    print "Hello world!";
?>
```

- Prolog

```
hello :- write('Hello world!')
```

- Python 3

```
print("Hello world!")
```

- Ruby

```
puts "Hello world!"
```

QA

8

Perl also has:

```
use warnings;
use v5.14;
say("hello world!");
```

PHP also has `echo`, which is subtly different from `print`.

The Python example is Python 3, in Python 2:

```
print "Hello World!"
```

Ruby also has `print`.

`PHP also has echo.`

Performing Calculations and Manipulating Data

Programming Foundations



QA
Learn. To Change.

Overview

- Performing calculations
- Using operators
- Controlling order of operator execution
- Using Math Libraries
- Rounding
- Converting data
- Working with strings
- Manipulating dates

QA

8

Performing calculations

- Why do programs perform calculations?
  - To solve problems/answer questions.

```
If it takes a truck 4 hours to travel a distance of
200 miles. How long will it take it to travel 300
miles
```

- Programmers translate these problems into equations:

```
distance = 200
time = 4
averageSpeed = distance/time
```

- And then answer questions:

```
newDistance = 300
newTime = newDistance/averageSpeed
print("Time to travel", newDistance, "is", newTime)
```

QA

8

To perform the calculations above we used = to perform assignment, / to do divide, * for multiply.

Calculations are a major part of what computer programs do. From calculation statistics to working out climate change models and even predicting the economy.

Operations on data

- What operations are sensible?
  - Assign a value to a variable =
  - Add/subtract/multiply/divide numbers + - * /
  - Join together (concatenate) characters + . &
  - Add or remove a scalar to, or from, an array
    - Not legal in all languages
- What operations are not sensible?
  - Change a constant? ✘
  - Assign 256 to a single byte? ?
  - Add two characters ✘
    - "A" + "B" = ?
  - Assigning incompatible data types

QA

8

Assigning a value to a variable sets up data to be operated on and usually involves the "=" operator.

Storing a value in a variable is not particularly useful by itself. We probably want to carry out operations on it, add something to it, multiply it, search it, or compare it with something else. The symbols (operators) used for arithmetic operations are mostly familiar. When it comes to operations on other types, things are not so straightforward.

So what operations are sensible on, say, a character string? If we use the arithmetic operators on strings then it is not necessarily obvious what they will do. In fact most languages will not allow them to be used. Still others (mostly Object Oriented languages) allow *operator overloading*, where you can define the operators to mean anything you like.

Some operations on some types are just plain illegal, and most languages will give an error if you try one of these combinations. It can be seen that operators are typed just like data.

Simple operations?

- Multiply
  - Usually *, not x
- Modulus
  - Usually %, not Mod
- Concatenate (join) strings
  - Sounds easy, but almost every language is different!
- Bit operations
  - Used for masks, and setting flags
  - Implementation-dependent
- Unary operators
  - Operate on one variable (to the right)
  - !(not), +(positive), and –(negative) are common

QA 5 8

The operators used for arithmetic operations are mostly familiar, although an asterisk is used for multiply instead of a lower case x (some languages use x for other operations).

Most languages either provide different operators for non-numeric data items, or make you call a library routine. They are unfortunately inconsistent. For example, to concatenate (join together) two text string variables:

C: `strcat(szOne, szTwo);`

Pascal: `sOne = concat(sOne, sTwo);`

C++, Java, Python: `One = One + Two`

Visual Basic: `One = One & Two`

Perl, PHP: `$One .= $Two;`

Many languages allow operations on bit patterns (see slide after the summary), but there are obvious portability problems with them. Different operating systems and machines have different word lengths, and integer numbers are represented in different ways. What happens to the sign bit? Some implementations retain that bit, some include it in the operation. Even different implementations of the same language differ! Avoid if you possibly can!

Most operators have two 'arguments', the left and right hand sides (lhs and rhs). These are known as *binary operators*, a phrase we do not use much because of easy confusion with bit operators. Some languages have ternary (three argument) operators as well. Operators with only one argument are known as unary operators. The common ones are shown, but languages often have more. For example in C, &MyVar means the address of MyVar.

Performing operations

- Declaring a variable

```
count = 1
```

- Modifying a variable

```
count = count + 1  # Add
count = count * 10 # Multiply
result = count - 5 # Combining variables
```

QA 8

Once a variable has been initialised it can then be modified by performing operations on it.

For a numeric variable these might be math operations.

Standard operators

- Standard Operators

+	Add
-	Subtract
++	Increment (C#, Java, C)
--	Decrement (C#, Java, C)
*	Multiply
/	Divide
%	Remainder

- Compound Operators

+=	Compound Add
-=	Compound Subtract
*=	Compound Multiply
/=	Compound Divide
%=	Compound Remainder

QA

8

Most languages provide the following operators and a set of compound versions.

The compound version of an operator is a kind of shorthand syntax for what would be more complicated code.

So,

count=count+1

Becomes:

count+=1

The lines are directly equivalent.

Using the standard operators

- Math Operations

```
count = 1
count = count + 1
count += 1
count *= 2
print(count)  # Prints?
```

- Modulus Operator

```
test = 10
result = test % 3 # Remainder of 10 divided by 3?
print(result)
```

QA 8

The compound operations displayed above take the variable, apply the operation to it, and assign the result back to the variable.

Operator precedence

- Most languages use the standard mathematical rules for operators...

```
total = 2 + 3 * 4
```

Is the result 20 ...
or 14?

- The rules can be easily remembered as BODMAS
  - Brackets
  - Order — raising to a power, as in x2
  - Division
  - Multiplication
  - Addition
  - Subtraction

```
total = 2 + (3 * 4)
```

Is this clearer?

QA

8

There are variations on the BODMAS theme, PODMAS, where the 'P' stands for parentheses, and BIDMAS, where the 'I' stands for indices, which is another way of describing raising to a power.

The 'order' part doesn't apply to all languages: some do not support the widely used ** operator for doing this. There is also a maths library function, usually called pow(), that will do this instead.

Other operators are less obvious, for example where do bit operations come in this list? Does a unary negative have the same precedence as binary subtract? What about Boolean operators like NOT? We cannot give you the answer, because it depends on the language you are using.

There are other problems with relying on precedence. A calculation which relies on precedence is an accident waiting to happen — someone will misunderstand it. When looking for a bug in code we are usually under pressure, possibly with the user (or your boss) breathing down your neck. Are you sure you will always spot the correct precedence? It is far safer, and no less efficient, to use brackets to force the precedence you want, and make the statement clearer.

String operations

- Does this make sense?

```
greeting = "Hello"
target = "World"
message = greeting + " " + target
print(message)
```

- String Joining (concatenation) can often be performed using +
  - Usually also applies to compound operator +=

- Other String operations include (among many others)

```
length = len(greeting) # Length of string
print(message[0:5])  # prints Hello = chars 0 to 4
uMessage = message.upper() # ?
print(uMessage)
```

QA

8

Strings can usually be joined using the + operator.

The C# equivalent would be:

```
string greeting="Hello";
string target="World";
string message=greeting + " " + target;
Console.WriteLine(message);

int length=message.Length;
Console.WriteLine(length);
```

When things can go wrong

- You can perform the wrong operation on a data type.
  - Wrong operation on wrong data type:

```
name = "Tom"
name += 1
TypeError: can only concatenate str (not "int") to str
```

  - Operation on non-existent data item:

```
count += 1
NameError: name 'count' is not defined
```

  - Mix and match:

```
count = 1
message = "There are " + count + "apples"
TypeError: can only concatenate str (not "int") to str
```

QA

8

You need to make sure you apply appropriate operations to the right kind of data. Performing a maths operation on a string isn't going to work. Treating a number like a string will also be a problem.

Explicit data conversion

- When would we need to convert one data type to another?
  - Number ➔ String to display it?
  - String ➔ Number to perform a calculation on user input?
- Most Languages make it easy(ish)
  - Converting from strings to numbers

```
ageString = input("Enter your Age:")
age = int(ageString)
```

  - Converting numbers to strings

```
count = 12
countString = str(count)
```

QA 8

You can take control of a data type by specifically converting one data type to another.

Operating on different types

- Maths operations are allowed between numeric data types

```
x = 10.0   # float
y = 5      # int
z = x * y  # ?
print(type(z))
```

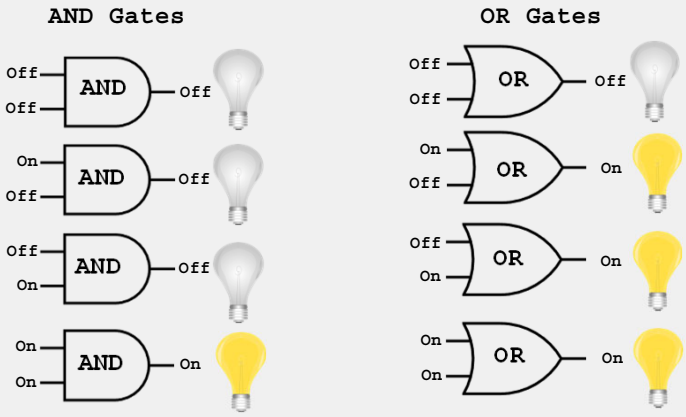
QA

8

Multiplying an int and a float will result in a float. Typically the widest data type in the equation will be the resultant type.

Bit operations

- Basic operations include And/Or
  - Implemented by Logic Gates
- Logic gates take two inputs and have one output



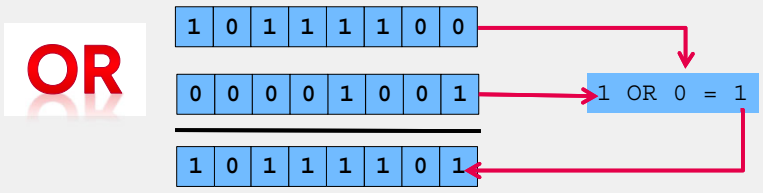
QA

8

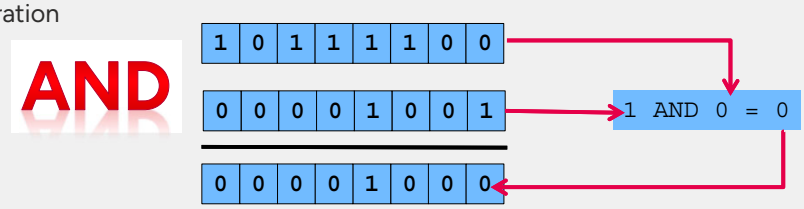
Most languages allow logical bit operations to be perform on simple integers. They operate at the bit level.

Perform logic operations on data

- OR Operation on two data items



- AND Operation



QA

8

When ORing two values together, each bit is compared. Only one bit needs to be set for the resultant bit to be set to 1.

For AND operations, both bits need to be set for the result to be set.

Bit operations

- Logic Operators can apply to whole numbers
  - Act on the bit pattern of a number
- & (and) logic requires both bits to be set to for the result to be 1

```
0b010 & 0b111 == 0b010
```

- | (or) logic requires only one bit to be set for the result to be 1

```
0b010 | 0b101 == 0b111
```

QA

8

You can experiment with binary and logical operations using 0b to specify a binary number and & or | to perform AND and OR operations.

More complex operations

- Most Languages have a Math library
  - Access to standard log, sin, cos functions, etc.
  - Even hold the constant for Pi

QA

8

Exercise

QA

8

Question

- What is the result of 12 + 2 * (12 + 2)?
  - A. 196
  - B. 40
  - C. 38

QA

8

Question

- How do you determine the length of a string in Python?
  - A. "hello world".length()
  - B. "hello world".count
  - C. count("hello world")
  - D. len("hello world")

QA

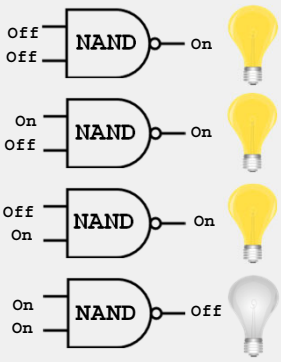
8

Summary

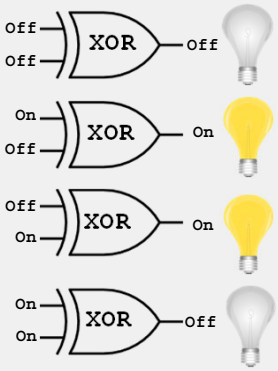
- Operators allow new data items to be created from existing ones.
- Math operators can be used in complex equations.
- BODMAS dictates the order operations apply.
- Strings can be manipulated using operators.
- Operations can be performed on a per bit basis using bitwise operators.

Other logic gates

- NAND Gate



XOR



QA

8

Organizing Data into Containers

Programming Foundations

QA
Learn. To Change.



Data containers

- Arrays and lists
- Sorting
- Other linear types
  - Stacks, Queues, Deques
- Non-linear types and keys
  - Linked lists, Trees, Associative arrays
- Compound types
- Class
- Object orientation
- Summary
  - Defining a class in Python
  - Associative Array examples

Question

- If you could only take five songs to a desert island, what would they be?
- To answer the question, you would build a list of songs:
  - Agadoo
  - Mr Blobby
  - Crazy Frog
  - Bob the Builder — "Can We Fix It"
  - Teletubbies Say "Eh-Oh"
- Lists/Arrays allow us to group related (or unrelated) things together.
- We use collections in programming often called arrays or lists.
  - In Python, they are called Lists.

```
topFive = ["Agadoo", "Mr Blobby", "Crazy Frog", "Can we Fix
It", "Say Eh-Oh"]
```

QA

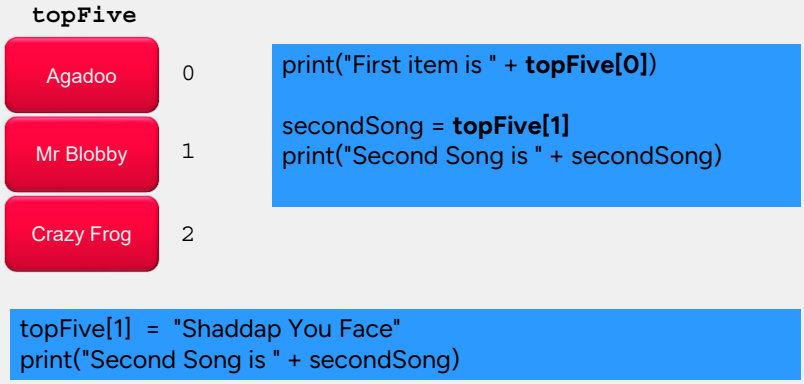
8

So far we have been defining and calculating individual pieces of data. It's often useful to group common or related data together into some kind of container. Containers are just easier to work with. After all, when you go to a supermarket you usually put your groceries in a container although you call it a basket. You can then work with a basket of groceries as a collection which is much easier to work with.

In programming we can organize data into many different types of structures but the most common is the array / list.

Accessing items in a list

- List Items accessed by their position in the list
- Usually starting at zero



QA What will happen now?

8

You can think of a list as a set of stacked boxes each with a number painted on the side. Starting at zero and incrementing by one each time.

The topFive list contains one song per box all under the topFive variable name.

We can access each song using the square brackets (the indexer). **topFive[0]** will give us the first item in the list.

Modifying a list

- Lists can be modified
  - append, insert, remove

```
topFive.append("Paperback Writer")
```

  - Adds the item to the end of the list

QA

8

Lists can usually be modified using a number of methods for appending, inserting at specific positions and removing items.

Sorting

- A common requirement to sort lists/arrays.
  - Sorted arrays are often quicker to search.
  - There are many different sort algorithms.
  - Typically, we usually use a language-supplied routine.
- Getting items in order might not be as simple as it appears.
  - What is the "right order"?
  - Does `É` come between `E` and `F`, or after `Z`?
    - It depends on the *locale* collation sequence.
  - What about objects other than characters or numbers?
  - In which order should you sort bank accounts?
- Sorts work by comparing the items in the list.
  - Most languages allow custom comparison routines.
  - If you know how to compare the objects, then you supply the code.

QA 8

It is surprisingly common to sort lists of values. This could be anything from lists of filenames through to lists of accounts, or any other type of object.

There are many sort of algorithms. There was a time when programmers had to learn the various different types, but these days most languages will supply a library routine. The most common forms of sort today are the **Quicksort** (or partition-exchange sort) and the **Merge sort**. You will be glad to hear that we do not discuss the actual algorithms in use, and most programmers will not know (or care) what they are. Their most significant difference is in the way that duplicates are handled.

You might think that it is obvious what order things come in, but is it? Even simple objects like characters have some ambiguity. Do we sort in character-set order, in which case É comes after Z, or alphabetic order? But what happens with different alphabets? The answer is that each culture has its own *locale*, which (among other things) defines character sort order, or *collation sequence*. Microsoft technologies call this *National Language Support* (NLS).

So how are other types of object compared? Consider a list of bank accounts: how do you compare two bank accounts in order to sort the list? The options might include by owner's name, by bank account number or by balance. It is up to the programmer to supply a comparison routine to enable the sort to compare. For example, if there are two items, a and b, the comparison routine will give:

-1	if a < b
0	if a equals b
+1	if a > b

A few languages have comparison operators that return just those values.

Sort order of numbers and text

- What is the difference between 0 and "0"?
  - 0 is a number
  - "0" is a text string
- 42 is greater than 9 `(42 > 9)`
- "42" is *less than* "9" `("42" < "9")`
- Try this in your IDE:

```
stuff = [42, 9, 3, 100, 62]
print(sorted(stuff))
#[3, 9, 42, 62, 100]


stuff = ["42", "9", "3", "100", "62"]
print(sorted(stuff))
#['100', '3', '42', '62', '9']
```

A list of numbers

A list of text strings

?

QA

8

We have to be very careful comparing text strings that represent numbers. The sort order or *collation sequence* may be surprising. Numeric comparisons are fairly simple, and fast. Text string comparisons may be slower, and work in a different way.

We would not hesitate to say that, in a sorted list, "Anne" would come before "Jim", because the "A" is before "J" in the alphabet. The fact that the second characters ("n" and "i") are in the reverse order does not affect the outcome, neither does the fact that there are fewer characters in "Jim" compared to "Anne".

Text strings containing numbers are no different. when comparing "42" with "9" the first characters are compared, "4" comes before "9", therefore "42" is less than "9".

To get a more intuitive order for text strings that contain numbers you must convert them, or alter the sort to do that for you. For example:

```
print(sorted(stuff, key=int))


# the key named parameter is the name of a function which denotes the
sort logic. The int() function will convert the items into integers and
use the resulting value to perform the comparisons. Note: the list
still contains the original text strings.
```

Membership operators

- Lists/Arrays usually have membership operators to test if an item exists within

```
fruit = ["Apple", "Orange", "Banana", "Pear"]

if "Pear" in fruit:
    print('Pear is in the fruit list')

if "Kiwi" not in fruit:
    print('Kiwi is not in the fruit list')
```

Tuples

- Read Only sequence of items.
- Tuple membership can't be changed.

```
fruits = ("Apple", "Orange", "Banana", "Pear")

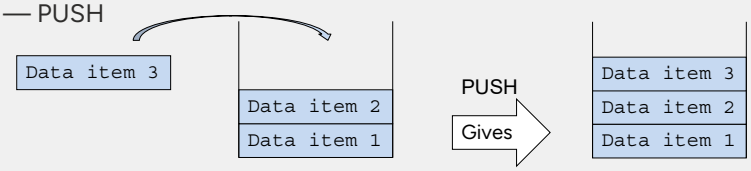
print(fruits[1]) # Orange

fruits[1] = "Clementine"
```

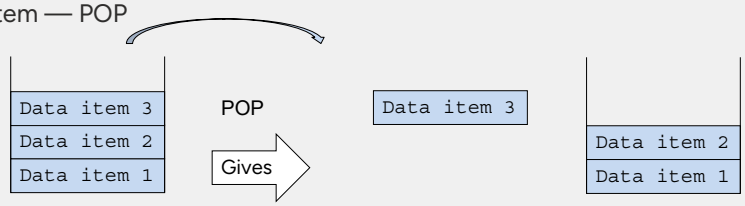
TypeError: 'tuple' object does not support item assignment

Other linear types — stacks

- Sometimes known as FILO (First In, Last Out).
- Add an item — PUSH



- Retrieve an item — POP



QA

8

Stacks store items by pushing them onto the stack - we can only add an item to the top of the stack (TOS - Top Of Stack), nowhere else.

We can only read or retrieve an item, **pop**, that is on the top of the stack, not one that is in any other position. Note that a pop operation removes the item from the stack, but stores that removed item in a location that can be used - it is not lost.

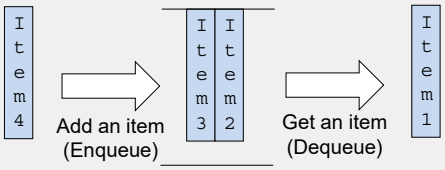
That seems rather restrictive, after all we can randomly retrieve any item from an array, provided we know its position. So why use stacks? *Speed*. Stacks are very simple structures, and require no memory to remember each position. We only need to know the location of Top Of Stack, when the stack is empty, and the size of each item. Stacks are used in many operating system and program features where speed is important, for example when passing parameters to a function.

Many hardware instruction sets include primitive instructions for stack handling, so stacks can be *very* fast.

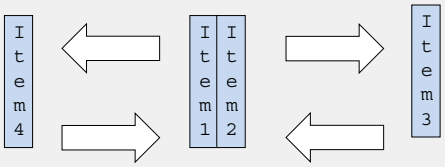
The truth is, most people use arrays, even when a stack would do the job. Many languages provide the stack features of push and pop for their array structures, so then you get the best of both worlds!

Other linear types — queues and deques

- Queue
  - Sometimes known as FIFO (First In, First Out)



- Deque (*deck*)
  - Can add or remove from either end



QA

8

Queues are useful containers in that they retain the sequence, so it is easy to retrieve data in the same order in which it was added. A Queue could be used to submit jobs in order for running later, or to submit a block of data to be processed.

Queues are often implemented as *singly-linked lists,* where each data item includes the address of the next item in the queue. Like stacks, they can be faster than arrays because we only need to know the address of each end.

The Dequeue or Deque, often pronounced *deck*, is more general purpose than a Queue in that we can add or remove items from either end. This means we need more functionality in our structure, so they carry more overhead than Queues or Stacks, i.e. they are *slower*.

Deques are often implemented as *doubly-linked lists*, where each data item includes the address of both the previous and the next items in the Deque.

Like stacks, several languages (e.g. Perl) include the features for Queues and Deques for their arrays, others (e.g. Python and C++) require an external library to be used.

Downsides of linear structures

- Searching:
  - Could take a long time, depending on how many items are searched.
  - Worst case is when the item is not there.
- Ordering:
  - Keeping an order can mean re-sorting, which is slow.
- Flexibility:
  - Adding or removing items can affect performance.
    - Particularly with arrays
    - This is why stacks and deques restrict access to the ends
  - The memory allocation might need to be restructured.

QA 8

The linear types we have seen so far are good in many programming problems. They are fast, and don't carry much *overhead* (extra data or processing connected with the structure).

A feature of linear structures is that they are *ordered*. That could be the order in which we added them, or some other order after a sort. Real-life is often not ordered or linear. What if we don't wish to process items in that order?

Sometimes we cannot predict which data item is required from a list next, we need *random access*. If you have an array of, say, employees, how do we find the one we are looking for?

If we need one particular item, then we have to search for it. That would take a long time, particularly if the item is near the end, or is not there at all!

We could sort the structure, but that could be slow as well.

Non-linear types and keys

- Arrays, Stacks, Queues, Deques — linear types.
  - Good if you know the position of the item you want
  - Bad if you have long searches
  - No use if you cannot predict which item is required next
- Non-linear types store items by a *key*.
  - A **unique** item which identifies the data required
  - The key should be related to the rest of the data
    - Employee number
    - National insurance number
    - Car registration number
  - Could someone's name be a key?
  - Could a postcode be a key?

QA

8

Considering the issues with linear structures, we need a mechanism which does not involve a huge search through the entire list of employees.

As with linear types, there are several approaches available. We shall look at some of the common ones in slides which follow. All these mechanisms allow us to search using a *key*. A key uniquely identifies a data item. It should be something which is easy and quick to compare, so a number is often the most efficient.

A person's name would be unsuitable as a key, because of duplicates. That is why organizations assign unique identifiers to just about everything recorded in a database, from employees to customers, to stock items and invoices.

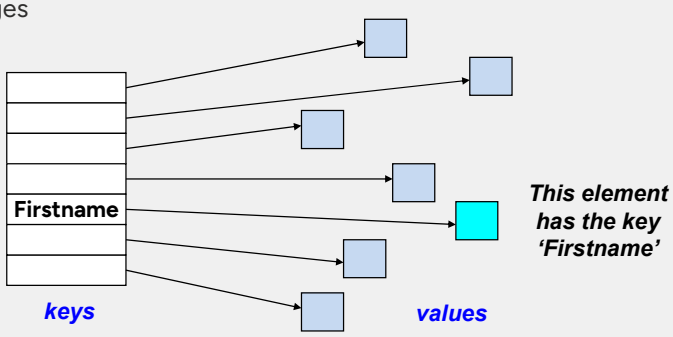
A postcode could be a key, depending on the application. In the UK, a postcode often refers to several different premises, which might mean that the data associated with the postcode is an array of addresses.

Database tables support more efficient access when each row has a primary key. The mantra often quoted is that the other fields within the row should relate to "the key, the whole key, and nothing but the key" (Bill Kent), often followed with "so help me Codd" (Ted Codd was one of the founding fathers of relational database theory).

Some languages (Python, C++, for example) support a container specifically for unique items (keys) called a *set*.

Associative arrays

- Accessed by a text *key*
- Not held together in memory, but linked by address
  - Often not ordered
- Also known as hashes, maps, hash-maps, or dictionaries
- Not supported by all languages
- Usually *dynamic* size



QA

8

Some languages (mostly high level) support Associative Arrays. These are composite types that are not held together in memory, but are referenced by a text key. Internally the key is often kept in a complex structure such as a tree, which allows fast list searching.

The advantage of this kind of structure is that once we have found the key, we can go straight to the data item. With a conventional array we may have to search through sequentially before we find what we want – the index is only a position, it may have no logical meaning which is relevant to the data being stored. Associative Arrays represent an overhead in memory, because the key and the element's address have to be kept. If we know the exact position of an item in a list, or we wish to process in sequence, then a conventional array is faster.

Another advantage is that the structure is dynamic, and can be added to as we proceed through the program. This can cause another headache for the language run-time library, because the list of keys may have to expand, which takes time.

An associative array is known as a hash in Perl and Ruby, a dictionary in Python and Java, and a map in C++.

There is an overlap in functionality between associative arrays and records. A record is a collection of fields, possibly of different data types, typically in fixed number and sequence. In some languages, records are referred to as structures. A difference between records and arrays is that associative arrays are dynamic, they can change while the program is running, while the structure of a record is fixed at the time the program is written.

This type of structure is often not ordered, although several languages give the programmer the choice. Generally, ordered associative arrays are not as efficient as unordered arrays, but Ruby has a neat internal work around.

Example — Python Dictionary

- Simple Dictionary
  - Key — Value pairs

```
employee = {"firstName": "Fred",  "lastName": "Bloggs", "age": 21}

print(employee["lastName"])  # Bloggs
```



QA

8

Dictionaries are made up of collections of **key:value** pairs. The key can then be used to retrieve the value from the dictionary.

Objects

- Most languages allow for the creation of composite data structures.
  - Data items made up of more than one piece of information
  - A Customer Object might contain data fields:
    - Name
    - Balance
- Composite Objects are usually based on a Record or a Class definition.
- Classes form the basis of Object Oriented Programming
  - A class is a template for creating an object
- Example:

```
myCar = Car("ABC123")
myCar.doors = 5
myCar.engine_size = 2.5
```

QA

8

A topic we will look at in more detail later on is objects. For now an object is a composite data type made up of one or more fields of information pertaining to a thing or object.

We use them to represent things that exist in the real and computer worlds.

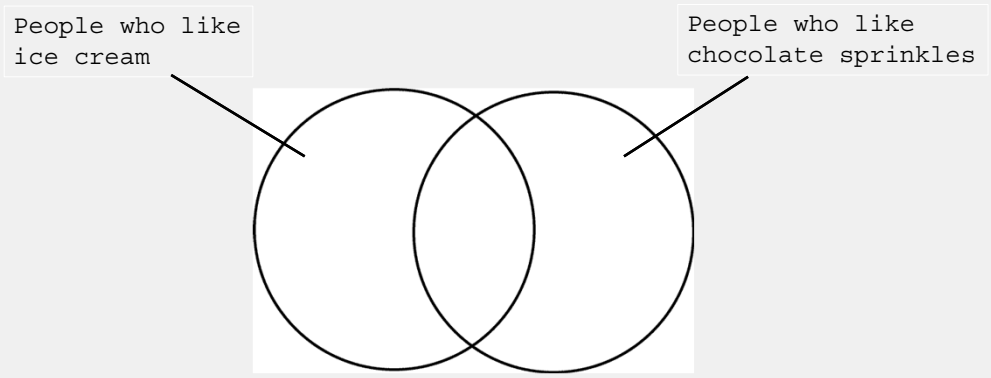
The example above is representing a Car object and storing the car's registration, door count and engine size.

In object-oriented programming, an object refers to a specific instance of a class. The object is a combination of variables, functions and data.

The class is a template for the creation of objects.

Sets

- Allow you to manage membership groups
  - Remember Venn diagrams in school?



Set operations

- Intersect
  - Common to both sets
- Union
  - Combination of both sets
- Minus
  - One set minus the common members of the other

Exercise

Question

- Which structure is fastest to search?
  - A. Dictionary
  - B. Stack
  - C. List
  - D. Queue

Summary

- Containers can hold a number of other data items.
- Some add and retrieve items only at one end.
  - Stack, queue, deque
- Some allow random access:
  - By position, in an *array*
  - By key, in an *associative array*
  - By field name, in a *record*
- Linear types are ordered, so can be slow to search.
  - Arrays, stacks, queues, deque
- Non-linear types are fast to access but use more memory.
- Records, Structures, and Classes define a new object type.

Array examples

- Construct an array of 6 integers and set third value to sixth value

- C

  This will also work in C++

```
int myArray[] = {43,67,22,33,90,99};
myArray[2] = myArray[5];
```

- C++

  Old C++ does not allow vectors to be initialised. Elements have to be pushed one-by-one

```
std::vector<int> myArray;
myArray.push_back(43);
...
myArray.push_back(99);

myArray[2] = myArray[5];
```

- C++ 11

  The C++ 2011 standard enables vector initialisation

```
std::vector<int> myArray =
                {43,67,22,33,90,99};
myArray[2] = myArray[5];
```

- C#, Java

  Notice the subtle difference from C

```
int[] myArray = {43,67,22,33,90,99};
myArray[2] = myArray[5];
```

QA

8

C only has static arrays, C++ also supports these but has a number of more flexible alternatives, `std::vector` is shown.

Initializing these arrays all use the values inside braces { }, except for old-style C++ where the array has to be constructed one element at a time. It should be noted that, with these languages, you cannot reassign using the brace { } notation with the same array. That is only allowed the first time the array is declared.

Array examples

- Perl

Perl whole-array variables are prefixed `@`. Scalars are prefixed `$`.

```
my @myArray = (43,67,22,33,90,99);
$myArray[2] = $myArray[5];
```

- PHP

Variables are prefixed `$`.

```
$myArray = array(43,67,22,33,90,99);
$myArray[2] = $myArray[5];
```

- Bash, Korn shell 93

No spaces around '=' allowed. Spaces, not commas, in the initialisation.

```
myArray=(43 67 22 33 90 99)
myArray[2]=${myArray[5]};
```

- Prolog

Variables must begin with an UPPERCASE alpha.

```
MyArray = [43,67,22,33,90,99].
```

QA

8

Perl variables get their type from an initial character (known as a *sigil*), $ denotes a scalar and @ an array. The individual elements have a $ prefix because they are scalars.

PHP uses a $ prefix for all its variables. Bash and Korn shell also use $, but in this case it is an operator that returns the variable's value. Prefixes like this are used in *interpolation* - the ability to embed a variable inside a string, yielding the variable's value.

Korn shell 88 (1988 version) declaration and initialisation of an array would be:

```
set -A myArray 43 67 22 33 90 99
```

The later Korn shell 93 (1993 to the present day) and Bash is rather more intuitive, as shown.

Replacing one value for another in Prolog is not shown. There are two reasons for this, the first is that lists are not normally used that way in Prolog, and the second is that the procedure required is quite involved, and there could be a considerable performance penalty. Prolog lists, although ordered, are not indexed, so accessing single elements randomly by position is not appropriate.

Array examples

- Visual Basic — VB unusually uses ()

```
Dim myArray As Integer() =
    New Integer() {43, 67, 22, 33, 90, 99}
myArray(2) = myArray(5)
```

- FORTRAN — …as does FORTRAN! But array indexes start at 1

```
dimension myArray(6)
myArray = (/ 43, 67, 22, 33, 90, 99 /)
myArray(3) = myArray(6)
```

- Pascal, Delphi — Pascal indexes also start at 1

```
Var myArray : Array[1..6] of Integer =
       (43,67,22,33,90,99);

myArray[3] := myArray[6];
```

- Python, Ruby — Simple?

```
myArray = [43, 67, 22, 33, 90, 99]
myArray[2] = myArray[5]
```

QA

8

Visual Basic has a surprising number of features in common with FORTRAN, but in VB index numbers start from zero.

The FORTRAN example is using a Vector, which is a one-dimension array. Shown is the "old" way of declaring a one-dimensional array, in FORTRAN 90 it could be declared as:

```
integer myArray(6)
```

FORTRAN arrays, by default, start indexing from one, but you can change this with:

```
dimension myArray(0:5)
```

The initialisation clause comes from FORTRAN 90, prior to that each element had to be assigned separately.

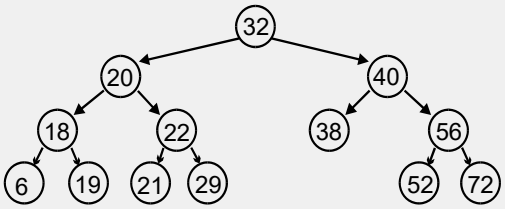
Pascal arrays start indexing from one. The initialisation syntax shown is not supported in all Pascal environments, particularly older ones. Those that do not support initialisation require each element to be assigned separately (as older FORTRAN and C++). But at least Pascal uses the more common square brackets to contain index numbers, which originally derives from ALGOL.

There are several ways of initialising an array in Ruby and in Python, the simplest is shown. Python and Ruby differ in detail, this is one of the few cases where they are identical. In Python, writable arrays are called *lists* and constant arrays are called *tuples*.

Like Perl, Ruby also has *sigils*, but they have meanings connected with *scope*, rather than *type*.

Containers for records — tree structures

- There are many different tree types; the simplest is a Binary-tree.
- The keys 32, 40, 20, 22, 56, 18, 6, 38, 52, 19, 72, 29, and 21 create the following tree:



- The tree is nicely balanced. For instance, only four comparisons are required to discover 98 is not in the tree.
- Only the keys are shown, the data could refer to a location on a disk.
- Such structures are often used by databases and directory indexes.

QA

25

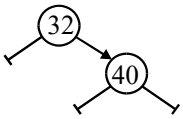
8

If the values `32`, `40`, `20`, `22`, `56`, `18`, `6`, `38`, `52`, `19`, `72`, `29` and `21` shown above are placed in the tree in the order listed, the tree will appear as shown in the slide above. Adding `32` to an empty tree gives the following:

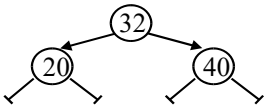
The tree contains one node with no children. The value 40 must be added to the *right* of this node since its value is greater than the value 32 stored in the root node:



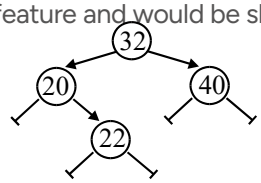
20 must be placed to the *left* of the root node, since 20 is less than 32:



22 is added next. Since this is less than 32, we move *left*; since it is greater than 20, we then move *right*:



The tree shown in the above slide is built up by this process; the resulting tree is quite nicely balanced. You can probably imagine that the tree would not be balanced if the nodes were added in sorted order. That would effectively create a linear feature and would be slow to navigate.



Associative Array examples

- These examples all do the same thing
  - Declare the associative array, dictionary or hash
  - Add the key — value pairs
  - Loop and print each key with its value
- Bash

```
declare -A planets
planets=([Mercury]=57.91
         [Venus]=108.2
         [Earth]=149.597870
         [Mars]=227.94)

for planet in ${!planets[@]}
do
  printf "%-10s %06.2f Gm\n" $planet ${planets[$planet]}
done
```

QA

8

There now follows a number of language examples using associative arrays. The one on this slide, Bash, is similar to ksh93.

Array handling in UNIX shells is, to put it mildly, clunky. It is not used a great deal, most people would use a more powerful language, like Perl or Python.

Associative Array examples

- C++
  - For C++ we also need to declare a special *iterator*

```
#include <unordered_map>

unordered_map<string,double> planets;
unordered_map<string,double>::iterator iter;

planets.insert (make_pair("Mercury",57.91));
planets.insert (make_pair("Venus", 108.2));
planets.insert (make_pair("Earth", 149.597870));
planets.insert (make_pair("Mars",  227.94));

for (iter=planets.begin(); iter!=planets.end(); ++iter){
     cout << iter->first << iter->second << endl;
}
```

QA

27

8

The C++ Standard template Library (STL) contains two different types of associative array. C++ unordered_map uses hashing, map uses a b-tree (ordered by key).

Note that the C standard library does not include such structures.

Associative Array examples

- Visual Basic .Net

```
Dim planets As New Dictionary(Of String, Double)

planets.Add("Mercury", 57.91)
planets.Add("Venus", 108.2)
planets.Add("Earth", 149.59787)
planets.Add("Mars", 227.94)

For Each key In planets.Keys
    Console.WriteLine("{0} {1} Gm", key, planets(key))
Next
```

Associative Array examples

- PHP

```
<?php

$planets = ['Mercury' => 57.91,
            'Venus'   => 108.2,
            'Earth'   => 149.597870,
            'Mars'    => 227.94 ];

$format = "%-10s %06.2f Gm\n";

foreach($planets as $name => $dist) {
   printf($format, $name, $dist);
}
```

Associative Array examples

- Ruby

```
planets = {"Mercury" => 57.91,
           "Venus"   => 108.2,
           "Earth"   => 149.597870,
           "Mars"    => 227.94}

planets.each do |planet, distance|
      printf("%-10s %06.2f Gm\n", planet, distance)
end
```

Associative Array examples

- Perl

```
my %planets = (Mercury => 57.91,
               Venus   => 108.2,
               Earth   => 149.597870,
               Mars    => 227.94);

while(my ($planet, $distance) = each(%planets) ){
      printf "%-10s %06.2f Gm\n", $planet, $distance;
}
```

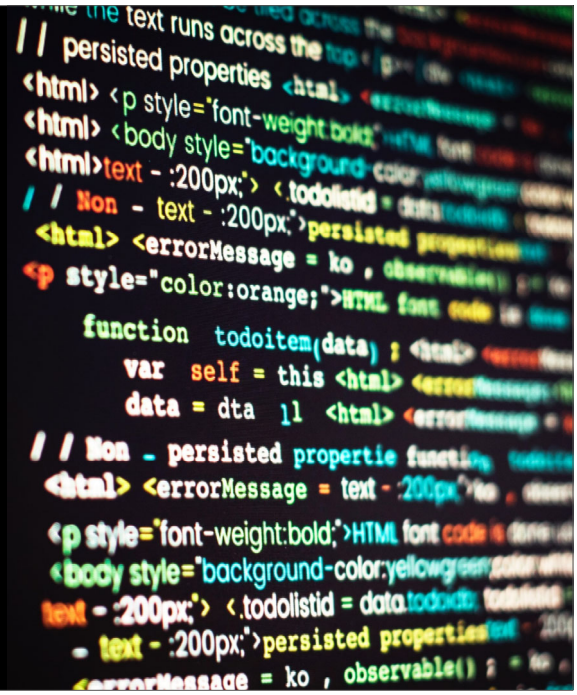
- Python 3

```
planets = {'Mercury' : 57.91,
           'Venus'   : 108.2,
           'Earth'   : 149.597870,
           'Mars'    : 227.94}

for key in planets.keys():
    print("%-10s %06.2f Gm" % (key, planets[key]))
```

Controlling the Execution Flow

Programming Foundations



QA
Learn. To Change.

Controlling execution flow

- Controlling program flow.
- Using if statements to control what executes next.
- Extending if logic to provide alternate conditions.
- Execute code multiple times using loops.

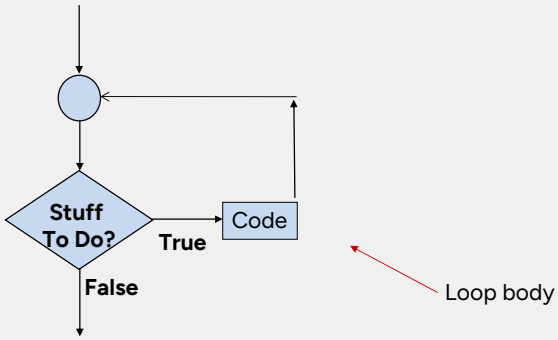
What happens next?

- Most programs need to control what happens next.
  - A lot can change when a program executes
- As things change you may need to:
  - Alter the sequence of instructions you execute
  - Selectively execute certain code blocks
  - Keep processing until you reach a required state
  - Deal with exceptions
- Programming Languages provide control flow instructions to help:
  - For loops to process lists or execute code a set number of times
  - If statements to selectively choose the code to execute
  - While loops to help you reach a required state
  - Exception handling to deal with errors

So far, we have concentrated on the syntax of data declaration and expression evaluation, and have confined the flow of control to be the default, i.e. sequential. Other than library function calls, this has been the case. Clearly, such programs are not very flexible, robust or useful.

For loops

- Repeat execution of the same code
  - For each item in my array
    - Process each item the same way



QA

8

The repetition (or looping) flow allows a sequence of statements to be executed more than once. The *body* of the loop (the statements within it) are executed *while* some condition is true, so this is usually known as a **`while`** loop.

Other loops might be supported by your language. A **`do...while`** loop tests the condition after the body has been executed, so it is always executed at least once. An **`until`** loop will execute until a condition is true.

For loop — example

- Process my array of fruit

```
fruits = ["Apple", "Orange", "Banana", "Pear"]

for fruit in fruits:
    print("Fruit is " + fruit)
    print("Length of fruit is ", len(fruit))
```

- For every fruit in the fruits array
  - Print the fruit
  - Print the length of the fruit
- For loop takes care of retrieving each item in the array in turn, then:
  - Assigns the item to the fruit variable
  - Executes the code inside the for loop (making the fruit variable available to it)

QA 8

The for loop retrieves each item in the array and allows that item to be processed by the code within the loop.

Languages like C# and Java have similar loops.

```
string[] fruits={"Apples","Oranges"};
foreach(string fruit in fruits)
{
   Console.WriteLine(fruit);
}
```

Java would be:

```
for (String item : fruits) {
   System.out.println(item);
}
```

Executing code a set number of times

- How do you execute the same code 10 times?
- In Python we can use the range function to generate a sequence of numbers we can loop through:

```
range(10) # Generates a sequence of numbers 0 to 9
```

- We can use this with the for loop:

```
for num in range(10):
    print(num)
```

- We could replace the code on the previous slide with:

```
for index in range(0, len(fruits)):
    fruit = fruits[index]
    print(fruit)
```

QA

8

In Python we can use a range function to generate a number sequence and iterate through that a fixed number of times. The range function provides us with an iterator which we could use to alter the collection as we iterate over it.

C based languages provide a for loop with the format

```
for(int i=0;i<20;i++)
{
    Console.WriteLine(i);
}
```

Making decisions

- IF it isn't raining and the sun roof is working, then:
  - We can drive to the beach
- Otherwise:
  - Let's stay in and play Scrabble©

- IF it isn't raining or the sun roof is working, then:
  - We can drive to the beach
- Otherwise:
  - Let's stay in and play Scrabble©

- What's the difference between the two sets of rules?
  - What has to happen for us to end up playing Scrabble© or enjoying the beach?

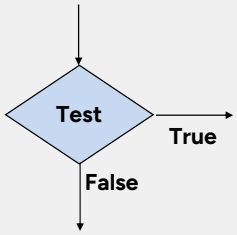
QA

8

Consider the two sets of rules above. How do they differ and consider how you might ender up at the beach using both sets of rules.

Flow control

- To make a decision, we need to be able to test conditions.
- A Boolean statement returns True or False.
    - Yes or no, success or failure, 1 or 0
- We need ways of executing code depending on a Boolean result.



- Executing statements depending on the result of a test is a fundamental aspect of programming.

QA

8

The are two main types of control statements in high-level programming languages, a branch and a loop. Actually a loop is just a special form of branch, we just keep branching to the same point. A simple branch, or jump, instruction tells the CPU to start executing instructions at a new location in the program. This is known as an unconditional branch. Conventional CPUs also have conditional branch statements, such as "branch if a value is zero".

Branching can have a bad effect on performance. Most CPUs look ahead in the program and prepare statements for execution while preceding statements are being run. Instructions may need several steps to execute, running these steps in parallel improves performance. They are still executed one at a time, but each is in a different stage in the pipeline (this is known as pipelining). How will this clever innovation handle a branch? Not very well! It cannot determine what the next instruction is until the branch is executed, so a branch will clear the pipe. Really clever CPUs actually speculate and start executing both possible outcomes, which obviously requires at least two instruction pipes.

What is truth?

- Testing if something is true
- Is the value in the age variable bigger than or equal to 21?
  - age>=21 ➔ True
- Is a person's salary greater than 43000?
  - salary>46350 ➔ False
  - If true they may have to pay 40% tax
- What does true mean?
  - It can mean 1 or -1 or just not 0
- False usually means 0

QA 8

When a logical test is made its outcome is returned using one of two binary states: true or false.

Boolean operators

- Operations that return TRUE or FALSE.
  - Usually used in flow control statements
  - Comparison operators, like == are discussed over…
- In general, the comparisons are symmetrical:
  - For example: x == 42 is the same as 42 == x
  - Some shell comparisons are not symmetrical
- Operators depend on type.
  - Does A equal B?
  - Is MyAge greater than 18?
  - Is MyName "Bill Gates"?
- …and programming language!

```
A == B
MyAge > 18
MyName == 'Bill Gates'
```

QA

8

Boolean algebra was invented by George Boole (1815-1864), but no one found it particularly useful until electronics came along. A Boolean statement is one where there are two possible outcomes, usually termed **true** and **false**. They are mostly used in conditional statements, but can often be used elsewhere.

Some languages (e.g. C++, Pascal, Java, Python) have a Boolean data type, but many others do not. When a variable of a different type is used with Boolean operators then usually zero (or an empty string) means false, and any other value (any bits set) is true.

It can be seen that comparing two variables of different types could have unexpected results, and many languages will not allow it. There is rarely a good reason anyhow, and it usually means someone made a mistake.

One might assume that a single equals sign is used to test equality, but that is not correct in many languages. In many languages two equals signs are used. This is because, in these languages, a single equals sign means an assignment. Inverting the result, turning true into false and false to true, is achieved using the **not** unary operator, often written as !. A few languages (PHP and Perl 6) use *three* equal signs to indicate that each side must also be of the same type.

Comparing text strings often requires different operators. In languages like C a text string is not a fundamental type, so a library function must be called. In un-typed languages like Perl, a variable can be either numeric or text, so we need a different operator depending on the type of comparison we want (the context). Languages like C++ and Python achieve object comparisons by *operator overloading*, that is, using the same operator symbol to operate on different types.

Boolean operators

- Operators can vary between languages.

<	**less than**	*expression* < *expression*
<=	**less than or equal**	*expression* <= *expression*
>	**greater than**	*expression* > *expression*
>=	**greater than or equal**	*expression* >= *expression*
=	**equality**	*expression* = *expression*
==	**equality**	*expression* == *expression*
===	**equality of value and type**	*expression* === *expression*
!=	**inequality**	*expression* != *expression*
<>	**inequality**	*expression* <> *expression*
	object identity is the same	*object* is *object*

QA

8

Different languages use various symbols to test values, and those shown are the most common. Some languages use both the text versions, like **eq** and ne, and the symbol versions, like == and !=, for different types of comparison.

Note that a test for equality is often two, or even three, equals signs, not one, which is reserved for assignment in most languages. Visual Basic, and early UNIX shells, are unusual in using a single = for both assignment and equality.

In most programming languages these operators are *symmetrical*, that is:

```
x == y
```

is the same as

```
y == x
```

Logical operators

- More complex questions:
  - Is MyCarColor red or blue?
  - Is Today Friday and Time greater than 16:00?

`not`	**logical NOT**
`and`	**logical AND**
`or`	**logical OR**

- These are evaluated based on precedence.
  - AND — Stop evaluation when a condition is false

```
condition1 and condition2 and  condition3 and condition4
```

  - OR — Stop evaluation when a condition is true

```
condition1 or condition2 or condition3 or condition4
```

QA

8

A single test may not be enough, so there is a Boolean *and* and *or*. Sometimes we use these words, and sometimes && and || are used instead, this is to make them different from the bit operators. Most languages have one or the other, Ruby, Perl and PHP have both (of different precedence).

The precedence of the logical operators is most commonly NOT, then AND, then OR.

Selectively executing code

- By testing if one or more conditions are true.
  - You can control when specific code executes:

```
if something-is-true and something-else-is-true :
    do something
```

- Else allows you to provide an alternative action.
  - Otherwise, do this:

```
if something-is-true and something-else-is-true:
    do something
else:
    do this instead
```

Example “if” syntax

- What does this do?

```
pin = input("Enter PIN:")
if pin == "12345":
    print("Access Granted")
else:
    print("Denied!")
```

- In Python:
  - Each clause has a : at the end
  - Code to execute if the logical test evaluates to true is indented
- C, C++, Java, C#:
  - Clauses are all surrounded with parentheses ( )

QA

14

8

C#

Console.WriteLine(“Enter PIN:”);

```
pin=Console.ReadLine();
if (pin=="12345")
{
   Console.WriteLine("Access Granted");
}
else
{
   Console.WriteLine("Denied!");
}
```

Creating rules

- Capturing some income tax rules:

```
salary = 49500
highTax = 0
lowTax = 0
```

- The rules:

Not approved by HMRC ☺

```
if salary >= 11850 and salary <= 46350:
    lowTax += (salary - 11850) * 0.2
elif salary >= 46351 and salary <= 150000:
    lowTax += (46350 - 11850) *0.2
    highTax+=(salary-43000)*0.4
elif salary > 150000:
    lowTax += (46350 - 11850) * 0.2
    highTax += (150000 - 46350) * 0.4
    highTax += (salary - 150000) * 0.45
else:
    print "Tax Free!"
```

QA

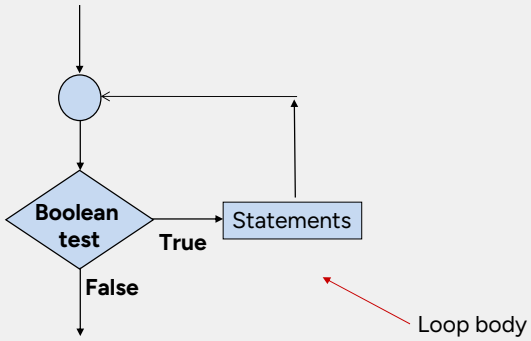
8

Often you need more than one test. If statements let you add multiple clauses to cater for many scenarios. The first one that matches wins and its code gets executed.

Python uses the **elif** keyword to introduce additional conditional tests. Many languages use **elseif** or **else if**.

"While" loops

- Keep executing the same code until certain conditions are met.
  - A bit like a continually executing "if" statement.



QA

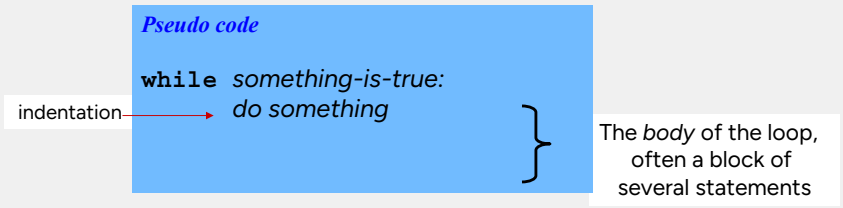
8

The repetition (or looping) flow allows a sequence of statements to be executed more than once. The *body* of the loop (the statements within it) are executed *while* some condition is true, so this is usually known as a **`while`** loop.

Other loops might be supported by your language. A **`do…while`** loop tests the condition after the body has been executed, so it is always executed at least once. An **`until`** loop will execute until a condition is true.

Simple loop statements

- "while" statements have a similar role in most languages…
  - …but differ considerably in detail.
- Basically just a repeated "if".



QA

17

8

The **`while`** loop has a syntax not too dissimilar to that of the simple **`if`**, and the condition syntax will be the same (including logical operators **`and`** and **`or`**). Its flow of control, however, is entirely different. The condition is the first expression evaluated once the compiler has seen the keyword **`while`**. If the test is true, the 'body' of the loop is executed once. The test is re-evaluated and the process is repeated. When the test fails, control goes to the statement after the **`while`** statement.

The **`while`** loop has a 'positive' feel to it. While something is true, keep processing. It should be noted that there must be something within the loop body which will affect the test, otherwise it will be an infinite loop - the loop will go on for ever!

Basic “while” loop

- Keep executing the code in the loop until the test returns false

```
count=0
while count < 20:
    print(count)
    count += 1
```

Keep executing this

This needs to change otherwise INFINITE LOOP

Putting “while” to work

- Keep asking for a PIN until it's correct:

```
masterPIN = "12345"
pin = ""
while pin != masterPIN:
    pin = input("Enter PIN:")
```

- Give some feedback.
  - What about retries?

```
retries = 0
while pin != masterPIN and retries < 3:
    pin = input("Enter PIN:")
    if pin == masterPIN:
        print("Access Granted")
    else:
        print("Invalid PIN")
        retries += 1
```

When things go wrong

- When an error occurs, it needs to be dealt with.
- Errors can crash your application.
- What should you do when an error occurs?
  - Trap it
  - Log it
  - Take an alternate action
  - Retry
  - Tell the user what is happening – did they buy movie tickets or not?
- Throwing and Catching Errors is an important part of programming.
  - Throw errors from your code when you detect a problem
  - Catch errors from code you call

Exception handling

- Traditional error-handling techniques include:
  - Returning a value from a function to indicate success or failure
  - Ignore the error
  - Log the error, but otherwise ignore it
  - Put an object into some kind of invalid state that can be tested
  - Abort the program
- In Python, an exception can be thrown.
  - An exception is represented by an object
    - Usually of a class derived from the Exception superclass
    - Includes diagnostic attributes which may be printed
  - Throwing an exception transfers control
  - The function call stack is unwound until a handler capable of handling the Exception object is found

QA 8

For the majority of programmers, their least favorite chore is testing and handling error conditions. Often there are many ways for something to fail, resulting in exception code that can dwarf the core program logic.

One of the traditional approaches to signaling errors is to return some kind of status value from a function. There are a number of problems with this:

Can lead to cascading if else if code that can be hard to read.

Overloaded operators do not have a 'spare' return value for exceptions.

Return values are too easy to ignore.

One technique is to have an object know whether or not it is in a good or bad state, which can then be queried. The problem with this is that, although it can handle constructor problems, it can lead to clumsy code. It also requires extra members for manipulating and representing the error, making the class less cohesive.

Aborting the program is a last resort. It is up to the caller to determine what the failure policy should be, and for the called component to detect any failure.

The Python solution is to represent exceptions as objects. As objects they can be self describing, whereas something like a simple integer value – as used in other languages – is not very informative as it says nothing about the details of an exception. The next feature of Python's exception handling is that there is a separate path of control for exceptions than for normal function returns. Exceptions can be caught by defining a handler, but there is otherwise no need to detect an exception in a function solely to return it to that function's caller so that it can do the same – you only catch exceptions you are interested in.

Managing exceptions

- Place the code that could error inside the try block.
  - Catch the error in the except block.
  - Multiple except blocks can be used to catch different errors.

```
filename = "foo"
try:
    f = open(filename)
except FileNotFoundError:
    errmsg = filename + " not found"
```

QA

8

Statements within the **`try`** block, particularly when functions are called, could raise different exceptions depending on circumstances. In this case it is possible to test for more than one exception in a structure not unlike a case statement, with multiple **`except`** tests.

The same handler code can be executed by supplying a tuple of exceptions to the **`except`**.

Below is an example of multiple except blocks:

```
filename = "foo"
try:
  f = open(filename)
except FileNotFoundError:
  errmsg = filename + " not found"
except (TypeError, ValueError):
  errmsg = "Invalid filename"

if errmsg != "":
  exit(errmsg)
```

Note: Python uses the **except** keyword to catch exceptions. Many other languages use the **catch** keyword.

Exercise

Question

- Which statement executes the same code until one or more variables reach a certain state?
  - A. for loop
  - B. while loop
  - C. if statement

Summary

- Testing a condition yields TRUE or FALSE.
  - A Boolean result
- The simplest, and most common way to test is with an if statement — execute code if a condition is true.
- Repetition is achieved using a form of while loop — execute code while a condition is true.
- Loops can also be used to access arrays one element at a time.

Example "while" syntax

- These examples all loop 42 times:

- C, C++, C#, Java — Code blocks after the 'while' are marked by braces { }

```
while ( i < 42 ) {
    other processing
    i++;
}
```

- Perl, PHP — Very similar to C etc.

```
while ( $i < 42 ) {
    other processing
    $i++;
}
```

- Bash, Korn shell — These shells support several forms of loop; all are marked by do … done

```
while (( i < 42 ))
do
    other processing
    (( i++ ))
done
```

QA

8

The slide shows a selection of language syntaxes for a **`while`** statement - of course there are many other syntaxes, but these are representative. Others are included in the following slides.

In C, C++, C#, Java, and Perl, the {…} block also represents the limits of scope for variable declarations. A variable declared in such a block is not visible outside the block. This is discussed later.

Perl supports a `continue{}` block, but it is not commonly used.

Some languages also have a form of **while** loop known as a *counting* loop. The first line consists of three parts, an initialisation of the loop counter, a while condition or bound, and a change to the loop counter (usually an increment).

C, C++, C#, Java, PHP, Perl:

```
for (i = 0; i < 42; i++ ) {
    other processing
}
```

Bash, Korn shell:

```
for (($i = 0; i < 42; i++ ))
do
    other processing
done
```

Example "while" syntax

• Visual Basic	The "Then" is optional when the statement is multi-line	`While i < 42`
		`  other processing`
		`  i += 1`
		`End While`

• FORTRAN	The example is FORTRAN 90, FORTRAN 77 would normally be expressed in UPPERCASE	`do while ( i .gt. 42 )`
		`  other processing`
		`  i = i + 1`
		`end do`

• Pascal, Delphi	The begin and end are option if only one statement follows the condition	`while i < 42 do`
		`begin`
		`  other processing`
		`  inc(i);`
		`end;`

QA

8

Contrast against the if statement syntax. Here Visual Basic and Fortran are considerably different.

Visual Basic has a counting loop:

```
for i As Integer = 0 to 41
    other processing
Next
```

FORTRAN also has a counting loop:

```
do i = 0, 42, 1
    other processing
end do
```

In Pascal/Delphi, `Inc()` and `Dec()` (increment and decrement) are extensions to the standard, but are often implemented.

Example "while" syntax

- Prolog

"If you are thinking in loops in Prolog, you're probably thinking wrong"
This is the closest you can come to a simple loop. It uses recursion.

```
whiley(N):- N<42,
    other processing
    M is N+1,
    (M=42;whiley(M)).
```

- Python

Code after the "while" is marked by consistent *indentation*.

```
while i < 42:
    other processing
    i += 1
```

- Ruby

Ruby does have a while loop, as shown, but some form of *iteration* is much more common.

```
while i < 42
    other processing
    i += 1
end
```

QA

8

Prolog does not support the same constructs as other languages, and trying to make appear like it does is not necessarily constructive. In the spirit of consistency we have provided a solution that produces the same result as the others, but Prolog does not have a `while` loop, or any other sort. Iterative (repeating) code is achieved by *recursion*, calling the same block of code by itself. In the example, **`whiley`** is a function name that we made-up, it is called *until* M=42.

Python and Ruby **`while`** loops are very similar to their **`if`** counterparts.

Python also supports an `else:` but is not commonly used.

Old versions of Ruby also supported the trailing :, as Python. A more Rubyish way of iterating than a while loop would be:

`42.times{ |num|` *process num* `}`

Concise, as you can see.

Array processing

- Processing each item is a common task for a loop.
  - Known as iterating through an array
  - A count is kept of the current position — the iterator
- Some loops require the program to maintain the iterator.
  - Must have some way of knowing when to stop!

Stop test (assumes 'size' holds the limit)

```
int i = 0;
while (i < size) {
    printf("Array contains: %s\n",MyArray[i]);
    i = i + 1;
}
```

C/C++

```
for (int i=0;i < size; i++)
    printf("Array contains: %s\n",MyArray[i]);
```

C/C++ Alternative syntax

QA

8

Arrays (and lists) that we saw earlier are often accessed one item at a time using a loop. Typically we might have an integer variable initialized to zero which represents the current index number (position) of the item of interest. So the first time around the loop we can access the element at position zero (the first one). We add one to (increment) the index number, then loop around and access the next element, and so on until we have accessed all the elements in the array. Here is the problem though: we must know *exactly* how many elements are in the array to prevent us from running off the end! In a C or C++ program that could crash the program.

The `for` loop makes use of the four items initialisation, test, body and update. The syntax of the `for` loop groups three of the items in parentheses and separates them with ;. The body is in the same position as the other looping constructs.

The flow of control adheres strictly to that of the `while` loop, i.e. the initialisation takes place (once only), then we go into the test.

Language-supplied iterators

- Some maintain the iterator for the programmer…
  - …which is usually more efficient than maintaining your own.
  - Some languages call this a "for" loop, some a "foreach" loop.
- Examples assume myArray is an array containing data.
  - Each item is held in element in turn.

Python

```
myArray = ['one', 'two', 'three', 'four']

for element in myArray :
   print "Array contains:", element
```

C#

```
string[] myArray = {"one", "two", "three", "four"};

foreach (string element in myArray)
{
   System.Console.WriteLine("Array contains:"+ element);
}
```

QA

8

On the previous slide we saw examples of stepping through an array one element at a time - a common thing to want to do. There were several possibilities of bugs though. We must make sure that we initialize the iterator to zero before we start, and we must remember to increment it, otherwise we would end up with a never-ending loop. We also need to keep track of exactly how many elements are in the array, otherwise we are in risk of running off the end into uncharted memory.

Why not let the programming language do all that for us? It knows exactly where the array starts and where it ends, so allowing the run-time library to do the work is much safer, and often more efficient as well (although that might not always be the case in Visual Basic - http://support.microsoft.com/kb/129931).

Not all languages support this kind of loop, C/C++ does not, but C# does.

For Korn shell 93 and Bash:

```
for element in ${MyArray[*]}
do
   print "Array contains: $element"
done
```

For Perl:

```
for my $element in (@MyArray) {
   print "Array contains: $element"
}
```

For Visual Basic:

```
For Each element As String in (MyArray)
    Console.WriteLine("Array contains: " & element)
Next
```

Structuring and Organizing Code

Programming Foundations



QA
Learn. To Change.

Overview

- Identifying reusable code
- Reducing the scope of data
- Creating functions
- Defining local variables
- Passing parameters to functions
- Returning data from functions
- Function complexity
- Organizing functions into libraries

Writing a program

- You could write a program as one block of script.
  - Single code block (or monolith)
  - Can grow to hundreds or thousands of lines
- Why might that be a problem?
  - Difficult to debug, understand and manage
  - Difficult to reuse code that you write
    - But you could use **copy-paste**

QA

8

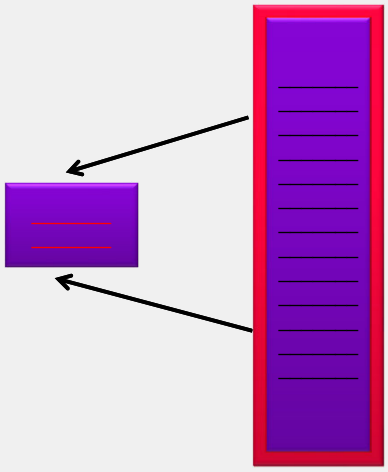
Writing a program as one monolithic script might seem tempting to begin with because everything is in one place and is easy to access.

The problem is that as the program gets bigger things get more difficult to manage and the ability to reuse code already written becomes a challenge.

The other problem is your data. Monolithic programs tend to keep all of their data as global variables. That means they are accessible by every line of code from everywhere. That's not a good idea.

Monolithic code

- Is monolithic code bad?
- What if we duplicated similar code multiple times?
  - Wasted effort, extra maintenance
- Why not extract the duplicate into a reusable block? Then…
  - Call the Reusable block when required



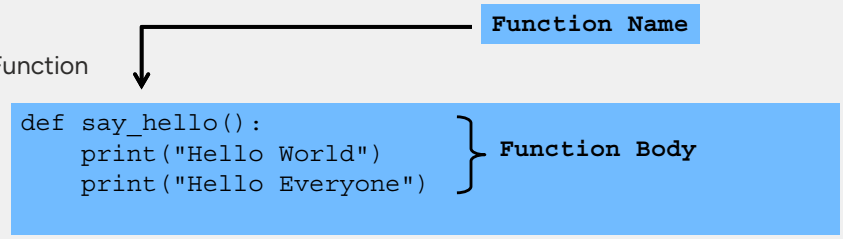
QA

8

Monolithic code makes it hard to reuse common sequences of code and that's one of the reasons why we tend to break our code up into reusable blocks of code (called functions) and organize those functions into libraries and modules.

Reusable blocks of code

- Reusable blocks of code are commonly known as Functions.
- Functions typically have two parts
  - Function name (although they can be anonymous)
  - Body — containing the code to execute
- Simple Function



QA

8

Functions are the cornerstone of code reuse. They are blocks of code that have specific names to identify them. These function names can then be used to execute the code block when required.

Function benefits

- Self-contained, isolated code blocks
- Reusable
- Data defined within is accessible only inside the function
- A function's locally declared data only lives as long as the function executes
- Should perform one provable task
- Has defined inputs and outputs

QA

8

One of the main benefits of functions is they enable code reuse but they are also great for reducing memory usage whilst increasing data isolation and testability.

Executing a function

- Once a function is defined it can be executed (invoked or run)
  - Use the name of the function plus **()**

```
say_hello()
```



```
def say_hello():
    print("Hello World")
    print("Hello Everyone")
```

```
Hello World
Hello Everyone
```

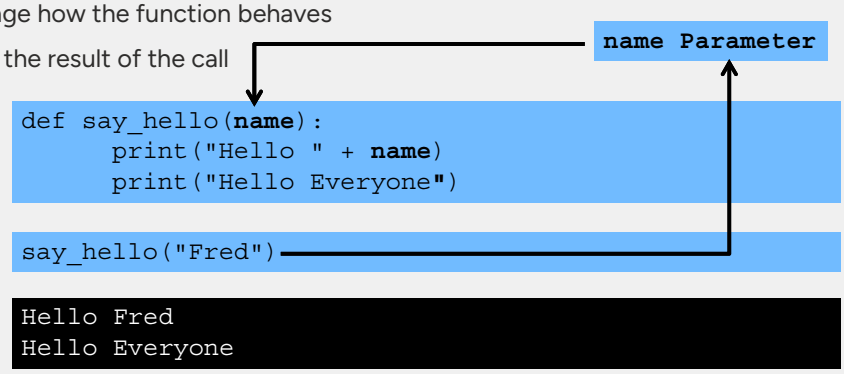
QA

8

To execute a function you use the name of the function followed by round brackets **()**

Making functions more useful with parameters

- say_hello is a bit limited
  - It always says hello to the world, not very personal
- Parameters allow data to be passed into functions which can:
  - Change how the function behaves
  - Alter the result of the call



QA

8

Functions can also have data passed into them using parameters. Parameters allow the caller of the function to pass data into the functions making it more flexible. Parameters are like variables that exist just inside the function.

In the example above say_hello is called and a value "Fred" is also supplied which is passed to the name parameter of the function. That now means that the name parameter contains the value "Fred".

When we print("Hello " + name) we actually print **Hello Fred.**

Multiple parameters

- Functions can have more than one parameter

```
def say_hello(first_name, last_name):
    print("Hello " + first_name + " " + last_name)
    print("Hello Everyone")
```

- Arguments passed by position (in order)

```
say_hello("Fred", "Bloggs")
```

- Arguments passed by name (any order)

```
say_hello(last_name = "Bloggs", first_name = "Fred")
```

QA

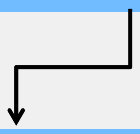
8

Functions can have multiple parameters which are passed in order based on their corresponding position. You can also pass arguments in any order by using the parameter name.

Returning data from functions

- Functions can return data to the caller

```
def say_hello():
    return "Hello World"
```



```
message = say_hello()

print(message) # prints "Hello World"
```

- You need to provide a variable to catch the returned value

QA

8

Some functions just perform an action and return no data. If your function needs to return information it can do that using a return statement.

Statically Typed languages like the C base languages define action and data functions differently

```
void do_something(int x){
   Console.WriteLine(x);
}

int do_something(int x){
   return x*2;
}
```

Writing a useful function

- Acme Bank wants all of their developers to calculate interest the same way so
  - They write a reusable function

```
def calculate_repayment_amount(principal, rate, months):
    repayment = principal * (1 + rate / 12) ** months
    return repayment
```

- To use the function

```
amount = calculate_repayment_amount(5000, 0.045, 12)
```

QA 8

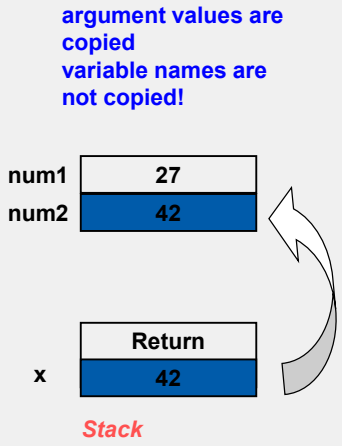
Once you have decided how you want to calculate repayment amount, write it once in a reusable function and use that everywhere. If you change your mind about how interest should be calculated then you just change it in one place.

Passing arguments by copy

- Many languages require:
  - The correct number of arguments
  - The correct type
  - Visual Basic specifies ByVal

```
function MyFunc (integer num2)
begin block
   declare integer num1
   num1 = 27
   print(num1 * num2)
end block

x = 42
call function MyFunc (x)
```



QA

8

Functions may be given values to work with, and these are called arguments, or parameters (strictly speaking the terms mean slightly different things, but in common usage they are the same). Most languages have a similar syntax, the function is called, with arguments specified in parentheses after the function name. Many languages also check that the number of arguments, and their type, are exactly what the function is expecting.

Passing arguments by copy, which is the default action in most languages, will copy the arguments to the local stack used by the function. A stack is a data structure whose items are pushed onto the top for storage, then popped off when on longer required.

New areas of memory are allocated for these values, the actual names of the variables only exist for our convenience when writing the program, they are gone by the time the program runs.

Passing arguments by reference

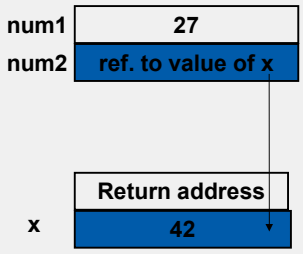
- Passing a memory address rather than copying the data
  - More efficient for large items
  - Some languages *only* pass by reference
  - Visual Basic specifies ByRef

```
function MyFunc (ref integer num2)
begin block
   declare integer num1
   num1 = 27
   print(num1 * num2)
end block
```

```
x = 42
call function MyFunc (x)
```

some languages require special syntax to specify a reference here

references to arguments are copied



QA

8

In order to alter data in another block we need to know where it is in memory. This is generically known as a reference, or pointer. Generally speaking a reference is a high-level abstraction of an item's virtual address, and cannot be manipulated. Pointers on the other hand contain an actual address, and can often be altered and used in arithmetic. References are supported in most high level languages (C#, Java, Perl, Python, etc.), and pointers in low level ones (C, PL/1) – C++ has both. We will use the term *reference* to mean either here.

Many languages require special syntax to use a reference, and it is often a separate fundamental type. Others use only references, and the difference between a reference and an actual data item is hidden.

Passing arguments by reference has an important advantage over by copy, other than that shown. A large data item, such as a record or array, could take quite some time to copy onto the stack, and could use a lot of memory. A reference, however, is a relatively small item (usually one or two words) and can be moved efficiently. Consequently some languages only pass by reference – the programmer has no choice.

Passing by reference has its down-sides. The function, which may have been written by someone else, has access to the caller's stack area and can trash it! To protect against this languages such as Pascal and C/C++ allow arguments to be defined as input only, or constant. The caller can then safely trust the function and avoid changing the caller's arguments if at all possible.

Pedantic note: when passing a reference variable to a function the reference *value* might be copied - a pass by copy. It could be said that many languages do not really have *pass by reference*. However the effect is the same.

Function call syntax

- The syntax varies.
  - You are going to get used to that.
- *Most* languages require parentheses after the function name to call…
  - …even when there are no parameters to pass
  - To call a function in C, C++, C#, Java, Perl, Python, PHP

```
value = myfunc()
```

  - Exceptions are most UNIX shells, Visual Basic and Ruby.
    - In Ruby the parentheses are optional.
    - In Visual Basic you need them to get a returned value.
  - If the brackets are omitted, then `value` now refers to the function.
    - Not all languages support this.

```
value = myfunc
value()
```

?

QA

8

Many languages require parentheses after the function name to call a function even when there are no parameters. The open parentheses is the function call operator. If you just did this:

```
myfunc
```

then no code would be called. Some languages would do nothing (not even report an error).

When used in an assignment, a call is returning a value to the left-side. If you omit the parentheses then all we are doing is assigning the *function* to the left-side. In a strongly typed language this will give a compilation error (depending on the type of the left-side). In a dynamic language like Perl or Python it will actually copy a reference to the function, and the left-side will become callable!

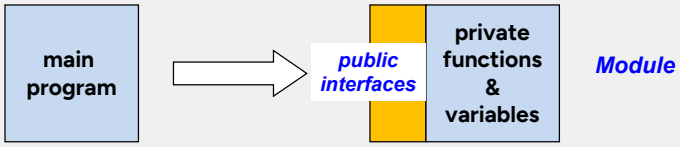
Visual Basic does not support this; it will give a syntax error.

Best thing about functions?

- Write them once, then reuse them everywhere
- Define short-lived local data variables (next chapter)
- Put them in libraries
  - Reuse functions across all your projects
- You avoid re-inventing the wheel
- Functions should be:
  - Well-named
  - Useful
  - Available
  - Reusable
  - Responsible for doing one thing
  - Not reliant on global data
  - Easily testable
  - Not overly complex

Modules and libraries

- Modules are collections of callable functions.
  - Often compiled separately into *libraries*
- Allows teams to work on small areas of functionality.
- Allows code reuse.
  - Useful functions can be used in many *different* programs
- A module is not a complete program.
  - Considered to be a "black-box"
  - Hides the implementation — *encapsulated*



QA

8

Being able to write functions in a program is all very well, but many of the stated advantages can only be realised if these are allowed to reside in separate files. These files are usually called modules, and depending on the system and language, can sometimes be compiled into libraries to be linked into the main program when required.

Not only can different teams work on different aspects of an application, but commonly required routines can be shared between programs, and even processes. A Windows DLL (Dynamically Linked Library) is an example of commonly required code loaded just once into the system and shared between processes. Other operating systems have similar mechanisms. The language run-time library is usually loaded in this way.

Third-parties, software houses, etc. sell add-on modules and libraries, sometimes as an interface to its main product. For example, Oracle supply add-ins to enable SQL to be called from languages (like C) which do not normally support it.

Multiple source files — why bother?

- Increase maintainability.
  - Independent modules can be understood easily
- Functional decomposition.
  - Simplify the implementation
- Encapsulation and information hiding.
  - Easier re-use of modules in a different program
  - Easier to change module without affecting the entire program
- Support concurrent development.
  - Multiple people working simultaneously
  - Debug separately in discrete units
- Promote reuse.
  - Logical variable and function names can safely be reused
  - Use or adapt available standard modules

QA

8

A Python module is somewhat like a separate source file or DLL in C or C++.

However, it is more than that:

Variables can be local to the module

Packages have an independent namespace

For OO-programming, each module can implement a single class

The term *package* is used to indicate a collection of modules, on a local disk or stored on the network. Specifically in Python it is the directory that the modules reside in.

The reasons for splitting-up an application into modules are all based on good structured programming techniques, however the overriding reason is code reuse - why reinvent when someone else has already written the code?

Creating a module or library

- Differs by platform/language.
  - Python Modules are created by saving code to a .py file.
  - In Java you define modules in .java files inside packages.
  - .NET requires class libraries which compile into DLL assemblies.

- To reuse a library or module:
  - Import or reference the relevant one from where you need to use it.

Importing a module

- Surprisingly, use the import command.
  - At the top of your program, by convention:

```
import mymodule
print(mymodule.attribute)
```

Case sensitive, even on Windows

  - Can specify a comma-separated list of module names:

```
import mymodule_a, mymodule_b, mymodule_c
```

  - Can specify an alias for a module name.
  - You have to specify the module name for each call.

```
import mymodule_win32 as mymodule
print(mymodule.attribute)
```

QA

8

We have seen the basics of importing modules already - after all you can't do much in a Python program without using **`import`**. Here are a few more details.

Notice that the case of the module name must match that of the file name. By default this also applies to Microsoft Windows, unless the environment variable `PYTHONCASEOK` is set. We can specify an alias if required, and that is a commonly used feature.

Importing names

- Alternatively, import the names into your namespace.

```
from mymodule import *
```

  - Beware! Risk of name collisions!
- Import specific object name(s):

```
from mymodule import myfunc1
...
myfunc1()
```

How do we know which module myfunc1 came from?

- Or use an alias:

```
from mymodule import \
     (myfunc1 as mf1, myfunc2 as mf2)

mf1()
mf2()
```

Better or worse?

QA

8

Specifying the name of the module for each function call can be tedious, so we can import all the external names on the module into our own namespace. The problem is that this can lead to "Namespace pollution", so instead we can specify exactly which names to import.

If those names clash with existing names within the program (*name collisions*), then we can assign aliases to individual names. However it can then get very difficult to track back which names belong to which module, so choose your aliases carefully!

Note that we can only import public names, that is those not prefixed with an underscore, or those specified in `__all__`.

Namespaces

- A namespace is a space containing names.
- A module or library add-on to a program has its own Namespace.
  - The *qualified* name of the item includes the namespace name.
  - Qualified names are unique within the program.

library1

```
table = "kitchen table"
```

library2

```
table = DatabaseTable
```

```
import library1
print(libray1.table)
```

main program

- Importing unqualified names into your namespace is confusing.
  - Called *namespace pollution*
- Some languages can create separate namespaces.
  - e.g. C#, Java, C++, Visual Basic

QA

8

With most languages, external files containing code (a *code unit*) have their own namespace. Names created in one file can also be used in others without a clash, or *collision*. This enables these code units to be produced by different people, and even by different organizations and companies.

By default, most languages will assume a plain name to be present in the current namespace. So, in the example, in each library we could use the name "table" without any ambiguity. That would not work in the main program unless it too had a name called "table". Notice that to use the names from a library, the main program has to prefix the name with the namespace name. This full name, which should be unique within a program, is called the *qualified name*. The syntax in the example is Python, but the `.` (dot) separator is common. Other languages may use `::` or `->` between the namespace and the name.

Some languages allow names to be exported to a namespace. For example, "table" could be imported into the main program from one of the libraries, so we no longer have to prefix the name with the library name. Although that can be convenient, it can also be dangerous. We can get a *namespace collision* where a name's value from one namespace can overwrite the value in another. It can also make life complicated for anyone trying to read the code, since it might not be obvious that the name has come from elsewhere. Importing external names into your namespace, *namespace pollution*, is generally considered a bad idea.

Recursion

- Functions often call themselves.
  - Many problems are recursive, particularly in a hierarchy.
- Simple example in Python:

Loop

```
i = 0
while i < 10:
   print(i)
   i += 1
```

Recursive function

```
def to_ten(i):
  print(i)
  i += 1
  if i < 10: to_ten(i)

to_ten(0)
```

Python function

- Most solutions involving recursion can be done using a loop…
  - …but the recursive solution is usually more elegant.
  - The danger with recursion is that the call-stack can overflow…
    - …which will crash your program.

QA

8

Functions which call themselves are known as *recursive*, or *reentrant* functions. The example shown is not a good reason to use recursion. It's just to give you an idea of how it is achieved. You can probably see that some of the code is common between both solutions, and that in this case recursion does not make the solution simpler or more elegant.

The main downside of recursion is that each function call extends the call-stack, which keeps track of return addresses and often parameters passed. This means the call-stack keeps growing, and can slow things down by using a lot of memory. Eventually it will hit a limit (which varies) and will crash the program resulting in a *stack-overflow*.

Recursion example

- Walking a directory tree (Python)

```
import glob, os.path

def list_dir(dirname):
    print("Searching", dirname, "...")

    for fname in glob.iglob(dirname + '/*'):
        if os.path.isdir(fname):
            list_dir(fname)
        else:
            print(fname, os.path.getsize(fname))

list_dir('.')
```

`glob.iglob`	gives an iterator of files by pattern (*globbing*)
`os.path.isdir`	returns True if the name passed is a directory

QA

8

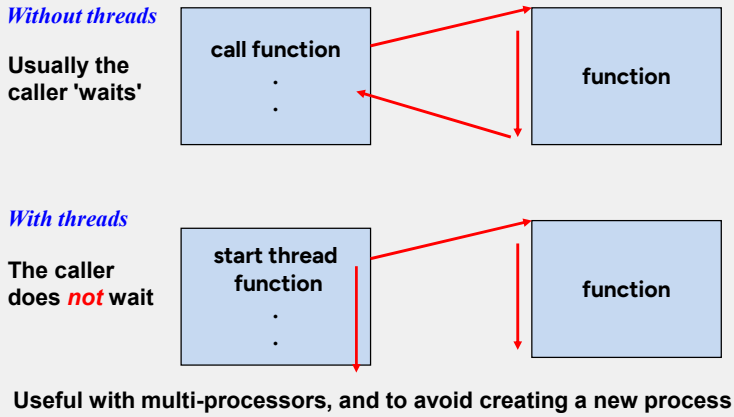
This is a typical use of a recursive function, used in a hierarchy - a directory (folder) file system. In the example we are starting from the current directory (`list_dir('.')`) and getting an iterator of the files stored there (`glob.iglob`).

If the file is a directory we call `list_dir` again, this time passing the name as a parameter. Otherwise we just display the name along with the file size (`os.path.getsize`).

An iterator is a special device which returns one value at a time, as required, rather than the whole collection / list in one great lump.

Asynchronous functions — Threads

- A thread is a function which executes *at the same time* as the caller.



Useful with multi-processors, and to avoid creating a new process

QA

8

Normal execution of a program is serial: a program executes its instructions one after the other. Sometimes we want to do more than one thing at the same time. This can be done just by starting another program, but that can be expensive in computer time and resources. A thread is a little like running a program within a program. It is faster to start-up because we do not have to create another process. We can share data between threads without the kernel being involved, so that is more efficient. A typical example might be where a program is displaying multiple windows on a screen, while one window is waiting for something to happen the others can be busy.

Multi-threading has its down-sides. When programming threads we have to remember that there is no memory protection between the threads of execution, so if one function overwrites memory that might affect the others. This can mean that areas of memory have to be 'locked' to allow concurrent threads to access them without harm. Many other functions have a process-wide effect and the programmer must be aware of these.

Using functions on other computers

- Can we execute functions resident on other computers?
  - Yes — aka Remote Procedure Calls
- Various implementations provide access to remote functions.
  - Remote Procedure Call — RPC
  - Web Services
  - REST Services
  - Micro Services
  - Messaging Services

Exercise

Question

- Which of the following are advantages of functions?

  A. Fast

  B. Memory efficient

  C. Reusable

  D. Easier to test

Summary

- Functions help to structure a program
- Functions are reusable blocks of code
- Libraries or modules can contain collections of functions
- Functions can contain their own private data
- Parameters allow data to be passed into functions

Code used by more than one program

- Generally called a module, library or package
  - Usually independent of the main program

Language	To load external code
Bash, Korn shell	`. name`
Visual BASIC	`Imports name`
C, C++	`#include <name>`
C#	`using name;`
FORTRAN 2003	`IMPORT name`
Pascal, Delphi	`uses name;`
Perl	`use name;`
PHP	`include 'name';`
Prolog	`import_module(name)`
Java, Python	`import name`
Ruby	`require 'name'`

QA

8

Most programs connect to required external libraries when they start running, but this is not always the case. Many languages have features available to connect to libraries on-demand at runtime - which might reduce the size of the page tables in the process.

The statements shown are the basic commands required by these languages. Many have variations and caveats, but those are best discussed in a course dedicated to one language.

C/C++ also requires linker options to include other code.

Perl also has a `require` statement (the difference is the timing of the load).

PHP also has `require`, `include_once`, and `require_once`

Prolog importing is rather more complex than is implied here, but those complexities are best discussed on a Prolog course.

Python, and others, have several variations on the `import` command, it is the simplest that we see here.

Ruby also has an `include` statement which embeds code into a *class*.

Bash (not Korn) also has the `source` command, which comes from the C-shell (csh or commonly tcsh).

Arguments and parameters

- Parameters are defined in the function.
  - Many languages allow default values.
- Arguments are the values supplied by the caller.
- Most common type is positional.
  - Arguments have to be supplied in the correct order.
- Some languages support named parameters as well.
  - Often as an associative array.
- Positional are the simplest (Python example):

```
def myfunc(a, b, c):
    while c > 0:
        c -= 1
        print(a * b)

myfunc('hello', 2, 3)
```

Parameters

```
hellohello
hellohello
hellohello
```

Arguments

QA

8

A function that cannot get data from the outside would be very restricted in what it could do. Fortunately we can pass data into a function, usually provided it is of the correct type. The words "arguments" and "parameters" are often confused and interchanged in everyday speech. Parameters are the local view that the function has of the data, and arguments are the values passed by the caller.

Many languages also support default parameter values, including PHP, Python, C++, and Visual Basic.

The example shows a Python function. It has three parameters, a, b, c. It uses c as a loop counter, and multiplies a and b together. The caller passed three arguments, 'hello', 2, and 3. Notice that these are comma-separated and enclosed in parentheses ( ), which are mandatory in most languages. In Perl and Ruby the parentheses are optional, and in UNIX shells they are not supplied at all.

Returning results

- Functions can have a value and type.
  - Most types can be returned.
    - If nothing is returned, this is usually called a void function.
  - Calls can be used anywhere that the type may be used.
    - In formula, print statements, function arguments, etc…
    - Returned values generally do not use the stack.

```
integer function MyFunc(integer num2)
begin block
   declare integer num1
   num1 = 27
   return num1 * num2
end block
```

```
x = 42
y = function MyFunc(x)
```

Use arguments for inputs and return for outputs

QA

8

Functions in most languages can return a result other than by altering an argument. Some languages have both action functions and return-value functions. Functions that do not return a value are known as *void* (which is often a reserved word).

The call itself can be used anywhere that a variable or constant of that type can be used. For example, a function returning an integer may be used as an index to an array. Be careful of "nesting" function calls unnecessarily, it can make code unreadable and difficult to debug.

The returned value is usually not stored on the stack (it is often in a hardware register instead).

Overview

- What is data scope?
- Reducing the scope of data
- Global data
- Local data
- Block data
- Scope and Scope Resolution
- Constants

Who can see your information?

- When you post a message or photo to social media you choose:
  - Who sees it
    - Public Post — everyone can see it
    - Just Friends
    - Friends of Friends
    - Private Group
    - Just Me
- You control the scope that posting has — either:
  - Wide — Public
  - Narrow — Just me
- Public Posts
  - Can be liked and commented on by everyone
- Just Me Posts
  - Only accessible by you — Total Privacy

When you use social media you have the option to control how wide an audience gets to see the post. You control the scope of the post.

Controlling the scope of data

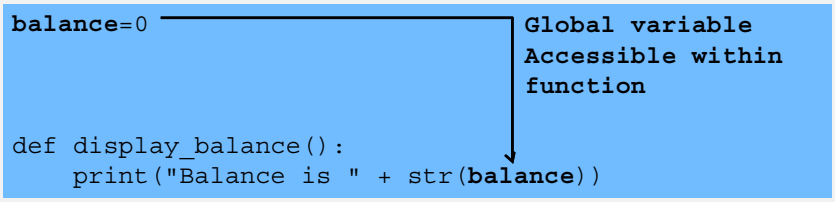
- Data you declare in a program also has SCOPE.
  - Global Variables — accessible by every line of code in the application
  - Function variables — exist only inside functions as they execute
  - Block variables — exist just inside if blocks and for loops etc.
- Question:
  - What's good about global variables?
  - Is there anything bad about them?

The same goes for data in a program. Where you define your data dictates which parts of your application can access it.

Global data can be accessed from everywhere whereas function data is only accessible from within the function it appears in.

Accessing variables within scope

- Global variables can be accessed by all code.
  - Including code within functions.



- Example:

```
balance = 99
display_balance()
```

```
Balance is 99
```

In the above code we have a balance variable that is accessible from within the display_balance function. The balance variable is effectively global.

Accessing variables within scope

- Global variables can be accessed by all code
  - Including code within functions

```
balance = 0


def display_balance():
    balanceString = str(balance)
    print("Balance is " + balanceString)
```

Local variable accessible only within function

- Example

```
balance = 99
display_balance()
print(balanceString)
```

balanceString doesn't exist!

By contrast the balanceString variable declared within display_balance is a local function variable. It will only exist as long as the display_balance function executes.

If we try and print balanceString after having called display_balance we would get an error because it only existed inside the function while it was executing.

Function scope

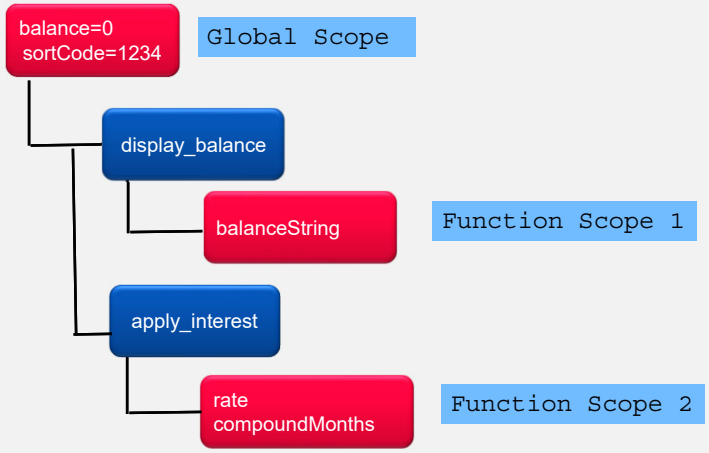
- Functions can see data declared inside their container and globally.
- Function data is hidden to the outside world.
- Function data usually only exists while the function executes.
  - Can help reduce an application's memory footprint
  - Data can be allocated in a function that lives beyond the function’s execution

Functions can see data declared inside their container and globally. Data defined inside a function is private to that function and typically exists only as long as the function executes.

A function can have data passed into it via parameters and return data using returns statements.

Scope resolution

- Functions have access to their container scope



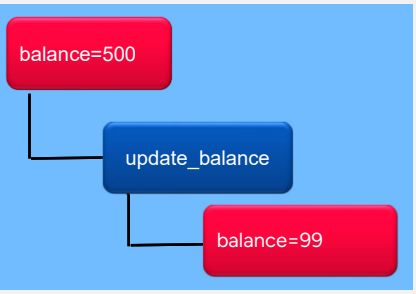
When the display_balance executes it has access to global variables balance and sortCode and function variable balanceString. When it finished executing balanceString is lost from memory leaving only variables balance and sortCode.

Redefining global variables in functions

- This code has an unexpected effect when it runs

```
balance = 500

def update_balance():
    balance = 99
    print(balance)
```



- So what gets printed?

```
update_balance()
print(balance)
```

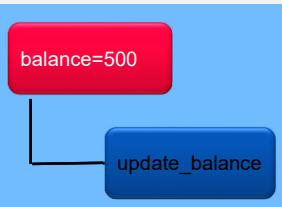
In this example Python creates a new local variable also called balance inside the update_balance function. It prints that variable then exits destroying the local variable. Once update_balance has finished executing only the global balance variable remains which is the one we print.

Managing scope

- Local scope always wins.
  - A local variable declared within a function always takes precedence over a global variable with the same name.
  - Global Variables aren't overwritten by local variables.
- What if you want to update a global variable from a function?
  - Python needs to distinguish between local and global variables.

```
balance = 500

def update_balance():
    global balance
    balance = 99
    print(balance)
```



C# produces an error if you attempt the below code:

```
decimal balance=500;

void update_balance(){
   decimal balance=99;
   Console.WriteLine(balance);
}
```

The error produced is:

“A local or parameter named 'balance' cannot be declared in this scope because that name is used in an enclosing local scope to define a local or parameter”

The alternative would be to omit the type declaration of balance that would enable the higher level balance variable to be updated.

```
decimal balance=500;

void update_balance(){
   balance=99;
   Console.WriteLine(balance);
}
```

```
update_balance();
Console.WriteLine(balance);
```

Block level variables

- Some Languages allow variables to live inside a smaller scope inside specific code blocks.
  - Includes C, C++, C#, Java
- Code blocks typically include:
  - If, while, and for loops
- Any variables defined within these code blocks exist:
  - Only during the execution of that block of code
  - Are not accessible to the rest of the function

C, C++, C# and Java allow you to define variables within the scope of programming structures like if, for, and while loops. These block level variables conform to the same scope rules in that they exist only in the block in which they were defined (e.g. the if, for or while loop).

Global variables — just misunderstood?

- No!
- Global Variables can be accessed potentially from any part of the application.
- That means:
  - Different parts of your application are dependent on the same data.
  - Anything that reuses the dependent code will have to know to make sure the global data is set up correctly first.
  - Multiple parts of the system can change common state at any time and make it very difficult to understand state and debug.
- Much better to create loosely coupled independent code that:
  - Isolates its own data
  - Allows data to be passed to it safely when required
  - Returns relevant data back to callers

Are global variables just misunderstood? Well...no. Not really. They do make your code more co-dependent and introduce a potential unpredictable element to the application.

Your goal should be to write loose coupled reusable code that doesn't rely on globals.

Constants

- Many languages allow the definition of constant data items.
- Constants are data items that can't be changed once they are defined.
- They can be:
  - Global
  - Local
- Global and Local constants are fine because their state is predictable because they are constant.
- Example — C#

```
decimal vatRate = 20.0M;
vatRate = 22.5M;
```

```
const decimal vatRate = 20.0M;
vatRate = 22.5M;
```

Many languages allow you to define data items as constant. That means once you have initialised their value they subsequently can't be changed.

You can use constants for any data items whose value you know at compile time. Often these are known physical, mathematical or rule based values. VAT, Gravity, Avogadro's number.

Note: The 'M' suffix is used to define the literal values of 20.0 and 22.5 as decimals. Without the suffix the C# compiler assumes the literals to be doubles.

Alternative to Scope

- Data can be allocated to the Free Store or Heap.
  - Allocated data becomes long lived and requires "freeing" from memory
  - C, C++, Objective-C allow direct data allocation to heap/free store
  - Allocated data is accessed via data pointers
  - Care must be taken to deallocate data or a memory leak will occur
- Java, C#, VB.NET, Python, etc. allow the creation of Objects.
  - Objects are automatically allocated onto a free store/heap
  - A Garbage Collector processor is used to free up unused objects
  - Objects are accessed via reference variables that point to the objects

Many languages also allow you to allocate data onto a free store or heap. When you do this you remove the data item from normal scope and allow it to exist beyond scope boundaries. Pointers or references are used to maintain contact with the created data.

Java, C#, VB.NET, Python allow you to create objects which are typically allocated onto a heap, accessed via references and disposed of using a garbage collector. You will look at this in the next chapter.

The story so far

- Global Variables and Global Functions with Modules

cars.py

```
speed = 0
reg = ""

def accelerate(speed_to_add):
    global speed
    speed += speed_to_add
```

```
from cars import *
accelerate(10)
```

- Effectively there is only ever one Car (per import) to work with.

In Python we can define scope within a module as we have in the cars.py module.

When we import cars we get the speed variable and the accelerate function but we only get one of them. We can accelerate the car and record its speed but what if we wanted multiple cars?

Ideally we need to be able to define Cars each with their own isolated identity and data scope. That is kind of what objects are all about.

How can we?

- Allow multiple Cars to exist each with their own:
  - Speed
  - Registration Number
  - Ability to Accelerate independently of each other
- Answer:
  - Structures and Classes allow — Compound Data Types
  - Can contain Data and Behavior (functions)

Introducing classes

- A Cake Recipe is a template for baking a cake.
  - Contains ingredients and instructions
  - Follow the recipe and create a cake from it
  - You can use the same recipe many times to create multiple cakes
- Recipes are templates for building edible things.
  - In programming we use Classes not recipes
- Classes describe how to build objects and contain definitions for:
  - Data — Things an object knows
  - Functions — Things an object can do

Classes are a way of defining a template that a programming language can use to create instances of individual objects.

Defining a class

- Defining a Car Class:
  - Including an Initialize function

```
class Car:
    def __init__(self):
        self.speed = 0
        self.registration = ""
```

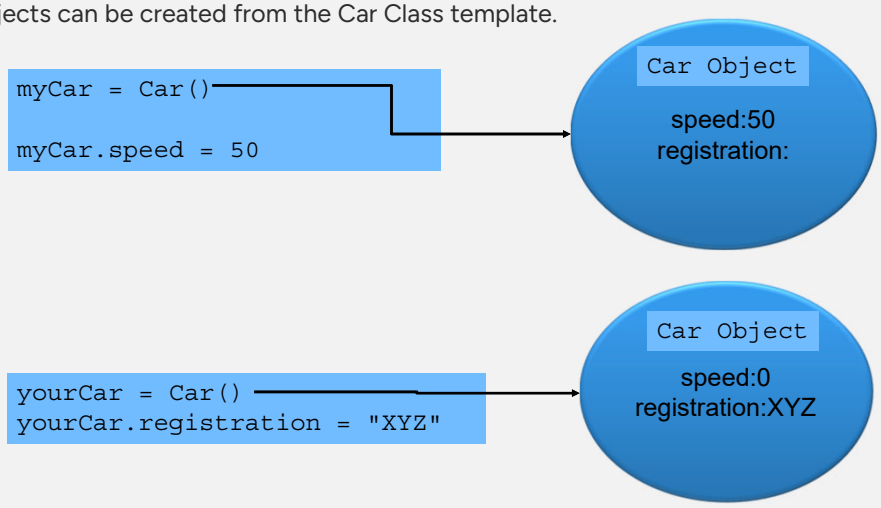
  - Car Class sets up member variables:
    - Speed
    - Registration

To define a class you use the **class** keyword and then provide and **__init__** function. Inside this initialisation function you can define the data fields that belong to each instance (object) of the class.

In the code above we are describing a Car with a speed field and registration number.

Creating a Car object from the Car Class

- Car objects can be created from the Car Class template.



Once you have defined your template you can use it to create instances of Car objects in memory.

Above we have created two separate Car objects and set the speed field of one and the registration field of the other.

The important thing to remember is we have defined two distinct objects with their own unique identity in memory storing their own separate data.

Adding functionality to objects

- Add Functions to the Car Class.
  - Functions can access the data defined in the Class:

```
class Car:
    def __init__(self):
        self.speed = 0
        self.registration = ""

    def accelerate(self):
        self.speed += 5

    def brake(self):
        self.speed -= 5
```

- Car can now accelerate and brake.

As well as defining data, classes can also define functions to operate on the data.

Above we have added an accelerate and brake function.

Creating and operating multiple cars

- A class is used as a template to create:
  - Separate instances of objects
- Each object holds its own:
  - Data State
  - Unique identity
  - Location in memory

```
myCar = Car()
myCar.accelerate()
print(myCar.speed) # Prints ?
```

```
yourCar = Car()
yourCar.accelerate()
yourCar.accelerate()
print(yourCar.speed) # Prints ?
```

Calling accelerate on one of your Car objects will change the speed of just that car.

Object-oriented programming

- Identifies the objects that interact within a system:
  - Customer, Account, Transaction, etc.
- Analyzes the data and behaviors exhibited by those objects.
- Captures data and behavior in classes.
- Applies principles of:
  - Encapsulation — hides data from direct access via functions
  - Inheritance — allows new classes to be created as refinements of exiting ones
  - Polymorphism — helps developers to maximize the flexibility and reusability of software components

This is just the beginning of Object Oriented Programming which extends these basic principles with ideas such as encapsulation, inheritance and polymorphism.

Exercise

Question

- Variable with the smallest scope is defined as:

  A. Global

  B. Block

  C. Function

Summary

- Global Variables can be accessed and changed from everywhere.
- Function Variables are accessible only within a function.
- Functions can return data they create internally.
- Functions can be passed data they need via parameters.
- Block level variables exist only as long as the code block executes.

Example: C# Class

- Defining a Car Class

```
class Car
{
    public int speed {get;set;}
    public string registration {get;set;}

}
```

Overview

- Volatile and Persistent Storage
- Accessing the File System
- Other Storage Locations

Random Access Memory is volatile

- Your data will not stay in memory forever.
- At some point you will need to save it somewhere.
- But where?
  - Disk/File
  - Local Database
  - Cloud Resource



QA

8

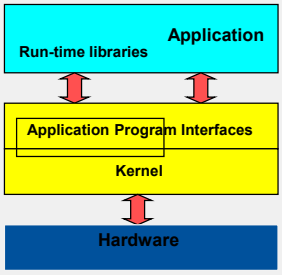
So far we have been storing data in memory. The problem is once you turn the power off or your program unloads all of the data in memory is lost. That's because RAM is volatile.

To preserve your data you will need to persist it somewhere. The simplest location is on a local disk as a file but more sophisticated options are available including local databases plus many remote storage options.

What is a file?

- Usually a physical area of disk.
  - Operating system and/or libraries usually hide the format.
    - Floppy disk, CD-ROM, Flash card, FAT, NTFS, ufs, etc...
- Input (read) and Output (write).
  - Basic operations.

Device drivers provide the kernel with hardware-specific instructions



QA

4

8

Most people have a rudimentary idea of what a file is, and where a file is stored. From a programmer's point of view almost any computer system resource can be viewed as a "file". A file is merely a source or repository of data.

The Application Programming Interfaces for files perform simple operations like read and write. These are the same basic concepts for many hardware devices. We provide a device with data (write) or get data from it (read).

The action of drawing a character onto a VDU screen is quite complex. We really do not want the effort of having to know which pixels to switch on to draw the letter 'A'. It is much easier to pass the character to a library to let it do the work. Writing to a screen is sometimes known as "printing", even though there is no paper or ink in sight. This is because in the early days we used electronic typewriters - the switch over to VDUs was invisible to many applications because we just got a new set of interface libraries.

Reading from a keyboard should similarly be straightforward. We should not need to test to see if the <shift> key has been pressed to decided if the user typed 'a' or 'A'. The libraries do that for us. Many systems, for example Microsoft Windows, allow this level of control, however it is very complex, labour intensive, and distracts from the main purpose of the application.

Passing data across a communications line can also use ordinary file interfaces, although we usually have more work in making the initial connection. Just as we generally write a record to a disc file, we can pass records to another application running on a remote machine. The details of the protocol should be none of our concern. We should only need to specify which to use when the connection is made.

File systems

- Operating Systems use different disk formats:
  - Microsoft Windows
    - File Allocation Table (FAT)
    - Windows NT® File System (NTFS)
    - Case insensitive filenames (but case persistent)

      C:\WINDOWS\SYSTEM\ACLEDIT.DLL
  - UNIX
    - System V (S5)
    - Unified File System (UFS)
    - and others ...
    - Case sensitive filenames

      /usr/bin/ksh
  - Linux
    - ext2/ext3
    - ReiserFS
    - and others...
    - Case sensitive filenames

      /usr/bin/ksh

QA

8

An Operating System's file system determines how file names are stored, and how the data is physically arranged. File names are usually stored in containers called directories, folders, or libraries, and the syntax varies between file systems.

How are files stored on the disc? Are they on a contiguous area of the disc? Probably not, for space efficiency they may be scattered in fragments. Fortunately we do not have to worry about any of this, the lower levels of the operating system handle it.

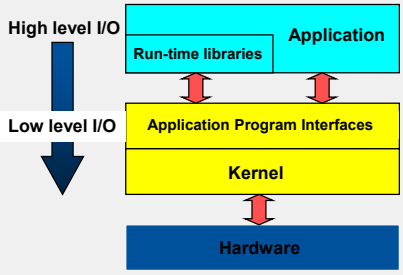
There are various implementations of file systems, and most are specific to manufacturers and operating systems.

Microsoft Windows supports at least two versions of the File Allocation Table (FAT16 and FAT32). These are considered to be obsolete, but still commonly used for USB mounted memory sticks. NTFS (NT File System) is a more efficient, resilient file system, but again there are different variants, almost each release of Microsoft Windows NT brings with it a new one. For example, NTFS5 came with Windows 2000, and was a *journaling file system* - where a track is kept (the journal) of pending writes to speed up error recovery.

UNIX, being older, has even more file system formats, and the two most common (s5 and ufs) are shown. Most versions of UNIX have their own "native" formats as well. The fact there are differences does not matter unless files are accessed by more than one operating system. Linux supports UNIX and Microsoft formats (as well as Linux native) for dual-boot systems, which often run Microsoft Windows and Linux. Of the Linux file systems shown, ext3 is a *journaling file system*, and ReiserFS is tuned specifically for large numbers of files.

I/O libraries and layers

- High-level I/O:
  - Language or package run-time libraries
  - Abstract the idea of a "file"
  - Make file I/O easy to use
  - Use own buffering for efficiency
  - Often the same on all operating systems (portable)
- Low-level I/O:
  - Operating system APIs
  - Greater control
  - Needs specialist knowledge
  - Often **not** portable



QA

8

A programmer's view of Input/Output depends on a number of factors. If the I/O to be performed is fairly standard then the programming language probably provides all that is required. The run-time library takes care of the detail, and often provides a layer of buffering for efficiency. Where a language does not provide the functionality required, a third-party software package might be used as well.

There are a number of advantages to using high-level I/O. The underlying layers are finely tuned for efficiency and should be robust. The programmer does not need detailed knowledge of the operating system or the hardware, which makes the program code portable to other computers. Less work for the programmer also means shorter development time and less code should mean fewer bugs.

Low-level I/O is the opposite. The programmer needs detailed knowledge of the operating system or hardware, and the program will probably not run on different computers. Buffering may have to be provided by the application, and interrupts handled correctly. More work for the programmer means longer development time and maybe more bugs.

So why use low-level I/O? The main advantages are control and functionality. The programmer can use unique features of the operating system not supported by portable languages. A specialist task may simply be unavailable in a 'one-size-fits-all' run-time library. Not using multiple layers of intervening code can make a program much faster, but can have the opposite effect if the programmer is not able to drive the operating system in the most efficient manner.

The general rule is to use high-level I/O whenever possible.

File data types

- Text — ASCII or Unicode, UTF-8 is very common
  - Advantage: mostly portable and easy to use
  - Disadvantages: contains little or no structure, can be slow
- Binary — language dependant, but can be converted ("unpacked")
  - Advantages: precise and can contain structure, fast
  - Disadvantage: laborious to unpack — requires low-level detail
- Serialized data interchange
  - Text — language independent, but slow
  - Can contain structure
    - JSON — JavaScript Object Notation
    - XML — Extensible Markup Language

QA

7

8

Old computer systems had very complex file formats, but most systems today have very simple ones. Essentially a file's format depends entirely on how it is to be used, in other words it depends on the *application*.

Since all files are represented as a series of binary-digits (bits), then you could say that all files are binary. However this term has become to mean files that cannot be interpreted immediately as text. The advantage of binary (non-text) files is that they can hold values such as floating-points accurately, and there is no conversion required from text to machine formats - so they are faster and use less disk space. The disadvantage is that they are not necessarily portable, and they are a little more difficult to maintain.

We know now that what text is depends on several things, including the locale of the observer, so what is a text file is actually more complex than it first appears.

Opening a file

- I/O generally requires calls to either:
  - Kernel APIs
  - Run-time libraries
- Generates a temporary *reference* to the file
  - Used for fast access
  - Sometimes called a *File descriptor*, *Handle*, or *File Object*
- Opening a file is expensive
  - Security checks usually performed on open
  - Could require network access
  - Always requires some kind of search

QA

8

Before we can perform operations on a file, we generally have to open it. There are a few languages where this is not required (mostly shells) but this is not typical.

Opening a file performs several tasks. Obviously the file must be found, and the operating system may have to search several layers of directories (folders, or libraries), or even make network requests if the file resides on another machine. It then has to determine if we are able to use the file in the way we have specified (see next slide).

We really do not want to perform these checks every time a record is read or written as they can be very expensive. Generally most languages return a "short-cut" to the file on a successful open. This short-cut, also called a file descriptor, file handle, or file object, can then be used to identify the file for subsequent actions. Since we have a valid short-cut, the operating system remembers which operations we are allowed to do, where the file is, and our current position within the file.

Opening a file — checks

- There are many reasons a file open may fail.
  - Security check could fail
  - File name could be wrong
  - File may be *locked*
  - *Always test the result of an open!*
- What to do if a program cannot open a file?
  - Does it depend on the error?
  - Can you ask the user for a different file name?
  - Can you continue to run without that file?
- On error, always report (at least):
  - The full filename
  - The system error number or message

QA

8

There are many reasons why an open request may fail. For example, opening a file for writing requires that we have write permissions on the file. It may also mean that we must have exclusive access to it. Depending on the method used, writing to a file usually involves preventing other users from accessing it at the same time - known as locking.

It is important that we do not assume that we can access the file. We should always test to see if the file was successfully opened before we try to use it. Most systems return a well-known error value in the short-cut on failure, this might be -1, false, or NULL.

Opening a file — modes

- Required use must be specified at *open* time:
  - To check security permissions
  - To check interaction with other processes
  - To set the starting position for read or write operations
- Sequential Access:
  - Access the file record-by-record, in order
    - *read* from start of file file is unchanged
    - *write* from start of file file is created or truncated
    - *append* from end of file file is extended
- Random Access:
  - Often opened for read *and* write
  - Set position: *seek*, get position: *tell*

QA

8

When we open a file we must say what we are going to do with it so that the operating system can decide if the request is legal.

Opening a file for read puts us at the start of existing data, but does not alter the file.

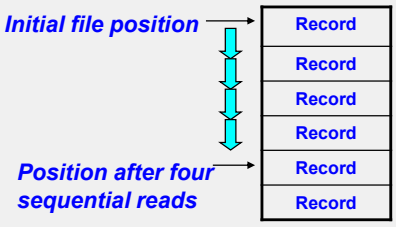
Opening for write usually destroys any existing data, or creates a new file if it does not exist. Special modes may be required if we do not want that to happen. The current file position is also at the start of file.

Opening a file for append allows us to write to the file but the records are added onto the end of any existing data. The file is usually created if it does not exist, and the file position is at the end of file.

For complex file updates these simple modes are not enough. Sometimes we want to alter data in situ, or we may want to access data by key. In these cases we need to know where each record resides in the file, a bit like a memory address, but this time we use the offset in bytes from the start or end of the file. We can get the record position by using a tell function and often they are stored in a file, known as an index, for later use. Once we know the position of a record, we can go straight to it using seek.

Sequential access

- Common with *text* files.
  - Also known as *flat* or *sequential* files
- Each successive read or write advances the current file position by one record.
  - The I/O library reports when End-Of-File is hit
- The most common form of file I/O.
- Also the easiest.



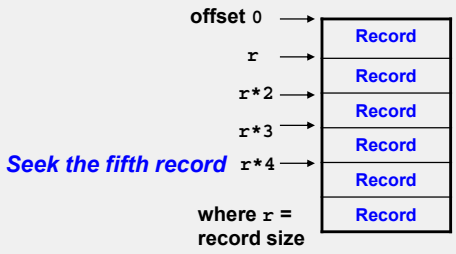
QA

8

Sequential access is often used for files which consist of lines of text. Many IO systems consider one line of text to be a variable length record. When we open the file for read the current file position is at the Beginning Of File (BOF, also known as BOD, Beginning Of Data). Each time we read a record the current position is advanced by the number of characters read. This is fine when processing files in the same way as reading lines on a page.

Random access

- Common with *non-text* (“*binary*”) files.
- We get the current file position:
  - Usually by a *tell* operation
- We can set the current file position:
  - Usually by a *seek* operation
  - Usually to an offset relative to:
    - Beginning of file
    - End of file
    - Current file position



QA

8

If we only want one specific record, then we have a problem with sequential access. Imagine trying to look up a word in a dictionary in this way. Looking for the definition of 'Zebra' would involve reading every single word in the dictionary from A until we found it. We do not look-up a dictionary in that way. We know that there is a built-in sequence and look for the section starting with the letter 'Z'. We use the first letter as a key.

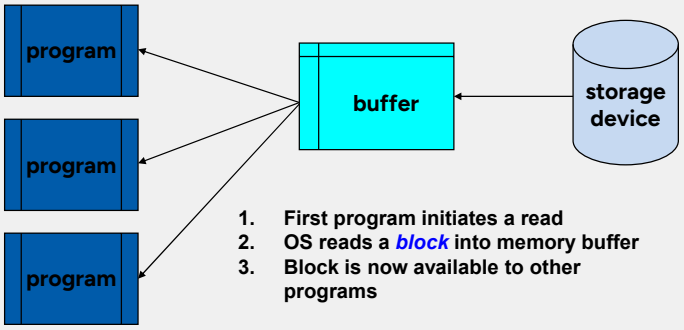
When we first create a file to be accessed by key, we create an index. Within the index each key is saved, along with its position in the file. When we wish to find a record by its key we only need to search the index, which is relatively small, rather than the entire data file, to get the offset. Indexes can be searched sequentially, or using a "binary chop", but usually a more complex algorithm is applied to find the key. Often the index is small enough to fit into memory, making the search faster still. This is the basic principle used by Database Management Systems.

With fixed length records we can find a specific record if we know its sequential position, its offset is the record size multiplied by the record's ordinal number.

Database Management Systems use a variety of techniques to find data, often a system similar to operating system paging is employed. Fortunately a programmer will usually access such systems via a high-level interface, such as SQL, and need not be concerned with the physical design.

Buffering

- IO rarely acts directly on a device.
- Transfers are usually *buffered* (or *cached*).
- More efficient.



QA

8

When a program reads a file from a disk (or other media), or writes to it, it rarely accesses the device directly. Reads and writes usually occur via a buffer, or cache, which is a temporary storage area in memory.

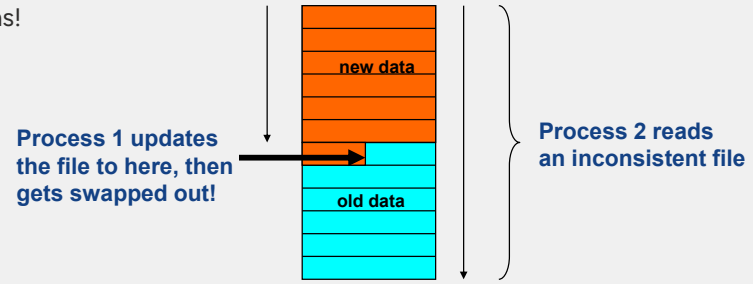
One reason for this is that all device input/output is relatively slow compared to memory access, so a single large transfer is quicker than many small ones, each of which may require the media (disk heads) to be repositioned. Another reason is that each 'real' device access requires a kernel call, which is much slower than a local function call (as much as 20 times slower in some cases). A buffer allows a large amount of data to be transferred using only a single kernel call.

There is an optimum size for a data block for every device, and the buffer will be pre-set to be that size. Even if the program only reads a single character from the file, in fact a whole block will be read. This is based on the assumption that the data will be used eventually. Some systems read more than one block if the file is accessed sequentially (known as read-ahead), but this has to be controlled by low-level interfaces. Of course once the data is in memory it can be accessed by other programs without further device IO - until it is changed.

Terminal IO is often handled in this way. The program may read a character at a time from the keyboard, but the run-time library may read a whole line. The program may write characters to the screen, but they might not appear until the screen buffer is flushed.

Concurrency issues

- Is it OK for two processes to read the same file?
  - Yes! Each has its own file reference and current file position.
  - Many processes can read the same file at the same time.
- What if one wants to write?
  - Problems!



Imagine the problems if two processes both write at the same time!

QA

8

Data stored in a file often has to be shared between processes. These may be copies of the same program being run by several users, or different programs needing to use the same data.

While these processes are only reading then we should not have any problems, but if one of them writes or updates the file at the same time, things can go wrong. As we know a program can be scheduled at any time by the operating system, and can suddenly loose the processor in the middle of doing something. If a program has only partially completed its task, then the data is inconsistent. Some of the data has been updated and some has not.

A process which is reading the file then gets partially updated data. For example, a personnel record could have had a tax code increased, but not the corresponding salary.

If two processes try and write at the same time, it gets worse. We have no idea which parts of a record (or file) is the most up to date, and we will probably end up with a mixture. The data is corrupt.

We cannot predict when processes will be swapped out, and in which order changes will be made. The only certainty is that if it can go wrong, it will.

We need a solution that will prevent other processes from accessing a file while someone is updating it.

The solution — locking strategies

- Where processes wait for another to complete its work.
  - Exclusive file lock:
    - No other process is allowed to open the file for read or write
    - Safe, but holds other processes up
  - Exclusive record lock:
    - No other process is allowed to use the record for read or write
    - Safe, other processes *may* have to wait
    - No good for "whole file" updates
- Shared read lock:
  - Any number of processes may read
  - A process attempting to write must wait for the reader/s to complete
  - A process attempting to read must wait for a write to complete
  - Only one process may write at one time
  - Can be used on a whole file or a record

QA 8

The solution is known as locking.

File locks can be applied at several levels, the most drastic is an exclusive file lock, where a process demands that no one else may open the file while it is using it. This has a number of disadvantages. The process demanding the lock will not be able to open the file if someone else is using it, and must keep trying until the file is free. Once the exclusive open is successful then no one else can perform an open on that file, which could cause programs to fail. However this may be the only option if the entire file has to be updated in one go.

An exclusive record lock is less of a problem. Here we just prevent multiple access to a region of a file. Other processes can access the same file at the same time, so long as they do not try and use the locked area.

In practice we do not want to make a program crash if it tries to use a locked file or record, instead we usually want them to wait. Functions within the operating system are available to manage this, and to queue and release processes in an orderly manner. The option to not wait, but to try something else instead (called non-blocking), is also available.

The mechanisms used to implement file locks differ between languages and operating systems, but usually they consist of mutex (mutual exclusion) flag, and semaphore (counting) primitives. Using these primitives allows the principle to be applied beyond files, and is often used to control concurrency on other shared objects, like shared memory.

Alternate storage locations

- Database Management System
  - Server-based or local
  - Access via an API
  - Transactional?
  - Fast to search
- Cloud Storage
  - Microsoft Azure
  - Amazon AWS
  - Google Cloud Services
  - Web or REST Services
  - Micro services

QA

8

Persisting data to other storage locations usually involves the use of specific libraries for services such as remote database like SQL Server or Oracle and cloud services like Microsoft Azure and Amazon AWS.

There is also a huge proliferation of web, REST and micro services that are mostly accessible using standard network protocols.

Exercise

Controlling The Scope of Data

Summary

- Data in memory is volatile.
- To retain data it must be persisted.
- Files are the simplest persistent storage containers.
- Files can contain:
  - Simple text
  - Binary
  - Structured data (XML, JSON, etc.)
- File Access achieved using:
  - OS API
  - Language Libraries
- Other containers include:
  - Databases: local and remote
  - Cloud Services

File I/O examples

- The following slides show examples from several languages.
- They all:
  - Open two files and save a “file object” or “handle”.
  - One file is open for read, one for write.
  - We then loop, reading a line from one and writing it to another.
  - We use a maximum buffer size of 4096 characters.
- Basically, this is a copy operation.
  - Its purpose is to illustrate the language.
  - There are often several ways to do this in each language.
    - We have tried to use the most typical mechanism.

Bash/Korn shell

```
#!/bin/bash

exec 3> "output.txt"

while read line
do
    echo "$line" >&3
done < "input.txt"

exec 3>&-
```

The read/write loop

QA

8

Note that no error handling is included. Error handling is possible in Bash and Korn shell, but rarely done.

The basic structure is to open the output file first, then iterate through the input. Most languages do something similar.

Most shell programmers would not do this to copy a file, they would use the cp(1) program instead.

C

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 4096

int main (int argc, char *argv[])
{
    char line[MAX_SIZE];
    FILE *infh  = fopen("input.txt","r");
    FILE *outfh = fopen("output.txt","w");

    if (infh == NULL) {
         perror("input.txt");
         exit(1);
    }

    if (outfh == NULL) {
         perror("output.txt");
         exit(1);
    }

    while (fgets(line, MAX_SIZE, infh))
        fputs(line,outfh);

    fclose(outfh);
    fclose(infh);
    return 0;
}
```

Open the files

Error checking

The read/write loop

QA

8

This does include error handling - standard C does not support exceptions (although Win32 C does).

C++

```
#include <fstream>
#include <iostream>
#define MAX_SIZE 4096

int main (int argc, char *argv[])
{
    char line[MAX_SIZE];
    std::ifstream infh("input.txt");
    if (!infh.is_open()) {
         std::cerr << "Unable to open input.txt" << endl;
         return 1;
    }
    std::ofstream outfh("output.txt");
    if (!outfh.is_open()) {
         std::cerr << "Unable to open output.txt" << endl;
         return 1;
    }

    while (infh) {
         infh.getline(line, MAX_SIZE);
         if (line[0]) outfh << line << "\n";
    }
    outfh.close();
    infh.close();
    return 0;
}
```

Open the files

Error checking

The read/write loop

QA

8

The C code on the previous slide would work in C++ as well, however this is preferred.

The error handling shown is rather basic, it could also use `std::strerror(errno)` to get the error message, alternatively exception handling could be used.

C#

```
using System.IO;

class hello
{
    public static void Main()
    {
        StreamReader infh  = new StreamReader("input.txt");
        StreamWriter outfh = new StreamWriter("output.txt");

        do {
            string line;
            line = infh.ReadLine();
            outfh.WriteLine(line);

        } while(infh.Peek() != -1);

        infh.Close();
        outfh.Close();
    }
}
```

Open the files

The read/write loop

FORTRAN

```
program fileio

      character*4096 line
      integer EOF

      open(UNIT=10, FILE='input.txt', status='old',action='read')
      open(UNIT=11, FILE='output.txt',action='write')

      DO
          read(10,900, IOSTAT=EOF) line
          IF (EOF /= 0) THEN
              close(10)
              close(11)
              EXIT
          ENDIF
          write(11,900) trim(line)

      END DO

900   format (A)

      end program fileio
```

The read/write loop

Java

```
import java.io.*;
import java.util.Scanner;
public class fileio
{
    public static void main(String args[])
    {
        Scanner infh;
        try {
            infh = new Scanner(new FileReader("input.txt"));
        }
        catch(FileNotFoundException e) {
            System.out.println("Unable to open input file");
            return;
        }
        PrintWriter outfh;
        try {
            outfh = new PrintWriter(new FileWriter("output.txt"));
        }
        catch(IOException e) {
            System.out.println("Unable to open output file");
            return;
        }
        while(infh.hasNextLine()) {
            String line = infh.nextLine();
            outfh.println(line);
        }
        infh.close();
        outfh.close();
    }
}
```

The read/write loop

QA

8

There is a more modern mechanism, using stream readers (similar to the C# example).

Perl

```
use warnings;
use strict;

open(my $infh, '<', 'input.txt') or
    die "Unable to open 'input.txt': $!";

open(my $outfh, '>', 'output.txt') or
    die "Unable to open 'output.txt': $!";

while (my $line = <$infh>) {
    print $outfh $line
}

close $infh;
close $outfh;
```

The read/write loop

QA

8

Old legacy Perl code often uses uppercase file handles:

```
open(INFH,'input.txt')
```

But that has issues and is deprecated.

PHP

```
<?php

    $outfh = fopen ("output.txt", 'w');
    $infh  = fopen ("input.txt", 'r');

    while ($line = fgets($infh, 4096))
        fputs($outfh,$line);

    fclose($outfh);
    fclose($infh);

?>
```

The read/write loop

QA

8

No error handling is shown.

`fputs` is an alias for `fwrite`

Python

```
outfh = open('output.txt', 'w')

with open('input.txt', 'r') as infh:
    for line in infh:
        outfh.write(line)

outfh.close()
```

The read/write loop

QA

8

In Python the open() statement can be used as an *iterator* on a text file.

The example shown uses open() as a context manager, which will ensure all files are closed, even on error. A simpler example would be:

```
outfh = open('output.txt','w')

for line in open('input.txt'):
    outfh.write(line)

outfh.close()
```

Ruby

```
outfh = open('output.txt', 'w')

open('input.txt').each { |line|
    outfh.write(line)
}

outfh.close
```

The read/write loop

QA

8

Alternatively:

```
outfh = open('output.txt', 'w')

for line in open('input.txt')
    outfh.write(line)
end

outfh.close()
```

Visual Basic .Net

```
Imports System.IO

Module Module1

    Sub Main()
        Dim infh As New System.IO.StreamReader("input.txt")
        Dim outfh As New System.IO.StreamWriter("output.txt")

        Do While infh.Peek() <> -1
            Dim line As String
            line = infh.ReadLine()
            outfh.WriteLine(line)
        Loop

        infh.Close()
        outfh.Close()
    End Sub

End Module
```

The read/write loop

Building and Testing Applications

Programming Foundations



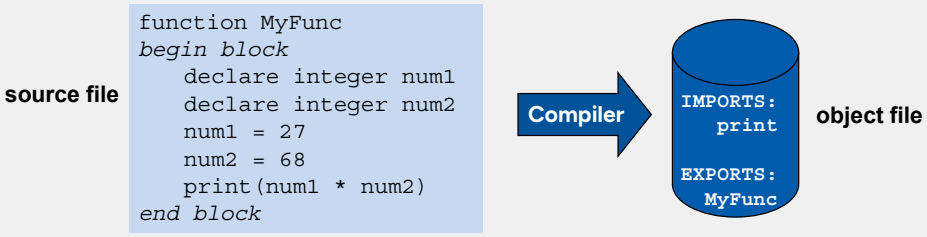
QA
Learn. To Change.

Building and testing applications

- Building Apps
  - Compilation and linking
  - Interpretation
  - Byte code
- Optimization
- Debugging
- Testing
- Source control
- Continuous integration
- DevOps

Compilation

- Converts *source* (program language text) to *object*.
  - Performed by a *compiler*
- Produces a relocatable object.
  - In a standard format, e.g. COFF (Common Object File Format)
  - Contains *unsatisfied external references*
    - Calls to run-time libraries, other functions, etc.



QA

8

Programs written in conventional languages such as C, C++, COBOL, FORTRAN, and Pascal, have to be converted into a low level format (usually machine-code) before they can be run. The initial step is to generate an object file. This is done by a software tool known as a compiler, which understands the language syntax.

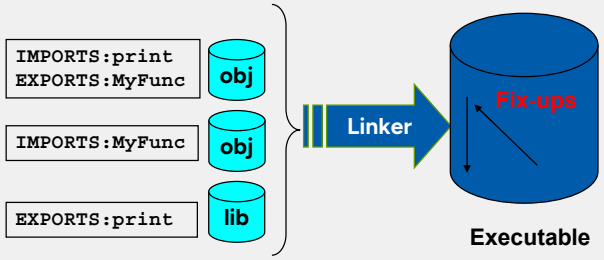
Programs hardly ever include everything required for their operation, they almost certainly use libraries of reusable code written by other people. For example, most language compilers include a run-time library of functions for doing file input and output. We might also be calling a third party product like Oracle. These other libraries can be written in any language, so an intermediate stage in a common format is required. These intermediate libraries are known as relocateable object files (or just 'object files'), relocateable because they do not use specific memory addresses but instead may be placed anywhere in memory.

In order to use these object files we must have a list of the interfaces which may be used by the outside world (other object files), this is called the export list. Most object files will use other libraries, so there is usually also an import list as well.

So, how are all these components tied together to make an executable program? That is the job of the linker, which we shall see on the next slide...

Linking

- The Linker:
  - Collects together relocatable objects and libraries
  - Resolves all external references
  - Produces an executable file in a system specific format



QA

8

The linker is responsible for taking a collection of relocatable objects and gluing them together to make an executable program file. Any external function called by an object file should be in its import list, so all entries in the import list must exist somewhere in the list of object files to be collected together. If any of these are unresolved, the link operation will fail.

Linking can also be done at run-time, where the bulk of the code is loaded as and when the program requires it. Microsoft Windows DLLs (Dynamically Loaded Libraries) can be used in this way. In this case, the linker still needs to resolve the import lists of all the objects, but has a small stub file which tells it where the reference may be found.

The linker also decides the memory layout, so it is important that specific memory addresses are not used in the object files. The final executable file, from which the program may be run, will be in an operating system specific format. For example, most Unix's use ELF (Execute and Link Format) but have different CPUs, so you cannot usually run an executable program from a different machine. However, all Microsoft Windows 32-bit operating systems use the same PE-COFF (Portable Executable — Common Object File Format).

Portability

- Executable files can usually only be used on the architectures they were built on.
  - Exceptions — *cross compilers*.
  - Example: building programs on a desktop for mobile devices (often built and tested using a USB cable).
- Hardware differences:
  - Executable files contain CPU specific instructions (Programs built on SPARC won't run on Intel).
- Operating System differences:
  - Different operating systems use different file formats.
  - A program built on MS Windows will not run on GNU/Linux (even on the same hardware).
- Portability is usually maintained at source code level...
  - ...but the programmer must follow generic standards.

QA

8

An executable file built by a linker is usually specific to a particular architecture. It will contain machine code statements that are only meaningful on a particular type of CPU, and will be in a format peculiar to the operating system it was built for. For example, a program built on Microsoft Windows will not run on GNU/Linux, even on the same hardware, because each has its own, different, executable file format. Programs built on Windows 95, however, will run on Windows XP because Microsoft uses a common format for all its 32-bit operating systems. Some features available on Windows XP, however, are not available on Windows 95, so the reverse may not be true.

All this means that we have to generate a different set of executable files to run on each hardware/operating system combination (platform) that we wish to use. We should be able to use the same source-code provided we stick to a recognised standard, and do not attempt anything that only works on one platform.

Emulators

- Emulators “pretend” to be a different operating system.
  - Can include the alien user interface.
  - Often implemented as libraries.
  - Not the same as Virtualization, which runs the real operating system.
- Examples:
  - WINE pretends to be MS Windows on GNU/Linux
  - Cygwin provides a semi-UNIX-like environment on MS Windows.
- *But!* They use a lot of system resource.

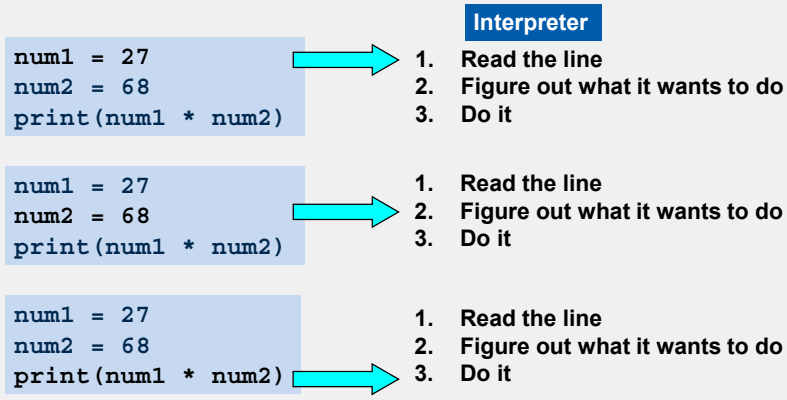
QA

8

One solution is for a software package to emulate a different platform, and translate formats and instructions into a form understandable by the current system. There are emulators which run under GNU/Linux that will run old Commodore 64 programs, as well as those from Microsoft Windows. They are inevitably slower than the original, except perhaps where the hardware has improved over time (as is the case with the Commodore), because an extra 'layer' has to be executed. They can rarely support the total environment of the other platform, often there are features that cannot work because of design differences.

Interpretation

- The program is interpreted and executed a line-at-a-time.



QA

- Can be more portable — most shell scripting languages are interpreted.

8

An interpreter removes the problems of compiling and linking by reading and executing each line in the source code as it finds it. Most shells are interpreters, as well as some versions of LISP, BASIC, and others. The advantage is that you can instantly run the program as soon as you have written it without messing with a compiler and linker. The interpreter is built for a particular machine, but the language commands may not, therefore they can be more portable. For example, the same Bourne shell script would run correctly on nearly all Unix machines, however executable programs will only run on the Unix platform they were built on (with a few rare exceptions).

There are significant disadvantages. The most obvious is that the program has to be re-interpreted every time you run it, even though it may not have changed from the previous run. A compiler and linker may be an inconvenient step, but it only has to be done once.

Since an interpreter only compiles one line at a time, it could find a syntax error after executing part of the program. The user then has to go back and undo the actions already carried out before a re-run can be attempted.

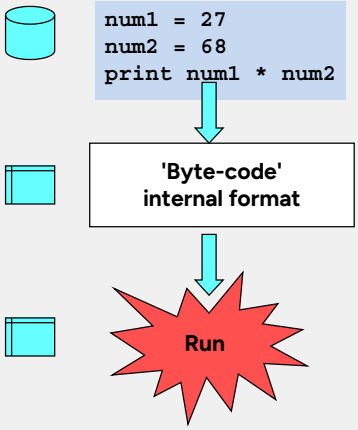
Being unable to "look-ahead", the interpreter cannot carry out any optimisations. These are discussed later, but a compiler can often produce more efficient code than the programmer, assuming it knows what is being aimed for.

Having no linker means that we cannot use code written in other languages unless they are running in a separate process. Creating a process is expensive and slow. For example, in CMD.EXE on Microsoft Windows the 'dir' command executes in a separate process, using the dir.exe executable program. The same applies to the 'ls' command on UNIX.

The third way: Byte-code

- The program is partially compiled internally, then run.

1. The program source code is read from disk
2. Any external references are found and also compiled
3. The main program is compiled into a fast internal format
4. The program is run from memory



QA

8

How can we get the portability and convenience of interpreted code without the performance problems and lack of flexibility? The third approach is to compile the source program into an internal format in memory, then, assuming no errors were found, run that.

There are a number of advantages. Provided the program is written to a standard, the source code should be portable, we just need the compiler on the machine we want to run it. Optimisations can be performed because the whole program is compiled before running, and links may be made to external modules without invoking another process. This system is used by Perl, Python, Ruby, and others. Java and C# code is not compiled at runtime, but as a separate step using a compiler which generates byte-code which can be stored in a file for later use. For example, Microsoft .Net defines the internal format (byte-code) and run-time environment, allowing many different languages to generate the compiler code, for example C#, and Visual Basic. It is possible for any .Net module to call another, even though they may be written in different languages and reside on different computers.

An issue is that the program still has to be compiled more times than is necessary. Fortunately the compiler often resides in a service or daemon, a program which is running continually, and the compiled application program can be retained in memory for later use.

Most compiler/interpreters also allow the compiled internal code to be "dumped" into a file for later reuse, and Python does this automatically, only updating the dumped "image" if the source code changes.

Optimization

- Problem:
  - It can be difficult to write highly efficient code.
  - Highly efficient code can be difficult to read and support. What is efficient on one computer might not be on another.
- Solution:
  - The compiler can optimize your code.
  - The compiler knows more about your computer than you do.
  - The code the compiler produces is only read by the computer — your source code is not changed.
- There's always a "but…."
  - Occasionally (not often), the optimizer can get it wrong.
- Statically typed languages can have more optimization applied.
  - Types are not dynamic, so predictions can be made.

QA 8

Most compilers have optimisers built-in. They take your source code and manipulate it to produce a faster, or smaller, executable file. This means that you do not necessarily have to write unreadable code to be fast, you can concentrate on good style.

Speed tricks on one machine might actually run slower on others, and knowing the tricks on all machines can be difficult — as well as resulting in non-portable code.

The compiler has been written for the specific architecture you are using, and it knows the low-level details, so it makes sense to use that expertise.

A statically typed language, like C, C++, C# or Java, has much more scope for optimisation than a dynamically typed one, like say Perl or Python. Variable types are fixed at runtime (although there are exceptions to that) and so predictions can be made of memory and processor usage.

Optimization example

```
declare integer num1
declare integer num2
declare integer i
declare integer array x
num1 = 27
i = 0
while ( i < 20 )
begin block
   x[i] = 42
   num2 = 37
   i = i + 1
end block

num2 = num2 + 3
num2 = num2 + 2
print(num2)
```

num1 is not accessed further!
Optimizer will remove it

Array access is more efficient
using memory addresses

Does not change in the loop
Optimizer will remove statement
from the loop

Can be done in one statement
num2 = 42

QA

8

We show some common optimisations on the slide. Variables that are assigned values but never used will be removed, and multiple assignments may be combined into a single statement. Loops are a particular target for optimisations, and any statements that can be placed outside the loop will be. Array access if often optimised to access memory directly rather than performing additions at run-time.

There are many other things that can be done to improve performance and most compilers support several levels of optimisations. They can usually be switched off all-together if they cause problems.

Debuggers

- Tools that allow the programmer to:
  - Step through each line of a program, noting the program flow.
  - View variable values as they change.
  - See which functions are called, and parameters passed.
- Important part of an IDE.
  - Microsoft Visual Studio and J-Builder have powerful debuggers.
  - Some IDEs have command-line debuggers as their "back-end".
  - Most interpreters/compilers have debuggers built in.
- Can be invoked for run-time errors.
  - "**J**ust **I**n **T**ime" debugging.
- Compiled/Linked programs have to be recompiled.

QA

8

Tracking bugs in programs can be difficult because the execution paths can be very complex. It is often hard to visualize what the program is actually doing, or how it got to the point of failure. Most people start off with putting 'print' type statements in the code to check on the order of execution (tracing) and the values of variables, but this quickly becomes unwieldy. The addition of extra statements can anyhow change the way the program operates.

A debugger gives an interactive way of tracing and stepping through the program, one statement at a time. Usually the program can also be run normally up to a designated point (a breakpoint), when it is interrupted and the programmer can inspect variable values.

At run-time, some operating systems dump the program's process address space into a file (often called a core dump) when a fatal error occurs. This can be debugged later. Microsoft Windows allows the option of invoking a debugger, which can be anything from Dr.Watson to Visual Studio.

Of course a compiled and linked program at run-time does not resemble the high-level language that the source code was written in. The program has to be specially compiled and linked for a debugger to be able to translate the machine code in the executable to the source language the program was written in.

If the program is interpreted then the interpreter should be able to do the job at run-time, and usually a simple command-line switch is all that is required.

The general adoption of Graphical User Interfaces and Integrated Development Environments with built-in debuggers have improved programmer productivity considerably.

Testing

- How do you know your code behaves as it should?
  - Does every function do exactly what it's supposed to?
- Options:
  - Use the apps UI to test the whole app and hope you exercise all the code.
  - Write some code to test the code you have written.
    - Code Testing Code = Unit Testing
- Simple example:

```
myCar = Car()
myCar.accelerate(5)
passed = myCar.speed == 5 # True if accelerate worked
```

QA

8

How do you prove your code does what it's supposed to. The answer is write some code that checks it. That's write code to test code.

Unit testing

- Most languages have a built-in test library:
  - Designed to facilitate Unit Testing
  - Can be automated

```
import unittest

class TestStringStuff(unittest.TestCase):

    def test_upper(self):
        self.assertEqual("fred".upper(), 'FRED')
    def test_uppercase(self):
        self.assertTrue("FRED".isupper())

if __name__ == '__main__':
    unittest.main()
```

QA

8

Unit Testing an important part of software development because it allows you to write multiple tests for every function that you write.

In the example above we are using a testing library to help us write test code that checks that certain things are true or false. We are actually test the string object but we could just as easily be testing one of our own functions or classes.

We can execute a unit test and it will report if it has passed or failed.

Additional forms of testing

- Automated User Interface Testing:
  - Captures keystrokes and mouse events during a User Interface Test.
  - Recorded script can be replayed multiple times.
  - Actual results can be extracted from UI and compared with expected.
- Load Testing:
  - Allows a simulated load to be applied to an application.
  - Often used with Web Sites and Web/REST Services.
  - Can use multiple agents (PCs) to run UI Test Scripts.
  - Very high levels of load can be simulated.
  - Knowing the peak load and break load of an application is key.

QA

8

Unit testing tests the raw code. User Interface Testing tests the interaction with the User Interface. You can record operations performed on a UI and then replay the recorded steps at later dates. Checks can be built in that ensures the UI produces the expected results.

Load Testing deliberately puts an application under stress and tries to establish the peak load an application can support and the load required to break it.

Performance testing

- How fast is fast enough?
- Performance Test Tools analyze code as it executes.
  - Tells you the slowest execution paths (sequence of function calls).
  - Helps you determine why a function is acting as a "rate limiting step".
- Helps you distinguish between:
  - Wall Clock Time: actual elapsed time.
  - CPU Time: time spent actually processing the code.
  - Difference is often down to code waiting for something else to happen.
- Performance Testing the code you write:
  - Doesn't take a lot of time.
  - Can identify potential bottlenecks in your code.

QA

8

Performance testing helps you establish the parts of your application that may contain bottlenecks causing the application to run slowly and reducing the numbers of users or transactions it can concurrently support.

Performance test tools analyse executing applications and determine the execution paths (sequence of function calls) that take the most time to complete. The results can be presented in terms of wall clock time (elapsed) and CPU time (time spent processing on the CPU).

Source code management

- How do multiple developers work together on the same codebase?
  - Version Control System:
    - Manages multiple versions of the same file
    - Tracks changes on a per developer basis
- Works on a user's local machine.
  - Can be pushed onto a server machine
  - Multiple developers can push and pull versions to server
  - Changes even to the same file can be merged
- Two developers can work on the same codebase and even the same file.
- Typical examples include:
  - git, Mercurial, Visual Studio Team System

QA 8

To facilitate multiple developers working on a common codebase it's usual for developers to use version control systems. These systems allow for the storage of multiple versions of source files. Changes can be tracked and multiple versions of sources files can be collected into a central repository. It's even possible for two developers to change the same source file and have those changes merged together in the central repository.

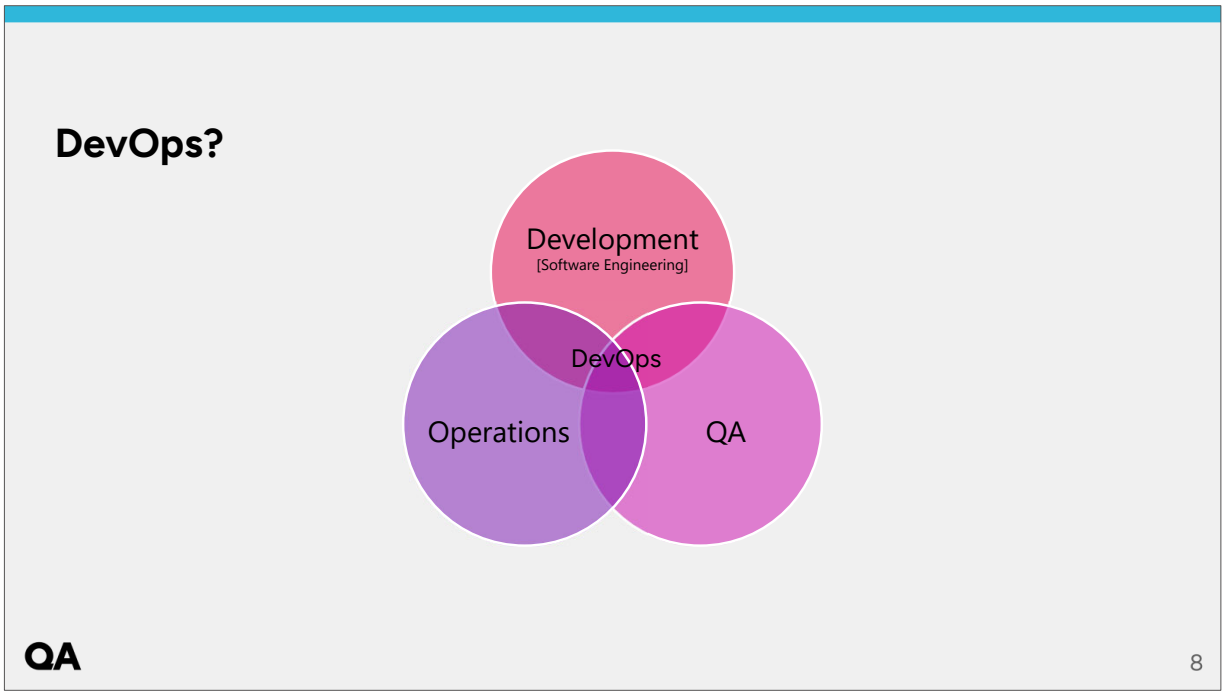
Continuous integration

- Many development teams automate the process of:
  - Building -> Testing -> Reporting -> Deploying
- As new features are added to the codebase and pushed to the VCS:
  - Automated process pulls the latest code
  - Builds the application
  - Runs all Unit and UI Tests
  - Deploys
  - Can execute load tests
  - Reports results to common repository
  - Notifies errors and completions

QA

8

With code held in a central repository, many teams develop automated processes that continually build, test and release applications as the source code is updated.



DevOps takes the idea of Continuous Integration to the next level. It combines usually separate disciplines of Development, Operations and QA.

Summary

- Programs are built in a variety of ways:
  - Compiled, then linked
    - Slow, complicated build process
    - Fast execution
    - Non-portable executable files
  - Interpreted:
    - Slow to execute
    - Inflexible
  - Semi-compiled into byte-code at run-time:
    - Slightly slower than fully compiled programs
    - Easy and flexible
- Buildings Apps typically involve:
  - Source Control, Automated Builds, DevOps processes

Software development method

- Traditional Software Development Lifecycle adopts a "Waterfall" approach:
  - Planning
    - Analysis
      - Design
        - Code
          - Build
            - Test
              - Deploy
- Each step requires the completion of the previous one
- Relies on documentation, agreement, and signoff
  - Rigorous, thorough, traceable, accountable

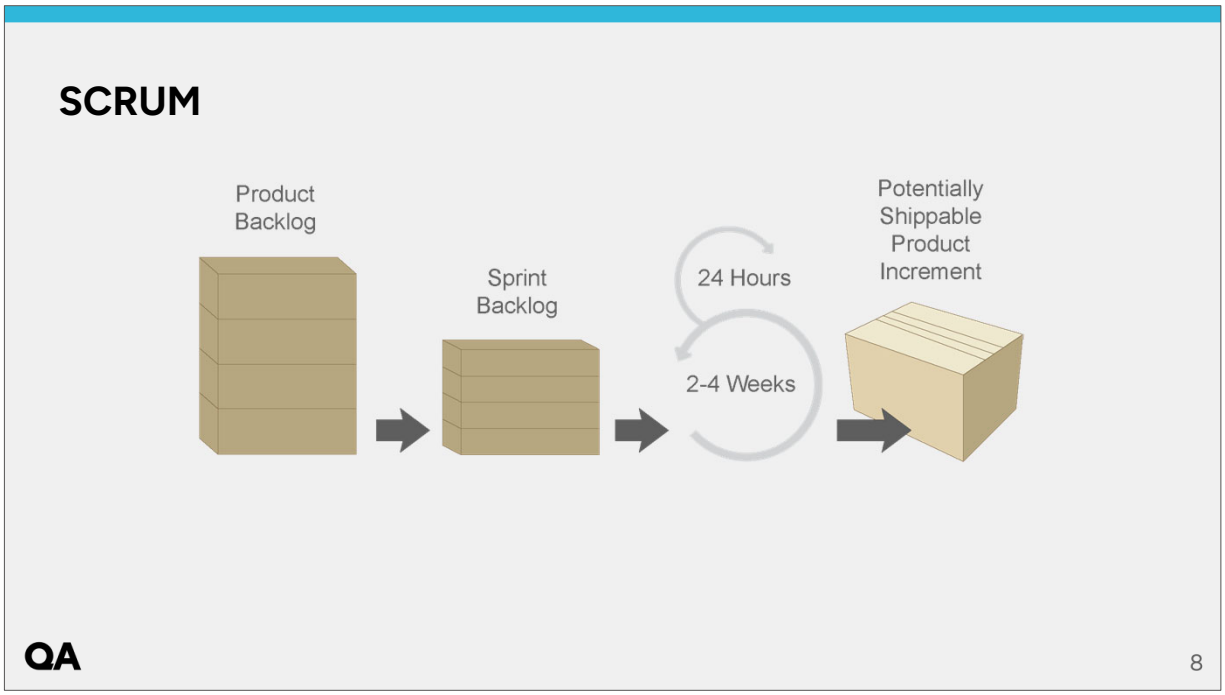
Problems with Waterfall

- A useful product is delivered at the end of a typically long lifecycle.
  - Users often don't know what they want at the beginning.
- Responding to changing requirements/needs is difficult.
  - Changing something often requires traversing back up the waterfall.
- Feedback and communication can be poor.

Agile

- Agile is an Umbrella term for a number of different methodologies.
  - SCRUM
  - KANBAN
  - DSDM
  - SAFE

- Four guiding values:
  - Individuals and interactions over processes and tools
  - Working software over comprehensive documentation
  - Customer collaboration over contract negotiation
  - Responding to change over following a plan



The key to SCRUM is the delivery of a continuous stream of potentially shippable product increments. Each increment can be evaluated by the stakeholders and the product modified to better reflect changing requirements.

In SCRUM timescale and quality are fixed, it is what is to be delivered in the sprint cycle that is negotiated.

SCRUM Team

- Development Team
  - Self-organized and empowered
  - Cross-functional/skilled
- Scrum Master
  - Promotes and upholds the SCRUM process
  - Removes impediments for the Dev Team
  - Interface with Product Owner and key Stakeholders
- Product Owner
  - Responsible for increasing the value of the product
  - Decision Maker and Sets priorities
- Stakeholders — Key business representatives, owners, customers

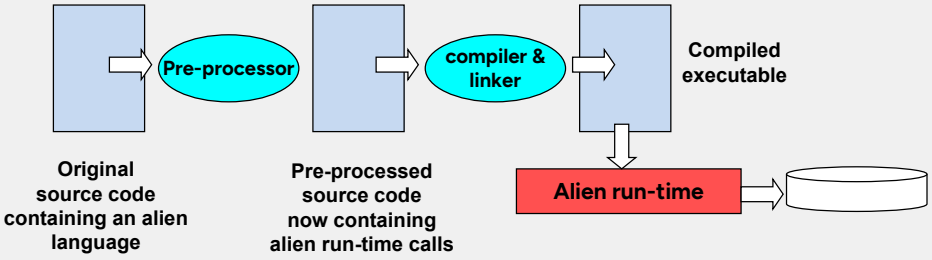
QA

8

Above are the key players in the SCRUM process. They all play a vital role.

Pre-processing

- Using an alien language within another:
  - Most common example is SQL, used for database access
  - Conventional languages do not support SQL
- Source code is "pre-processed" before compilation — changes alien statements into external function calls.



QA

8

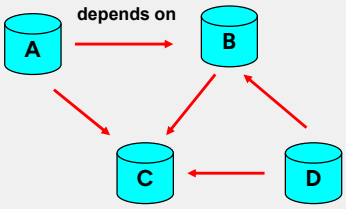
There are some occasions where a language does not have all the features required for an application. Mixed language working, where modules of a program are written in different languages, might be a possibility if compiled and linked languages are in use. If one of the languages is complied and linked (like C, or C++) and another is interpreted, like SQL, then they cannot be combined easily. Pre-processing is the answer.

Using a pre-processor we can embed our alien language, like SQL, into our "conventional" source code. Of course the compiler for, say, C++ does not know SQL, so would fail. Instead we run an extra program, the pre-processor, that converts the alien calls into something the compiler will recognise. Typically these are calls to external sub-routines that are resolved at run-time.

Some languages, notably C and C++, have a text pre-processor phase automatically invoked before the compiler. These statements change the original source code, rather than making run-time calls.

The 'make' utility

- Complex projects collect together many files, with complex *dependencies*.



A uses functions in B & C
B uses functions in C
C has no dependencies
D uses functions in B & C

- If file B changes, do we need to recompile any others?
  - No need to recompile C.
  - We need to recompile A and D, because B's functions may have changed.
- A "make" file documents these dependencies.
- The `make` utility ensures dependencies are satisfied.

QA

8

Even relatively simple projects can include a number of object files, and keeping them all up-to-date can be a chore. On large projects it can be much more difficult to manage, with object files coming from a number of different sources. The make utility enables us to document dependencies between object files, and to make sure we recompile and link the correct combinations. We could recompile and link the whole application each time someone changes something, but this can take literally hours for a large product.

make has its own scripting language to enable us to define the files used by a project, and how they interact. These commands are usually stored in a file called a Makefile, and will include the compiler call, and any compiler specific switches. There are several versions of make, and you may also come across the 'new-make' utility, named nmake, which is used by Microsoft (among others).

Make files

- Dependencies are specified in the "make file"
- Consist of target : dependency list …
  - Followed by one or more Command lines:

```
myprog: myprog.o initial.o menu.o utils.o
      cc -o myprog myprog.o menu.o utils.o -lcurses

myprog.o: myprog.c common.h utils.h menu.h gameunit.h
      cc -c -d myprog.c

menu.o: menu.c common.h utils.h menu.h
      cc -c -d menu.c

utils.o: utils.c common.h utils.h
      cc -c -d utils.c
```

Example shows C source files on the UNIX operating system

- An Integrated Development Environment (IDE) will maintain its own make files, or equivalent.

QA 8

The Makefile can be edited like any other text file. It can consist of macros, commands, and sets of dependency lists.

The dependency lists are where the relationships between files are stored. It is imperative that the lists are kept up to date.

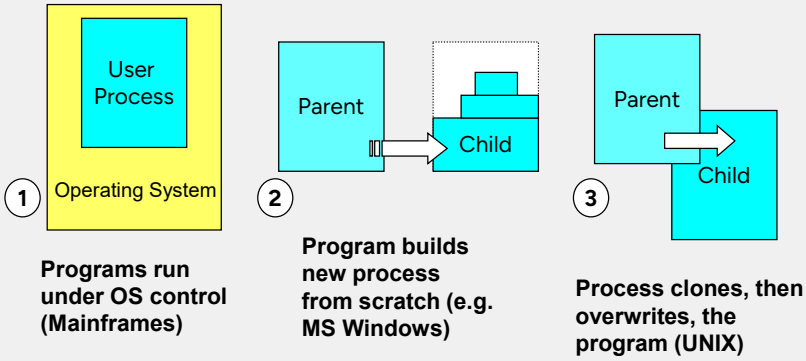
In the example, myprog is dependent on the object files myprog.o, initial.o, menu.o, and utils.o. Each of these are in turn dependent on a list of .c and .h source files.

Command lines which follow the list describe how the target may be "made". They must be indented by a <TAB>, not spaces.

If you use an Integrated Development Environment, like Microsoft Visual Studio or K-Develop, it will maintain the makefile for you.

Loading and running

- All programs have to be “launched” from another.
  - Often a “shell” (the first is started by the operating system). Usually known as the *parent*.
- There are several strategies used:



Programs run under OS control (Mainframes)

Program builds new process from scratch (e.g. MS Windows)

Process clones, then overwrites, the program (UNIX)

QA

8

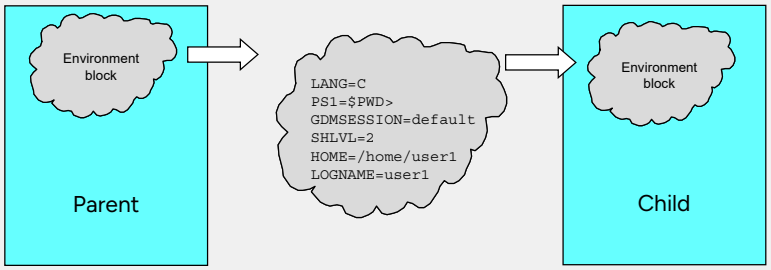
Every program has to be launched by another, which naturally leads to the question of "who loads the first program"? The answer is the operating system, although of course that is also a collection of programs, so who loads the operating system? This is like someone pulling themselves up by their own boot-straps (shoe laces), so this process is known as bootstrapping, often abbreviated to boot. This is a job for the hardware.

The way the process address space is created differs between operating systems. Mainframes tend to execute user programs under the complete control of the operating system, whereas Microsoft Windows builds the process from scratch. Unix takes a different approach and "forks" or "clones" the parent program to an almost exact copy of itself. This is then overwritten by the new program, although some attributes might be retained from the original (known as inheritance).

When a program "loads" another, the program executable file is read. The addresses in the program are then 'mapped' against virtual addresses in the process address space (remember how virtual memory works?). This might involve loading any dynamic routines that were specified as stubs during linking.

Process attribute inheritance

- Data is inherited from parent to child process.
  - Security ID
  - Open file handle tables (optional)
  - Environment block (optional)
  - …and others



QA

8

When a new process is created, data is copied from the parent to the child. Several pieces of critical system data are copied, including identifiers indicating the user account the process is to run under, variously known as the User ID (UID on UNIX/Linux) or Security ID (SID on Windows).

Other areas of data can be controlled by the parent, but by default any open file handles are copied, which means you should be careful which files are open in the parent when new processes are created.

The most commonly used block of data is the *environment block*. This contains environment variables, which are simple text variables that can be set and exported from a command-line shell. Actually any program can add or modify variables in the environment block, but mostly this is done by shells. Environment variables offer a simple and cheap way of passing data from parent to child, but notice that this communication is only one way. A child cannot pass data back to the parent using the environment block — it only has a copy, it does not have access to the parent's block itself.

There are other ways that processes can share data, this is generally known as IPC — Inter-Process Communication. They include shared memory and communications channels.

Coding style

- Virtues of a programmer
- Readability and style
- Naming conventions
- Error handling
- Programming for change
- Programming for performance
- Constants — aren't
- Portability and flexibility
- Summary

Virtues of a programmer*

- Laziness:
  - The virtue that makes a programmer hate to code the same thing twice — hence reusable code is born.
  - Hates answering the same questions over and over, so writes good documentation.
  - Hates reading documentation, so writes code clearly.
  - Writes tools and utilities to make the computer do all the work.
- Impatience:
  - Hates a computer that is lazy — an impatient programmer's code anticipates a need.
- Hubris:
  - Pride in programs that no one will criticize.

QA

* with thanks to Larry Wall, author of Perl

8

The three virtues of a programmer listed above come from Larry Wall, the original author of the Perl programming language. Of course, these are tongue-in-cheek; true laziness requires hard work.

These virtues all have serious points, and are not confined to Perl.

For "laziness", which most would naturally see as a bad thing, read "efficiency". Reusing code between programs and projects not only saves money in development time, it also reduces the amount of testing to be done, and ultimately results in more reliable programs.

The impatient programmer gets the computer to do the work. So, rather than expecting the user to copy items between applications (which is slow and error-prone), the impatient programmer writes a program to do it. If there is no tool or utility to do what you need then write one (that's how Perl came about).

Hubris, or excessive pride, is often considered a sin. What is meant here is that the author takes personal responsibility for their own program code. Hubris means we care about what we do.

The rest of this chapter is about how we achieve these virtues, in particular on looking ahead beyond the initial development of a program and considering what will happen when it is released into the wild.

Good programming practice

- Rule Number 1:
  - Follow the standards of your organization
  - Ask to see the Coding Standards/Guidelines
- Then be sure your code always uses:
  - Consistent naming
  - Effective commenting
  - Proper and effective code formatting

QA

8

Always follow the coding standards defined by your organization above anything you read here.

Good rules to follow

- Remember "Garbage In — Garbage Out"
  - If you allow bad data in, you will get bad data out.
- Choose the smallest data type for the job.
  - You don't need a 64-bit integer to store a person's age.
- Always assign and operate on like data types.
- Remember floating point issues.
- Use constants, not literal numbers, where possible.
- Create variables with the shortest scope and lifetime.
- Ensure objects are allocated and available for release as soon as possible.
- Use variables for one purpose only.
- Functions should perform one task:
  - Check the data passed into a function is valid.
  - Don't make functions dependent on global variables
- Classes should have a single responsibility. Identify what that is.
  - Avoid making classes do things they shouldn't be responsible for.
  - Avoid making a class dependent on other concrete classes if possible.
- Automate your tests.

Naming conventions

- Rules for the naming of identifiers:
  - For data types — structures, classes, etc.
  - For data items — constants, variables.
  - For code fragments — functions, methods, libraries.
- Makes code easier to read:
  - Easier to distinguish your items from those of a third party.
  - Helps avoid clashes with reserved words.
- Data variables and constants should be clearly named.
- Functions should state their intent.
  - Avoid vague functions names like **calculate**, preferring more specific **calculate_invoice_total**

QA 8

Using a naming convention for your own identifiers has so many advantages that just about every set of programming standards mentions them. It is part of a programmer's job to follow the company conventions — even if you disagree with them. Your program code is not the place to argue about conventions.

A naming convention will help avoid using a name which the language, or a third-party library, already uses — this prevents *name collisions*. It also helps them stand out against the background noise of program code.

Naming conventions can indicate properties of the thing being named. This is usually in the form of a prefix, and is discussed on the next slide.

Other naming conventions

- Camel case:
  - Text with humps, each word has its first letter in upper case.
  - Except, perhaps, the first:
    - lowerCamelCase
    - UpperCamelCase — possibly the most popular naming convention.
- Lower case with underscores:
  - Underscore only — languages confuse a hyphen with minus.
  - Popular in the UNIX and C worlds.
    - Words are separated by underscores
    - all_lower_case
- Avoid all UPPERCASEL
  - These are commonly used for macro names and other specials.

QA

8

In order for variables names to be descriptive, they often have to consist of several words. How do we separate those words, when white-space is not allowed within an identifier?

There are several options. Some old languages (and old versions of existing languages) have limits as low as eight characters for identifiers. In these cases, many programmers use names consisting of initials, AFAIK. Most modern languages have no effective limit on the size of an identifier, or that limit is so large that it is unlikely to be exceeded.

Camel case (it has other names) is where the first letter of each word is in uppercase, forming a visual boundary between words without the need for a space character. Opinions vary on whether the first letter should be upper or lower case. When upper case is used, it is often called PascalCase.

It has been argued that CamelCase is not easy to read, and an alternative is to separate words using an underscore. A hyphen is not normally allowed because it can be confused with a minus sign, as in **`x-axis`** (x minus axis?). Using lower case names with underscores is popular in C and UNIX.

Does it matter which style you use? Not really, so long as you are consistent. Reading program code is not the easiest task in the world, and having inconsistent conventions makes it harder. Remember: if you are modifying someone else's program, you should try to use their original style as much as possible.

Single character names are acceptable for temporary variables. For example, i, j, k, and l are often used for loop counts, although lower-case letter l (ell) can be difficult to distinguish from the number 1 (one) depending on the font, so is probably best avoided, as is the letter O (oh).

Comments

- Always comment the intent of your code.
- Don't comment self-evident coding structures.
- Make sure you comment workarounds and quick fixes.
  - Ideally rewrite them to be optimal.
- Always update comments when you update code.
- Remove comments from scripts before release.

QA

8

Comments are essential in your code and are expected. Don't overdo it. Comments are not an excuse for poor code.

Formatting

- Format your code to allow your code to naturally flow.
  - Proper indentation makes it easier to identify code blocks.
- Languages like Python use indentation to specify blocks.
- C-based languages use braces.
  - Allow each brace to sit on its own line and use indentation.

```
foreach(string name in names)
{
   if (name.length<5)
   {
      Console.WriteLine("Name isn't long enough");
   }
}
```

QA

8

Good formatting makes code clearer.

Readability and style

- More time is spent on maintenance than development.
  - So make sure you document what you do.
  - Code that is obvious today is not obvious tomorrow.
- Avoid “clever” one-liners:
  - They are rarely faster, sometimes slower.
  - Often difficult to debug.
  - Who are you trying to impress?
- K.I.S.S. — Keep It Simple and Straightforward

QA 8

It is easy to spot a program written by an experienced programmer. It will be easy to read, well commented, laid out with plenty of white-space, and use consistent indentation. By 'indentation' we mean placing spaces inside blocks of code. For example:

```
if (apples == 42)
    apples = apples + 1;
else
    if (oranges > 3)
        apples = 37;
    else
        fruit = apples + oranges;
```

How many spaces you should use for indentation is something programmers argue over. Four is a good number, but if everyone else uses three, five, or six, then go along with them. Don't use tabs, though; the size of a tab varies between tools and users, and is not portable.

A program written by a novice is difficult to read, with no indentation, no white-space, and no comments. Try to make your programs easy to read and easy to modify. It might be you who has to implement a change six months after the code was written. If it is someone else then you can feel pride in your code — hubris.

The Elements of Programming Style, Brian Kernighan and P.J.Plauger, uses Fortran and PL/1 but is applicable to most languages (with a few exceptions).

Error handling

- If anything can go wrong — it will!
  - Specifically test error conditions.
- If an opportunity exists to test for an error — take it.
  - When calling a library routine
  - When getting any data from the outside
- Always report the error to an expected location.
  - Write errors to an error stream, not the normal output stream.
  - Users probably don't need the full story, but make sure it is available to those that do.
- It is a common hacker's trick to cause a program to fail.
  - Can result in the display of sensitive data, or the opening of a backdoor.

QA 8

Even if your program code is perfect and never does anything wrong (!) then you still need to handle errors which occur elsewhere.

> What if the user enters an invalid value?
>
> What if the file you are trying to use does not exist, or you do not have permissions to use it?
>
> What if a library routine you are calling gets an error?

For the majority of programmers, their least favorite chore is testing and handling error conditions. Often there are many ways for something to fail, resulting in exception code that can dwarf the core program logic.

One of the traditional approaches to signaling errors is to return some kind of status value from a function. There are a number of problems with this: it can lead to cascading if-else-if code that can be hard to read, and return values are too easy to ignore

Aborting the program is a last resort. It is up to the caller to determine what the failure policy should be, and for the called component to detect any failure.

The solution is to represent exceptions as objects. As objects they can be self describing, whereas something like a simple integer value — as used in some languages — is not very informative as it says nothing about the details of an exception. Exceptions can be caught by defining a handler, but how you do that, and how you raise and catch exceptions, is language specific.

Get the computer to collect all the diagnostic information available, particularly if you are lazy and impatient.

Programming for change

- Defensive programming
  - Where changes are less likely to cause problems
  - Includes good naming conventions
  - Clarity of style makes the code easy to read
- Making your program flexible
  - Avoid artificial fixed limits
  - Make no assumptions on variable type sizes, word length, etc.
  - Adhere to standards as much as possible
    - Using language extensions ties your program to one vendor
- All this makes changing a program less work
  - Great for the virtue of laziness

QA 8

Defensive programming is a style of programming intended to reduce the possibility of mistakes during the maintenance phase of a program's life-cycle, remembering that this phase will be many times longer than the development phase. That maintenance is often carried out by people other than the original developer.

Defensive programming is a technique which requires some thought and a rather pessimistic outlook. The motto is: *never make assumptions*.

Coding using a clear, easy to understand, style is the foundation of defensive programming, but it does not stop there.

When testing a number of alternative values, always include a test at the end if it matches none of those — even if, logically, if can't be anything else. At the very least you would have caught an error.

Using well named constant or read-only variables to hold numbers is good, but they must be defined in a "well-known" or obvious location in the code. Of course documentation helps as well.

Artificial limits are often imposed when a programmer creates a fixed length data item. For example, how much memory should be allocated for someone's name? 24 bytes? 1024 bytes? If you do not allocate enough then data may be lost, allocate too much and you waste system resource. This is where dynamic memory allocation becomes useful — deciding at run-time how much memory to use. The down-side is often a performance penalty — no free lunches here.

Using manufacturer's extensions to the programming language might be neat today, but what if someone wants to use a different supplier?

Help!

- Many languages have style checkers and rules.
  - Python has PEP 008 rules.
  - Perl has perlstyle documentation and PerlCritic.
  - Java has Sun's code conventions.
  - Google Style Guides are available for several languages — http://code.google.com/p/google-styleguide/
- Code analysis tools are often based on Lint.
  - An old C-based tool, now much enhanced.
  - Microsoft Visual Studio includes Code Analysis tool FxCop.
    - Others are available, e.g. ReSharper.
- Some editors and IDEs assist with style as you type.

QA 8

If your organization has its own standards then you should always follow those in preference to any other guidelines — including any we have mentioned in this course! Style is subjective.

Most programming languages offer advice on programming style, and a few are shown on the slide. The link to the Java code conventions is:

http://java.sun.com/docs/codeconv/html/CodeConvTOC.doc.html

A number of style guides are available from Google, languages include R, C++, Objective-C, Java, Python, Shell, HTML/CSS, JavaScript and Lisp.

Programming clearly is sometimes hard work. So what does a lazy programmer do? Write a program which imposes style or spots bad practice. There are several, many are based upon an old UNIX/C program called "lint," which looked explicitly for bad coding style. Modern versions are much more sophisticated and can spot some types of bugs as well. Few go as far as Perl's `perlcritic` program (and website) which offers an opinion of your code at different levels of criticism: gentle, stern, harsh, cruel, or brutal! There is a serious point here — it is an easy trap to feel over-protective towards your own programs. It is important to listen to, and sometimes to seek, criticism in order to grow your skills. Only then can you attain the virtue of hubris.

Many integrated development environments, and even simple editors, encourage good style by automatically indenting, giving hints, and so on. They are only helper tools though, it is still your responsibility to write clearly.

Summary

- The virtues of a programmer are laziness, impatience, and hubris.
- Describe what you want to do before you do it.
- Programs are subject to change.
- Program in a clear manner.
  - This will help you and others to maintain the code.
- Use consistent naming conventions.
- Avoid having “magic numbers” in your program.
  - Define identifiers for your values in a well-known location.
- There are plenty of tools out there to help.
- From the Google Style Guides:
  - Use common sense and BE CONSISTENT.

Conclusion

Programming Foundations

QA
Learn. To Change.



Overview

- Checking objectives
- References
- What next?

Review aims and objectives

By the end of the course you will be able to do the following:

- Understand how computers work
  - What is memory?
  - What are CPUs?
- Understand and write computer programs
  - Understand computer data
  - Declare variables
  - Perform calculations
  - Control program execution
  - Write functions
  - Understand variable scope
  - Organize data into containers
  - Store data in files
  - Understand the development process
  - Learn some good programming practice

What next?


Course material
Doing it for real
Consolidate learning
Further reading
Support from QA

QA

Help from QA

- **Ever-increasing range of courses**
  - C#, Java, Javascript, Python, Swift, C, C++, Scala, and many more
  - Node.js, ASP.NET, Angular, jQuery…
- **Websites**
  - www.qa.com
  - Social networking
  - Blogs



In closing...



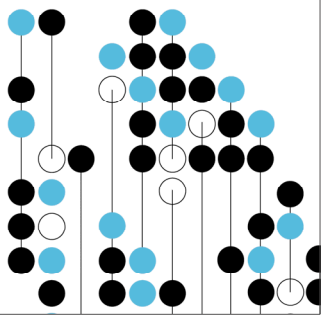
Remember, to make a copy of your work to take away



Please complete your course evaluation



Thank you



QA



Learn. To Change.